CS--1996--14

Gathering and Using Time Measurements in Distributed Systems

Margaret A. Dietz

Department of Computer Science

Duke University

Durham, North Carolina

GATHERING AND USING TIME MEASUREMENTS IN DISTRIBUTED SYSTEMS

by

Margaret A. Dietz

Department of Computer Science
Duke University

Date: _____
Approved:

_____
Carla Schlatter Ellis, Supervisor

_____
C. Frank Starmer

_____
Robert Wagner

_____
Kishor Trivedi

_____
Anselmo Lastra

Dissertation submitted in partial fulfillment of the
requirements for the degree of Doctor of Philosophy
in the Department of Computer Science
in the Graduate School of
Duke University.
1996

Copyright © 1996 by Margaret A. Dietz
All rights reserved

Abstract

It is often necessary to measure time in distributed computer systems. For example, timestamps are used for measuring the intervals between events (like banking transactions). They are also used for ordering events, as in the generation of transaction logs and debug traces. Perhaps their most common use is in performance evaluation, capacity planning, and the detection of unpredicted delays or bottlenecks.

Measuring and using time in a distributed environment presents some unique challenges. This thesis illuminates these problems and presents techniques for the proper measurement and use of time.

The first challenge is the clocks themselves. Chapter 3 describes the tradeoff between keeping clocks synchronized and keeping them accurate. The inaccuracies of networked workstation clocks are examined, and two statistical techniques are described for collecting time measurements in spite of these insufficiencies.

The second challenge is synchronization. There is often a need to order events occurring on several processors, and to measure the time intervals between those events. Chapter 4 is a workload characterization of message-passing in distributed applications. In the course of this work, a method for generating global event traces was developed to get a system-wide view of message traffic. The message traffic was found to have characteristics similar to those observed in network packet traffic.

The third challenge is performance prediction. Programmers need tools to help them understand the interactions between components of a system, and to project the execution time of prospective designs and configurations. Also, operating systems need fast tools to aid scheduling decisions and adaptation of software. We present a technique called *benchmark blending* that combines the speed of analytic models with the flexibility of benchmarks. Benchmark blending weaves time measurements with modeling techniques to create analytic models that: 1) quickly and accurately predict execution time, 2) implicitly incorporate much of the underlying system complexity, and 3) provide programmers with the needed insight to build efficient applications.

We conclude that time measurements are a necessary part of building and studying distributed systems. Their use presents some challenges. This thesis presents solutions for three of those challenges: clock instability, global event descriptions, and performance prediction.

Acknowledgements

This dissertation has been a long, hard journey, with many false starts, detours, and deadends along the way. Several people have had key roles in helping me find the right path.

My first and foremost special thanks goes to my best friend Jackie Darken: my anchor and spiritual healer. I hope our weekly lunch tradition lasts another five years – or perhaps even a lifetime.

Another special thanks to Tyndall Harris for his many roles: colleague, mentor, friend, and confidant. Without his motivational energy at key times, I most certainly would have failed in this effort.

A spirited thanks to Frank Starmer for five billion compliments and bottomless generosity. Frank, you're the greatest!

A genuine thanks to Carla Ellis for the very large personal favor of being my thesis advisor.

A grateful thanks to Ned Danieley and Ed Darken for reading my thesis and providing helpful feedback. Also thanks to Anselmo Lastra for reading this thesis as part of serving on my committee (he says he's going to read it).

And finally, a heartfelt thanks to all the others that have contributed their positive energy to this task: Richard Cooper, Vick Khera, Amy Marshall, Lars Nyland, Karen Sage, Jules Stapff, and Neil Sullivan.

Table of Contents

Abstract v

Acknowledgements vii

Table of Contents ix

List of Figures xii

List of Tables xiii

1 Introduction xiv
1.1 About time xiv
1.2 The use of time in distributed systems 2
1.2.1 Global time constructs 2
1.2.2 Performance evaluation 4
1.3 The challenges of using time 4
1.3.1 Centralized time service 5
1.3.2 Asynchronous clocks 5
1.3.3 The problems with synchronization 5
1.3.4 Clock instability 6
1.3.5 Validating analytic models 6
1.4 Our contributions 7
1.4.1 Clock instability 7
1.4.2 Measurement techniques 7
1.4.3 Time measurements in performance evaluation 8
1.5 Summary and chapter layout 9

2 Background 10
2.1 Clocks 10
2.1.1 Logical clocks 11
2.1.2 Physical clocks 13
2.2 Global traces 18
2.2.1 Definitions 18
2.2.2 Duda's algorithms 19
2.2.3 Ashton's comparisons 20
2.3 Network traffic characteristics 20
2.3.1 Arrival statistics 21
2.3.2 Arrival locality 23

2.4 Benchmarks and composite modeling 24
2.4.1 Benchmarks 24
2.4.2 Modeling overlap 26

3 Clock Instability 28
3.1 Introduction 28
3.1.1 Stability vs synchronization 29
3.1.2 The need for stability 29
3.1.3 The goals of this work 31
3.2 Detecting instability 31
3.2.1 The timechk program 32
3.2.2 Sun clock jitter 32
3.2.3 DEC clock jitter 34
3.2.4 DEC versus Sun clocks 35
3.2.5 Summary 35
3.3 Solutions 36
3.3.1 Turn off synchronization 36
3.3.2 perfclock - a stable software clock 37
3.3.3 Using statistics 38
3.4 Conclusions 40

4 Using Global Traces to Measure Message Traffic Patterns 42
4.1 Introduction 42
4.1.1 Problems in generating traces 43
4.1.2 Network traffic characteristics 44
4.1.3 Arrival locality 45
4.1.4 Summary 46
4.2 Workload & experimental setup 46
4.2.1 Isis overview 46
4.2.2 Definitions 47
4.2.3 The workload programs 47
4.2.4 Hardware and software configuration 49
4.2.5 Run-time configurations 49
4.2.6 Local tracing mechanism - mlogs 50
4.3 Global traces 52
4.3.1 Measuring rate difference (b) 53
4.3.2 Measuring time offset (a) 54
4.3.3 Quality of trace tests 55
4.3.4 Ashton's results 56
4.4 Communication characteristics 56

4.4.1 Traffic characteristics 57
4.4.2 Message arrival locality 62
4.5 Discussion 67

5 Benchmark Blending 69
5.1 Introduction 69
5.2 Benchmark Blending 71
5.2.1 Breaking into components 71
5.2.2 Designing benchmarks 73
5.2.3 Modeling benchmarks 73
5.2.4 Blending 74
5.2.5 Validating the model 75
5.3 Case study: sorting 75
5.3.1 The algorithms and their implementation 76
5.3.2 Building the model 78
5.3.3 Using the model 87
5.4 Conclusion 88

6 Conclusion 90
6.1 Synopsis 90
6.2 Contributions 90
6.3 Future work 91
6.3.1 Improving clocks and traces 91
6.3.2 Message delivery latencies in Isis 92
6.3.3 Distributed sorting 92

References 94

Biography 102

List of Figures

1.1: The timeline of an unstable clock 7
2.1: The printer algorithm in deadlock 13
2.2: SunOS 4.1 system clock 16
2.3: Representations of communication: time-line, cartesian plane 20
2.4: Time-line drawings of IC, OC, IO overlap with 2 channels 28
3.1: The timeline of an unstable clock 31
3.2: The timechk program and post analysis 33
3.3: Offset between Sun 4/65's showing three types of jitter 34
3.4: DECstation jitter 36
3.5: Sun clocks versus DEC clocks 37
3.6: Offsets between perfclock and the system clock 39
4.1: DGS test configuration 51
4.2: Example event record with annotated fields 52
4.3: Clock rate measurements 54
4.4 : The tbench communication pattern 55
4.5: Timing chart and trace files 56
4.6: Cumulative histograms of interdeparture intervals 59
4.7: Message size histograms 61
4.8: Size vs departure time 62
4.9: Persistent messages as a percent of total arrivals 65
4.10: Interarrival time distributions 66
5.1: Ticker-tape processor timings 72
5.2: Two-way merge-split 76
5.3: Odd-Even Sort communication pattern 77
5.4: Odd-Even Sort Functions 79
5.5: Sequential sort benchmark - SSORT(n) 80
5.6: Merge benchmark - MERGE(m) 81
5.7: The communication benchmark - COMM(p, b) 83
5.8: Odd-Even Sort timings 84
5.9: Neighbor Sort timings 85
5.10: 95% Confidence intervals of the prediction 86
5.11: The effects of buffer size on Neighbor Sort 87
5.12: Breakdown of Odd-Even execution time by component 89

List of Tables

4.1: Message passing statistics 57
4.2: Persistent message arrivals 63
5.1: Validation boundaries 87

Chapter 1

Introduction

Margie: *"I finally finished my thesis."*
Carla: *"It's about time!"*
Margie: *"Yes, it is."*

1.1 About time

We take time for granted. It is so pervasive in our lives that we wear watches on our wrists, hang clocks on the walls, and set our workstations to nag us at appointed times. The watches, clocks, and computers need not be precisely correct for these everyday purposes, so we rarely stop to consider their accuracy. We assume, correctly, that they are "good enough." Because of the assumption that our chronometers are "good enough," we sometimes overlook the need to examine their accuracy before making more precise measurements. In many cases they are not sufficient to use without concern for correctness.

Modern computers run at a hundred million instructions per second, or one instruction every 10 nanoseconds. It is difficult to imagine that something running that fast would have trouble counting milliseconds or even microseconds. Surprisingly, many computer clocks are only accurate to 10 or 20 milliseconds [Dan88, Sun90, DEC89, IBM90]. That accuracy is a measure of their self-consistency, i.e., the accuracy of an interval measured with the same clock. Computer clock accuracy with respect to the Universal Time Code (UTC) is considerably worse; system clocks are often off by as much as 5 minutes [Mil90]. As such, we must be careful in how we use these computer clocks to measure events.

This thesis is an exploration of the challenges and uses of time in a distributed computing environment. Distributed or networked programs use multiple independent clocks to measure both universal time and timed intervals. Using multiple clocks severely complicates the accuracy of time measurements. An analogous situation occurs in the timing of races. Road races, car races, and horse races are organized to begin and end at the same place, since this simplifies the time keeping. Some races can not be organized this way (bobsledding, downhill skiing, or white-water kayaking). These events occur in a geographically distributed environment. Timing is done with either the same chronometer (with very long wires attached), or with multiple chronometers that are highly synchronized.

Events in a computer system are logically distributed in the same way some races are geographically distributed. In such systems, the same techniques are used to time events: a central time source or multiple highly synchronized clocks. Both are expensive in terms of overhead, and thus they degrade the performance of the distributed system.

Using dedicated wires to implement a central clock is not only expensive in terms of hardware, but limited by capacitance and the speed of light. Consequently centralized time sources are usually implemented in software as a system service that is accessed over the network via a Remote Procedure Call (RPC). In off-the-shelf systems, or systems that span large distances, a network time service is the only practical method of implementation. Unfortunately, this approach contributes significantly to network traffic, fails to utilize the redundancy of multiple processors, and increases contention for the host processor.

On the other hand, synchronizing independent computer clocks is a difficult process at best [MO85]. Methods for hardware synchronization to UTC (via WWV short-wave broadcasts, or Loran [Mil92] and GPS positioning systems [Cla95]) are not commercially available. Their construction is complicated by variable atmospheric propagation delays and government restrictions. Instead, synchronization is accomplished through the exchange of timestamps among multiples machines over the network. This approach also has its drawbacks; it contributes to network traffic, and inherently decreases the stability (described in Section 1.3.4) of the clocks involved.

Usually, the clocks in a distributed system are neither central nor highly synchronized. This introduces inaccuracies into time measurements. Some of these inaccuracies can be overcome with careful attention to measuring techniques such as repeated sampling and post-processing of timestamps. Before describing these techniques, we discuss the common uses of time measurements in distributed environments.

1.2 The use of time in distributed systems

Time measurements are used in various ways in distributed systems. The uses fall into two general categories: universal-time constructs like event traces and transaction monitoring, and interval measurement for performance evaluation and capacity planning. These uses are described briefly below.

1.2.1 Global time constructs

There are situations that necessitate the construction of a global time reference for events occurring on multiple processors. It is generally the case that chronometers are synchronized with others in the system, but not necessarily to UTC. We use the term global rather than universal, to distinguish between UTC and the common time value over the set of processors in the system. Four examples of global time constructs are:

- event traces,
- transaction logs,
- snapshots of global status,
- causal ordering of message delivery.

1.2.1.1 Event traces

An *event trace* is a list of specific events and the time at which they occurred. Events (such as disk I/O, inter-processor communication, etc.) generate a record in a file local on each processor. The record consists of the event type, a timestamp, and event-specific information such as buffer size or message destination. The separate trace files from each processor can later be merged to generate a global tracefile that serializes all events in the distributed system.

Event traces are a major tool for debugging distributed programs. Most hard-to-find errors are due to rare race conditions that cannot be reproduced on demand. By logging events on each processor and combining them into a global trace, the cause of these rare errors can often be ferreted out [GH93, AG88]. Event traces are also used in quality control and benchmark suites [PA95].

1.2.1.2 Transaction logs

Transactions are a collection of events (typically disk reads and writes) that are viewed as an atomic operation by the system. That is, if one read or write occurs, then the system guarantees that all of them occur. This is accomplished with a *commit protocol* [EGLT76, Gra80, Lam81]. Transaction logs are event logs (or traces) that record the completion of each transaction.

Transaction logs are used for various purposes. They were originally developed to reconstruct index structures after a crash. For instance, if a crash occurs in the middle of a linked list update, the link may be lost or the structure corrupted. Making transactions an atomic, uninterruptable operation fixes this problem. Transaction logs are also used to synchronized multiple copies of a database and to recover from disk failure [Isis91, Sha93].

1.2.1.3 Global status

Sometimes a *snapshot* of the global situation is needed. Air traffic controller systems are an example of this. Information is gathered from multiple radar units, transponders, and database entries. Each of these units has its own clock to produce timestamps. This varied information is used to produce a global picture of air traffic at a particular instant. This necessitates that the clocks be synchronized or their timestamps be reconciled in such a way as to provide the illusion of universal timestamps [BRSS92].

1.2.1.4 Causal ordering

Causal ordering means that if event E_1 on processor P_1 causes an event E_2 on processor P_2, then all processors in the system are guaranteed to see the results of events E_1 and E_2 in the proper order. For example, if E_1 and E_2 are send events, then the messages (M_1 and M_2) are delivered in that specific order at all destinations regardless of network topology. [1]

Some interprocess message systems maintain causal ordering of message delivery. The Isis Distributed Toolkit used in this paper is one such system [ISIS91]. Causal ordering of messages is thought by many to greatly reduce the complexity, development time, and the debugging effort needed to implement distributed programs. The partial global ordering of messages according to causality is accomplished with *logical clocks* [Lam78, Mar84, SES89, BSS90]. Several types of logical clocks are also described in Chapter 2.

---

[1] A more formal definition of causal ordering is given in Chapter Two.

1.2.2 Performance evaluation

In performance evaluation, analysts are generally concerned with the passage of time, or with timing of intervals. Some example problems are:

- isolating (and demonstrating) bottlenecks,
- capturing events that lead to rare failures,
- tuning systems and applications,
- predicting behavior with system improvements.

One of the most common tasks of the performance analyst is isolating bottlenecks in the system and demonstrating their existence to management. This task is accomplished by the ad hoc application of several techniques including event traces, transaction monitoring, and interval timing. Often the advanced constructs (traces and logging) are not available since there are no synchronized clocks. In this case, bottlenecks must be detected through measuring the processing delays on each processor as the transaction progresses through the system. In the Isis workload study in Chapter 4, we demonstrate a technique for generating global traces in an environment with synchronized clocks. This technique greatly aids the isolation of performance bottlenecks.

One cause of rare failure is saturating available resources. This problem can be identified and sometimes predicted by monitoring resource usage. When resource use rises above a certain threshold, either the workload is reduced or more resources are made available. Implementing an on-line monitor of this sort may be difficult due to the amount of data involved. [PB95, Joh95] describe off-line systems to monitor thresholds. When saturation of resources is detected, event traces and transaction logs can be used to identify the events leading to resource saturation.

The monitoring systems described above can also be used to adjust resource allocation to improve performance. Resources (memory, buffer-space, or I/O channels) can be shuffled among processes to reduce paging and I/O latencies. Allocation can often be improved without the expense of constant monitoring through combining time measurements with analytic modeling or simulation. These new hybrid performance models yield results that more accurately reflect system performance than traditional probability models. Chapter 5 describes one technique for building models of this nature.

Performance models are also used for predicting behavior when changes are proposed for the system. The most common scenario is to justify buying more memory or a faster network. Analysts must make the case to management that specific changes will cause specific improvements. This is *capacity planning*. A very large number of industries (insurance, health care, banking, etc.) rely on the ability to predict performance to some degree [Ros95, MDRG95].

1.3 The challenges of using time

There are difficulties in using time in distributed environments that challenge the implementations of the above structures and techniques. Most of these problems stem from the fact that multiple clocks are used. We start by describing the single time service and why it is ineffective as a global time reference.

1.3.1 Centralized time service

As mentioned above, using a centralized time service is unsatisfactory because it increases contention for a single processor (the host of the service), fails to utilize the inherent redundancy of the environment, and increases network traffic. Even worse is the effect of network delays on timestamps obtained from this service. Network delays are neither consistent nor predictable. Waiting for a timestamp suspends outlying processors for longer time periods than closer ones. This seriously perturbs the behavior of concurrent programs, thus invalidating the technique for many purposes (generating traces, logging transactions, timing intervals for performance evaluation).

1.3.2 Asynchronous clocks

Since using a centralized time service is costly, one might consider using local clocks without concern for synchronization. This is a viable solution as long as the items under study (transactions, communication events, service times, etc.) remain on one processor, and that processor is not closely tied to resources on other processors. For instance, a local asynchronous clock would work fine for measuring the service time of the X11 Window Server.

However, with no synchronization effort at all, the individual processor clocks will soon drift apart significantly. This causes problems in several ways: 1) if that processor actively uses files from a distributed file server, then a greatly disparate time value many cause confusion in file timestamps or time-outs; 2) it is difficult to monitor transactions that migrate across processors as the clocks drift apart; 3) the use of multiple asynchronous clocks makes trace generation an extremely complicated process (Chapter 4); and 4) the detection of performance bottlenecks is constrained by the fact that intervals can only be measured within one processor, and cannot be accurately compared with intervals measured on other processors with different clocks.

In summary, using totally asynchronous clocks is useful in examining single processor applications within a distributed environment. However, allowing the clocks to drift apart significantly invites disaster from standard system services (file systems, print queues, shared tape drives, etc.).

1.3.3 The problems with synchronization

Synchronizing processors is a hard chore in and of itself (Chapter 2). It is made hard by network latencies and unpredictability. Consider the following analogy. A first grade teacher is attempting to synchronize her classroom clock. She sends one of the children to the next room to ask what time it is. The child leaves the classroom, stops for a drink, looks at a bulletin-board, and then finally arrives at the other class. After getting the current time, he stops at the restroom, and then at the window in the stairway. Finally, after rescuing a cat that inadvertently got in the school, he returns to his own classroom. How useful is the time value that he delivered?

Message-passing over a network is a similar situation. There are no guarantees or consistency about how long a message takes to travel from source to destination. Computer workstations work around this by requesting the time from several different sources and combining the responses to choose an appropriate adjustment for their local clock. Since even this adjustment is unreliable, the process is repeated at regular intervals every few minutes. This

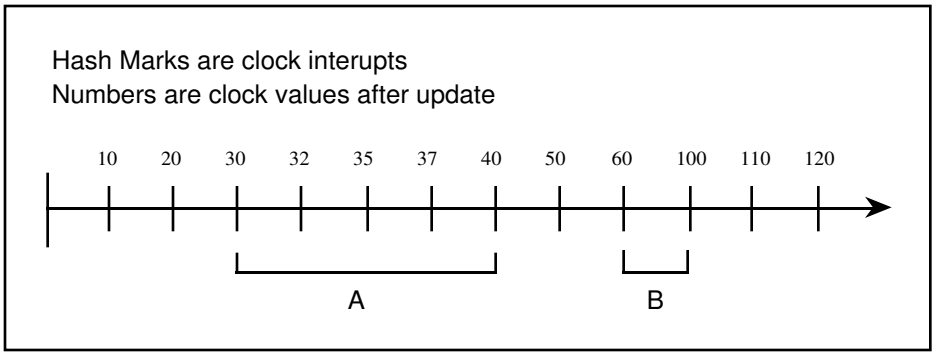


Figure 1.1: The timeline of an unstable clock
A) shows a slowed interval. B) shows a jump.

introduces tremendous instability in the local clocks as well as uncertainty about the degree of synchronization.

1.3.4 Clock instability

To stay synchronized, computers on a network exchange information about the time via a time service protocol and adjust their own clocks accordingly. To move ahead they may add time to the clock, but moving backwards is more difficult. Time cannot step backward, since this would confuse programs that use timestamps for determining precedence (e.g. *make*). To make backward adjustments, the clock rate is slowed down until the desired adjustment is made. Figure 1.1 is an example of the resulting instability. It shows the time values of a clock on a real-time timeline with each hash mark representing a hardware interrupt occurring every ten units. During period A the operating system slows down time to back up the clock. During period B it jumps ahead to catch up. Thus, synchronization destabilizes workstation clocks. Measurements made with these clocks may be grossly inaccurate.

The synchronization process described above causes instability (or uneven ticking) on the local clocks. This instability causes problems in measuring intervals between events occurring on a single processor, using a single clock. Such interval measurement is important for capacity planning, finding performance bottlenecks, etc. It also makes it impossible to guarantee the ordering of events in global traces and transaction logs.

1.3.5 Validating analytic models

When building analytic models of performance, perhaps the most important step is validating the predictions of the model with real observations. Gathering these observations and understanding their limitations are difficult and critical tasks due to the inadequacies of the usual workstation clock. It is first necessary to validate the accuracy of the clock before attempting to calibrate the model. One must consider the accuracy of the underlying validation data before making claims about the accuracy of the model.

1.4 Our contributions

This thesis includes three separate studies that examine and use time measurements in a distributed environment. All three studies are done in a workstation/LAN (Local Area Network) environment. Each workstation on a LAN maintains its own clock. There is no central time service. The clocks do maintain some level of synchrony with each other: sometimes coarse, and sometimes fine. This happens through the time protocol which regulates the exchange of messages about the current time. An example time protocol (NTP) is described briefly in Chapter 2.

1.4.1 Clock instability

The first study examines existing workstation clocks to determine their capabilities and limitations. We describe a pervasive problem: the tradeoff between synchronizing clocks and maintaining their stability. As mentioned above, synchronization destabilizes networked workstation clocks. Measurements made with these clocks may be grossly inaccurate.

This study has two goals: 1) to present techniques for detecting clock instabilities in a LAN environment, and 2) to discuss approaches to stabilizing clocks and collecting data correctly in spite of unstable ones.

Detecting instability is a non-trivial problem because most workstations have no well–behaved reference clock for comparison. It is easy to draw a timeline like Figure 1.1, but to create that precise picture of reality requires a precision time source, plus the software and hardware to interface it to the CPU in a deterministic manner [Mil92]. This study also investigates techniques for detecting instability and estimating accuracy without a precision time source.

1.4.2 Measurement techniques

The second study emphasizes the use of ansynchronous clocks to order events occurring on several processors, and to measure the intervals between events across processors. Chapter 4 is a workload characterization of message-passing in applications that use the Isis Distributed Toolkit. In the course of this work, a method for generating global event traces was developed to get a system-wide view of message traffic. The message traffic was found to have characteristics similar to those observed in network packet traffic.

The Isis Distributed Toolkit is one of several packages designed to ease distributed programming over a network of workstations [BJ87, ISIS91, LLM88, GBDJMS94]. These packages, or toolkits, provide the programmer with high level constructs like message passing, process control, and name service. The Isis Distributed Toolkit offers constructs like process groups and multicasts, which speed application development, while maintaining highly-efficient reliable message delivery across processors. The Isis package is particularly interesting, in that relative timing of events (to capture causal ordering) is significant to the semantics of the system.

In this study, we examined the communication patterns resulting from distributed application programs. We focused the work on actual production Isis applications. Isis is a mature product with more than 75 customers in the financial services, telecommunications, manufacturing, and scientific computing fields [Str93]. Thus we were able to study sophisticated

"real world" applications which were written explicitly to run in a modern distributed programming environment. This knowledge of the user workload is valuable to system designers in optimizing support for heavily used constructs and common communication patterns.

In order to study the communication patterns in Isis applications, we first had to design and implement a global trace mechanism. This job was complicated by clock deficiencies. We found it impossible to create this global construct with the existing clocks, which were unstable due to synchronization. Therefore, we traded complexity for stability, and we implemented global traces with asynchronous clocks. The description of this implementation is at the beginning of Chapter 4.

1.4.3 Time measurements in performance evaluation

Our final contribution is in performance prediction. Programmers need tools to help them understand the interactions between components of a system, and to project the execution time of prospective designs and configurations. Also, operating systems need fast tools to aid scheduling decisions and adaptation of software. We present a hybrid technique called *benchmark blending* that combines the speed of analytic models with the flexibility of benchmarks. Benchmark blending weaves time measurements with modeling techniques to create analytic models that: 1) quickly and accurately predict execution time, 2) implicitly incorporate much of the underlying system complexity, and 3) provide programmers with the needed insight to build efficient applications.

Prediction models used by an operating system must arrive at their prediction quickly. Simulation and benchmark techniques cannot be used to meet these needs since they take far longer than the given time constraints. Validated analytic models are ideal for picking strategies and configuring key parameters at run-time. However, analytic models have two drawbacks when used in this way. First, they have generalized parameters that are difficult to understand by the applications programmer. Second, building accurate analytic models is difficult, time consuming, and thus often beyond the resources of the application programmer.

B*enchmark blending*, the technique presented in Chapter 5, involves five steps. First, the application programs are broken down into large chunks called components. Then synthetic benchmarks are built to measure the performance of these components on the target system. Third, the measured execution time of each synthetic benchmark is described analytically. Next, the analytic descriptions are combined (blended) to form a model that predicts the execution time of the whole original program. Finally, the entire model is validated against existing implementations of the program.

In addition to being fast and accurate, models built with benchmark blending have two advantages over other prediction methods. First, benchmark blending implicitly accounts for the costs and benefits of operating system features (such as caching, paging) that are too complex to model analytically. This makes them much easier to build, and more likely to adapt to new architectures. Second, the parameters chosen for the model more closely match the parameters of the application. This provides better insight into system performance which can lead to faster algorithms and better use of system resources.

1.5 Summary and chapter layout

We conclude that time measurements are a necessary part of building and studying distributed systems. This thesis explores the use of time in this environment, and presents several new techniques for dealing with time measurements including: examining clock properties, measuring intervals with unstable clocks, creating and using global traces from asynchronous clocks, and modeling algorithms with a hybrid technique that combines time measurements with analysis.

The rest of this thesis is organized as follows: Chapter 2 provides background on time synchronization protocols, causal ordering techniques, previous work on global trace generation and performance modeling. Chapter 3 delves into the nitty-gritty of the clock stability issue. Chapter 4 describes the construction of a global trace of distributed events, and then demonstrates the use of this trace with an investigation of message passing attributes of Isis Programs. Chapter 5 describes *benchmark blending* and demonstrates its application by predicting the performance of sorting algorithms. Last, Chapter 6 provides a summary and discusses future work.

Chapter 2

Background

There is considerable literature about time in distributed environments. Here we present four aspects that relate to this thesis. Each are discussed in a separate section. The first section describes work on distributed clocks. It is divided into two subsections: logical clocks, and physical clocks. The physical clock descriptions are necessary background for the clock stability study in the next chapter. The second section discusses techniques for generating global traces. There is little written about this problem. We describe the methods of Duda, et. al., and Paul Ashton in contrast to our own techniques which are used in Chapter 4. The third section is an introduction to network traffic characteristics. It covers three different arrival-rate models, and includes a discussion on temporal locality of message arrival. Traffic characteristics are relevant to the results presented in Chapter 4. The fourth and final section considers several performance prediction techniques that are closely related to the benchmark blending technique presented in Chapter 5.

2.1 Clocks

Leslie Lamport published the fundamental work on distributed clocks in 1978 with his article "Time, Clocks, and the Ordering of Events in a Distributed System" [LAM78]. Lamport made a distinction between logical clocks and physical clocks. *Logical clocks* provide a method for ordering events. They are generally implemented by counters rather than an actual timing mechanism. *Physical clocks* more closely relate to real time. They are expected to progress at a regular rate at some level of accuracy. Physical clocks are implemented with some type of physical oscillator.

Since Lamport's paper, the mechanisms of logical and physical clocks have evolved separately. Their evolution, with respect to this work, is discussed separately.

2.1.1 Logical clocks

Most algorithms use clocks only to determine the order in which events occur. The order of events is sometimes crucially important to the correct behavior of the algorithm. The following example will demonstrate this point.

Say three processors, (P_1, P_2, P_3), cooperatively share a printer. A distributed protocol grants exclusive use of the printer to a processor when it wants to print. This prevents intermingling of the print jobs. Here is the basic operation of our example protocol when P_1 wants to print. First, P_1 broadcasts the message, Request_Printer. The other processors, P_2 and P_3, each respond with the message, OK. When P_1 gets two OK messages, it prints the file and then broadcasts another message, DONE. Meanwhile, the other processors refrain from printing, or sending additional OK messages, until the DONE is received. Thus P_1 has exclusive use of the printer from the time it receives the second OK message, until it sends the DONE message.

The above algorithm works correctly as long as the order of events can be determined. But suppose that P_2 sends a Request_Printer message just before receiving the request from P_1. It now must decide whether to wait for OK messages for its own request, or to respond with an OK to P_1's request. Processor P_1 faces the same decision. If both P_1 and P_2 decide to wait, then the algorithm will deadlock. However, if a consistent ordering of these two request messages can be agreed upon (whether or not that ordering corresponds to real occurrence in time), then printing will proceed in a mutually exclusive fashion.

2.1.1.1 Lamport's logical clocks

Lamport's logical clocks address precisely this problem without the added complexity of relating to real time. We will briefly describe how they work. Define a clock $C_i(a)$, on each processor P_i, that associates a timestamp with each event a. If $C_i(a)$ increments after every event, then the clock orders the events on P_i. This monotonicity is sufficient to determine the order of messages sent from the same processor, but since each processor has its own counter, the order of events across processors can not be determined. To address this problem, two more constraints are added: 1) all messages contain a timestamp, C_s(sent), which is the time on the sender's clock when the message is sent; and 2) when a message is received at P_i, if $C_s(\text{sent}) \geq C_i(\text{recv})$, then C_i is reset as equal to $C_s(\text{sent}) + 1$. This guarantees that a new message sent by P_i will contain a timestamp that indicates it was sent after the message just received from P_s.

Now what happens when logical clock timestamps are added to the shared printer protocol? When P_1 and P_2 must decide on which action to take, they can make a consistent choice by comparing the timestamps in the two messages. If the timestamp on the incoming request predates the outgoing request, then an OK is sent. If the incoming request has a later timestamp, then the procesor waits for incoming OKs. If the timestamps are equal, then the processor number is used to decide which was first.

Note that Lamport's logical clocks define an order over the timestamps, but do not guarantee the order of message delivery. Messages received out of order can still cause confusion in the printing protocol. In the example given in Figure 2.1, P_1 broadcasts a Request_Printer message (timestamped $C_1 = 0$) that is delivered immediately to P_2, but delayed on the way to P_3. Then, P_2 sends an OK ($C_2 = 1$) back to P_1. At this point P_2 sends a Request_Printer message ($C_2 = 3$) expecting to get "in line" to print after P_1 is finished. P_3 responds with an OK to this second

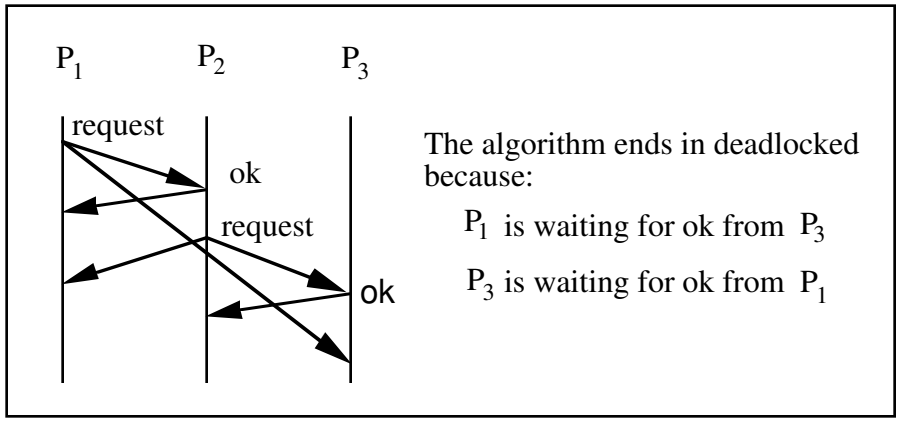


Figure 2.1: The printer algorithm in deadlock

request ($C_2 = 3$), before it even receives the first request from P_1 ($C_1 = 0$). At this point the algorithm is deadlocked since both P_1 and P_2 are waiting for an OK from each other. Timestamps are sufficient to clarify the ordering of send events, but they do not prevent inappropriate responses if the messages are received in different orders.

It is possible to adapt the algorithm to deal with messages received out of order. For instance, if a print request remains unsatisfied for a specific time-out interval, then the request is dropped, and the processor broadcasts a reset message to resynchronize the other processors. However, this too is complicated to implement since the reset messages may also arrive out of order. Dealing with the order of events in a distributed environment is very complex even when using logical clocks.

2.1.1.3 Causal ordering of message delivery

Some systems, like the Isis package [ISIS91] used in Chapter 4, attempt to relieve the complexity of distributed programming by providing a different programming model. The Isis message-passing mechanism guarantees that messages are delivered in *causal order*. Causal ordering means that if event E_1 on processor P_1 causes an event E_2 on processor P_2, then all processors in the system are guaranteed to see the effects (resultant messages) of events E_1 and E_2 in the proper order. The precise definition, taken from [SES89] follows.

Define the relation "happened before", denoted as $\rightarrow$, to be the transitive closure of the relation R. R is defined as:

Two events, E_1 and E_2, are related by R, iff any of the following two conditions are true:

1) E_1 and E_2 are two events occurring in the same process (E_1 before E_2),
2) E_1 corresponds to the sending of a message, *send*(M), from one process, and E_2 corresponds to the reception of the same message, *recv*(M), on any other process.

With the relation $\rightarrow$, we define the casual ordering (denoted $E_1 \overset{c}{\rightarrow} E_2$) of two events E_1=*send*(M_1) and E_2=*send*(M_2), as:

$E_1 \overset{c}{\rightarrow} E_2$ iff (if $E_1 \rightarrow E_2$, then any recipient of both M_1 and M_2 receives M_1 before message M_2).

2.1.1.2 Mattern and Fidge vector clocks

Causal ordering of message delivery is implemented using an extension to logical clocks called *vector clocks*. Vector Clocks are also called Mattern-Fridge Logical Clocks [Fid88, Mat89]. Their use in causal ordering was described by Schiper [SES88], and is implemented with minor variations in Isis [BSS90]. Vector clocks essentially substitute a vector of times for the local counters in logical time. Each clock, VC_i, is a vector of counters with one element for each processor. When a local event occurs at processor *i*, the *i*th entry to $VC_i[i]$ is incremented by one: $VC_i[i]$ is set to $VC_i[i] + 1$.

When P_i receives a message M from P_j, timestamped VC_j, the rule states:

for $j = i$, $VC_i[j] := VC_i[j] + 1$,

for $j \neq i$, $VC_i[j] := \max (VC_i[j], VC_j[j])$.

We then define the relation "<" between vector timestamps of events 1 and 2 as follows:

$VC_1 < VC_2$ iff, $VC_1[i] < VC_2[i]$, for all i.

Using "<", we can say that $E_1 \rightarrow E_2$ iff $VC_1 < VC_2$. And so vector clocks can be used to determine causal relationship between messages, and serve as the basis for an algorithm guaranteeing their delivery order.

2.1.1.4 Summary

The development of logical clocks from timestamps to causal ordering represents the attempt to formalize and guarantee the concept of "happens before" in distributed systems. These techniques are fundamental to fault tolerant distributed programming. They are not however, related to UTC or real-time in general. As such they cannot provide performance data. They also are not used to implement global time constructs (traces or logs) because they define causal ordering, which is weaker than the total ordering, or serialization, needed in these constructs. For these tasks, programmers turn to physical clocks.

2.1.2 Physical clocks

The term *physical clocks* was used by Lamport to distinguish clocks that relate to the concept of physical space-time. These types of clocks are more commonly referred to as system clocks, real-time clocks, or time-of-day clocks. Physical clocks have physical properties like accuracy, resolution, stability, and correctness. This section begins with some formal definitions of these terms.

2.1.2.1 Definitions

Through the rest of this thesis, the term *clock* alone refers to physical clocks. Though Logical Clocks may be used for some of the same purposes, the clocks in this work are all of the physical variety.

In [Mar84] Marzullo provides useful definitions for correctness, accuracy, stability, and synchronization. He defines these concepts in terms of a continuous function, C(t), which maps real time to clock time. *Real time* is solar time; the real space-time that UTC is synchronized with. Since there is no physical reference for real time, the definitions are in terms of a primary standard clock, C(t), and a secondary clock $C_i(t)$. Using these definitions, the terms are defined as follows:

$C_i(t)$ is *correct* at time *t* if $C_i(t) = C(t)$

$C_i(t)$ is *accurate* at time *t* if $\frac{dC_i(t)}{dt} = \frac{dC(t)}{dt} \cong 1$

$C_i(t)$ is *stable* at time *t* if $\frac{d^2C_i(t)}{dt^2} = \frac{d^2C(t)}{dt^2} \cong 0$

A *correct* clock has a value that agrees with the standard. An *accurate* clock is one that ticks at the same rate as the standard. A *stable* clock is one that ticks at a constant frequency. *Accuracy* is often expressed in parts-per-million, or milliseconds-per-hour, drift from the standard. In common usage (and sometimes in this paper) the term accuracy is used to describe both accuracy and correctness. In this case it is expressed as a maximum difference from the standard. A clock with 10 millisecond accuracy will deliver the correct time ± 10 milliseconds.

The *precision* of a clock, or its *resolution*, refers to what degree it resolves time. In the 1980's, computer system clocks generally had a resolution in the milliseconds. Now, ten years later, they usually have microsecond resolution. Note that the accuracy of a clock is not its resolution. Sun Sparcstations have microsecond resolution (the UNIX system call gettimeofday() returns a microsecond value); however, their accuracy is tens of milliseconds.

2.1.2.2 Implementation features

Physical clocks are typically implemented in a computer with a crystal oscillator and a hardware counter. Figure 2.2 shows the layout of the SunOS system clock. In UNIX operating systems, like SunOS, the system clock is used for keeping the time-of-day, performance and accounting functions, interval timers for time-outs, etc.

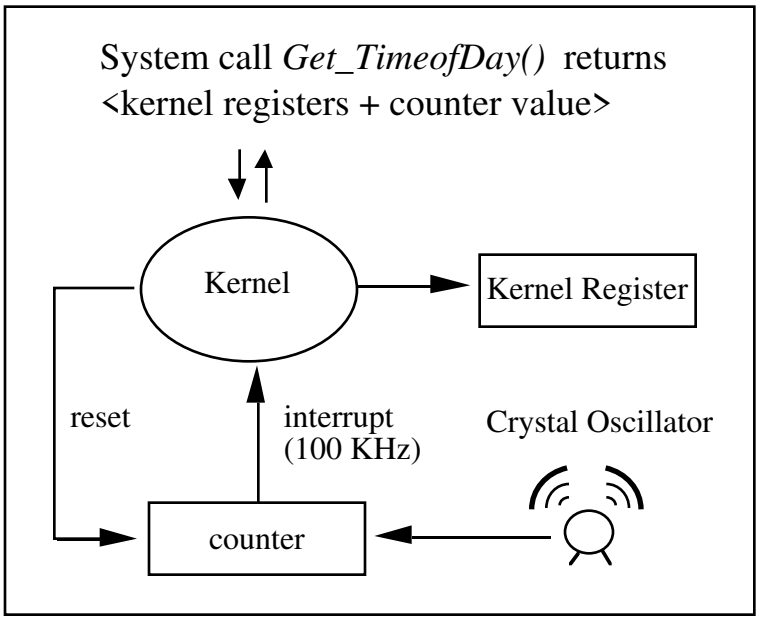


Figure 2.2: SunOS 4.1 system clock

The SunOS clock consists of an oscillator, a hardware counter, and kernel-level software support [Sun93a]. The oscillator generates a reasonably stable periodic signal. The oscillations are counted by the hardware counter which generates an interrupt every 10 milliseconds.[2] The kernel fields this interrupt by updating its internal clock register and resetting the counter. When a system call requests the time, the kernel combines the internal register with the current value of the counter to calculate a time with microsecond resolution.

There are a few sources of clock instability and inaccuracy in this design. First, the interrupts can be missed. This happened in early Sun Sparcstations and was overcome by changing the interrupt signal to be level triggered rather than edge triggered. Second, the oscillator frequency can drift. Oscillator frequency changes with temperature and other environmental factors. Mills measured oscillator accuracy in the range of 0.1 to 1 parts per million [Mil94]. Finally, the kernel software can introduce instability. The instability induced by synchronization efforts is discussed in detail in the next chapter.

2.1.2.3 Hardware synchronization

With all their instability and inaccuracy, free-running system clocks generally have widely different values for time. In distributed environments, there is an attempt to keep the clocks fairly consistent (correct with respect to each other). This can be done in hardware or software.

In [Mil90], Mills describes the limitations of hardware approaches. The U.S. National Institute of Standards and Technology (NIST) broadcasts radio services for the dissemination of standard time [NBS79]. These include short-wave and long-wave transmissions, as well as satellite transmissions. The short-wave transmissions have a time accuracy on the order of a millisecond. The long-wave and satellite (GOES) transmissions can be accurate to 50 microseconds if appropriate averaging techniques, corrections for propagation effects, and ephemeris factors are considered. Other systems are designed primarily for

[2] This is adjustable. 10 msec is the default value

navigation (LORAN-C, OMEGA, and GPS). GPS (Global Positioning System) holds some promise as a viable method of synchronization to 10 millisecond (10^{-8} seconds) accuracy. There are two recent developments in using GPS: an Windows NT utility that interfaces to Trimble and Rockwell GPS receivers [Bea95], and Tom Clark's Totally Accurate Clock project from NASA [Cla95].

Tom Clark's Totally Accurate Clock project interfaces a Motorola Oncore Basic GPS receiver with a small home-built PC board to create a clock with a tenth of a millisecond accuracy.[3] This project suggests the feasibility of equipping all processors in a distributed system with a GPS receiver interfaced directly to the memory bus. The drawback of this approach is economic. An interfaced GPS receiver would add $500-$1000 to the cost of each unit. Therefore, for a while at least, synchronization will be done with software.

2.1.2.4 Software synchronization

The second part of Lamport's paper on clocks [Lam78] defined a system of *physical clocks* that maintained synchrony with each other sufficient to totally order messages. The accuracy of these clocks was based on the minimum possible delay of a message. The problem with Lamport's algorithm is that to maintain synchrony, all the clocks push ahead to run at the rate of the fastest clock in the service. If this clock happens to be inaccurate, then the whole system becomes inaccurate. Worse yet, it could be hard to tell which clock is running fastest.

Marzullo introduced a better concept of a distributed time service in [Mar84]. He described and implemented algorithms to keep a set of clocks synchronized across local and wide area networks. In his algorithms each clock regularly polls the others for a timestamp, and then uses that information to make a best guess at how to adjust the local clock. The algorithms use an overlapping windows approach for determining the best guess, where the size of the windows depends on the estimated accuracy of each timestamp received from other systems. In order to provide Lamport's logical clocks functionality, Marzullo maintained monotonicity by slowing down the clock ticking for a determined period of time rather than making backwards adjustments.

There are now several protocols designed to synchronize systems in LANs. Both Digital Equipment Company (DEC) and IBM use proprietary protocols. DEC uses its Digital Time Synchronization Service (DTSS) [DEC89]. The DAYTIME protocol and the TIME protocol were developed for DARPA networks by Postel [Pos83]. The ICMP and IP Timestamps are specified by their respective standards [Su81]. David Mills developed an integrated approach to synchronizing DCN routers [Mil83]. Mills was also instrumental in the development of NTP (Network Time Protocol) which is the most widely used non-proprietary protocol [Mil89, Mil90, Mil91] . Since NTP is public domain we can describe its general architecture.

NTP categorizes machines into a hierarchy of time references. *Primary* machines are synchronized directly to time sources outside of the network (such as radio). *Secondary* machines synchronize to the primary servers and others in a specifically configured subnet. The

---

[3] Here are Clark's comments on the performance of the TAC:
"At the Haystack meeting I showed some preliminary results from the first two prototype TAC's running against two different Hydrogen Masers. Since that time I did a 2 week long run sampling every 5 minutes. In that period there were ZERO spurious readings, and the RMS fit to a straight line (Maser offset 2.13E-13) was 44.9 nsec on an HP5345 computing counter (which has 2 nsec internal clock)."

subnet is configured dynamically by a spanning tree algorithm using hierarchical level and minimum path delay.

When timestamps arrive at a machine they are filtered to compensate for statistical network latency. They are then combined into the best guess algorithm to determine the estimated current time. The offset between this and the local clock is fed as a correction into the phased-locked oscillator loop (in a hardware clock), or into the kernel time adjustment code (in a software clock).

Using NTP, the accuracy and stability of clocks on the Internet is generally in the tens of millisecond range [Mil90, Mil94]. However, to reach this level of accuracy, NTP must be tuned correctly and allowed to stabilize for one to two weeks.

2.1.2.5 Using low resolution clocks

During the 1980's, the resolution of system clocks was on the order of 10 milliseconds. Peter Danzig addressed the problem of using a clock with insufficient resolution in [DM88]. He demonstrated how to use repetition and statistical methods to report results three significant digits beyond the clock resolution. Later systems generally provide microsecond resolution, but Danzig's technique is still useful when the accuracy of the clock is in the millisecond range.

For example, suppose we wish to measure the overhead added by the UDP/IP protocol stack on a Sun workstation. The overhead is estimated to be 10 milliseconds, but the SunOS clock has only 20 millisecond accuracy. If we treat the clock as though its resolution equals its accuracy (i.e. throw away the bits lower than 20 milliseconds), then Danzig's technique can be applied as follows.

Let Δ be equal to the resolution of the clock. Then the duration of one pass through the protocol stack is δ, which is less than Δ. Note that the clock does not advance during every pass through the stack. In fact, the probability that it will advance is $p = \delta/\Delta$. If the clock advances, we measure the interval as one; if not, we measure it as zero. Let us define the event σ_i to be one if the clock advances during interval i, and zero otherwise. Then after n intervals, the clock has advanced S_n ticks, or

$$S_n = \sum_{i=1}^{n} \sigma_i$$

which yields an average interval length of

$$\delta = p\,\Delta = \frac{S_n\,\Delta}{n}.$$

Now, how many intervals must we measure if we wish to report δ to three decimal places? Danzig approaches this problem by noting that the σ_i's approximate the two possible outcomes of a sequence of Bernoulli trials [Tri82]. It is approximate because the outcomes may not be totally independent. But assuming they are independent, then we can use the Bernoulli distribution to find the minimum number of repetitions, n, that yields a 95% certain value of δ. In his paper,

Danzig uses the DeMoivre-Laplace limit theorem to derive a table of values for n for δ's ranging from 10 to 10^4 microseconds.

2.2 Global traces

As described in Chapter 1, a global trace is a list of events that occurred in a distributed system, which is ordered using the timestamp of a global clock. The primary issue in generating global traces is the absence of a global clock. There are two approaches to solving this problem: better synchronization of the individual local clocks, or the off-line synchronization of individual traces.

To date, most of the effort has gone into clock synchronization, but fine synchronization is proving to be difficult. In addition, the instability induced by synchronization efforts (as discussed in Section 1.3.4) decreases the accuracy of the timestamps in the local traces. This leads us to examine the off-line method. In this approach, separate traces are generated by each individual processor using free-running clocks. We call these *local traces.* By measuring the rate and offset of the individual clocks, local traces can be merged after the fact into a correctly-ordered global trace of events.

In Section 2.2.2 below, we describe the algorithms of Duda, et. al. [DHHB87] for generating global traces. Our own method is described in a later chapter (Section 4.3). Our method is more reliable since it takes advantage of several measurable attributes of the system. Subsequent to our work, Ashton [Ash95] did a comparison of several techniques including Duda's and a method close to ours. Ashton's results are summarized in Section 2.2.3.

2.2.1 Definitions

A *global trace* is a list of events, which are ordered with timestamps from a global clock, $C_g(t)$. A *local trace* is a similar list of events occurring on a single processor, P_i, using the local clock, $C_i(t)$. A global trace of events on multiple processors can be constructed from the respective local traces by computing a correction function, f_i for each P_i, that maps $C_i(t)$ onto $C_g(t)$. Local traces are sometimes referred to as *internal traces,* and global traces are sometimes referred to as *external traces.*

Given a stable local clock, the correction function that maps it onto $C_g(t)$ is linear. That is,

$$\text{if } f_i(C_i(t)) = C_g(t),$$

then there exists an α and β, such that for all t,

$$\alpha_\iota + \beta_i \cdot C_i(t) = C_g(t).$$

If we specify C_g as the reference clock, then the *rate* of C_i is the frequency at which it runs with respect to the reference. For example, if C_i runs 10% faster than C_g, then it has a rate of 1.10, and β_i = 1/rate = 0.9. The *offset* of C_i is the difference between it and the reference clock at a particular instant. Thus at time t_0, if a clock is 5 seconds fast, it has an *offset* of 5 seconds and α_ι = -offset = -5 seconds.

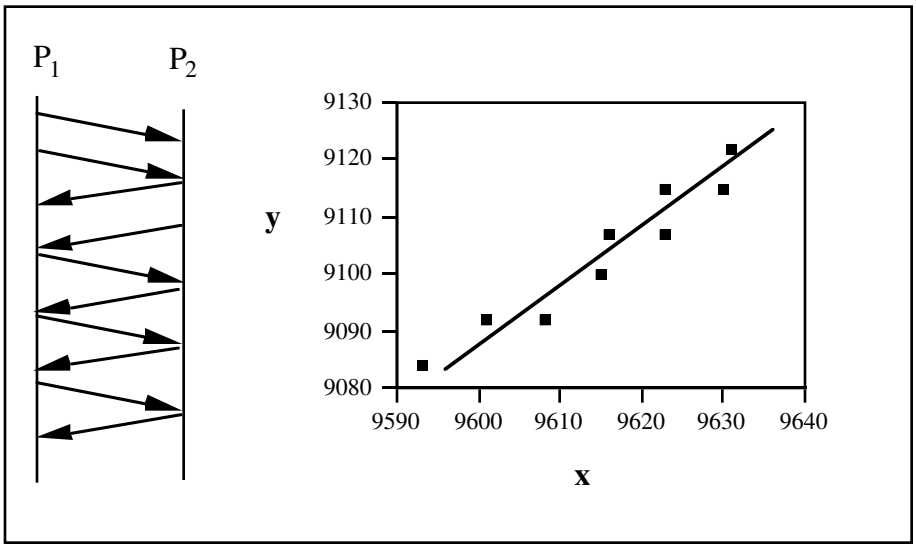


Figure 2.3: Representations of communication: time-line, cartesian plane

The process of generating global traces is: calculate α_i and β_i for each C_i in the system, apply those corrections to the timestamps in each local trace, and merge the traces into a single list ordered by the adjusted timestamps. The works of Duda and Aston address algorithms for computing the α's and β's.

2.2.2 Duda's algorithms

Duda et al. present two algorithms to compute α and β between two processors. Both methods consider a pair of traces that record the send and receive events of messages sent between processors P_1 and P_2. The pair of timestamps associated with each message is taken to be the Cartesian coordinates of a point. Thus message M, sent from P_1 to P_2, has C_1(SEND(M)) as the x-coordinate and C_2(RECV(M)) as the y-coordinate. Similarly message N, sent from P_2 to P_1, has C_1(RECV(N)) as the x-coordinate and C_2(SEND(N)) as the y-coordinate. Figure 2.3 shows how the communication between two processors can be represented on a timeline and on a Cartesian plane.

The first algorithm is the *regression method*. It attempts to find the line $\alpha + \beta t$ such that messages sent by P_1 fall above the line and messages sent by P_2 fall below the line. Note that all messages sent by P_1 include in their y-coordinate the time delay induced by network latency and are thus shifted upward. Similarly, the messages sent by P_2 include network latency in their x-coordinate and are shifted to the right (or effectively downward). Therefore when plotted, the data should appear as two parallel lines. Duda used standard linear regression techniques to find the slope and intercept (β and α) of the line that partitions the set of points in this manner. There must be an equal number of messages sent in both directions for this algorithm to work properly.

The regression algorithm works poorly if the dataset contains points with significant network delays. These points get shifted to areas above the top line and below the bottom line. Their inclusion in the regression analysis can cause the set to be improperly partitioned. Duda's second method was developed to ensure that the set is partitioned correctly.

The second algorithm is the *convex hull method*. In this algorithm two convex hulls are calculated, one for each set of messages in a specific direction. Next, two lines are found that run between the hulls, and are tangent to them. These two lines have the maximum and minimum possible values of β. Finally, α and β are calculated as the slope and intercept of a line that passes through the intersection of these two lines, and bisects the angle between them.

Duda uses simulation to show that the convex-hull method results in far superior estimates of α and β .

In our work, we use neither of Duda's methods because they fail to take advantage of measurable attributes. First, they have an unnecessary degree of freedom since we can calculate β directly from clock rate. Second, we can obtain a close estimate of network delay by measuring the average roundtrip time for messages between the two systems. Our methods (described in Section 4.3) use these measurable attributes to calculate α and β, thus eliminating two sources of error.

2.2.3 Ashton's comparisons

Subsequent to our work [DE95], Ashton published a report [Ash95] that compares six algorithms for computing α and β. He compares Duda's two algorithms and four of his own using traces collected from the Amoeba [Mul84] operating system. Two of the new algorithms, *c.hull* and *c.convex*, are variations of Duda's that incorporate estimates for minimum roundtrip time. The second pair of new algorithms, *2* and *c.2*, are two-point algorithms derived from software synchronization techniques. The *2* algorithm bears close resemblance to the method used in Chapter 4.

Ashton instrumented three well-used programs (*ls, gcc, musbus*) which were running on the Amoeba System. He generated global traces using all six algorithms. Next he defined two measures of goodness of a global trace and compared the traces using these measures. His first measure compared the drift rates, β, calculated for the two machines. Since β should be constant, a good fit will have small changes in drift. His second measure counted the number of messages that appeared to make the trip across the network in time less than the estimated minimum.

Ashton found that *hull, 2, c.2,* and *c.hull* were all good at holding drift rates constant and minimizing the occurrence of underestimated message delay. The *c.hull* algorithm delivered the best performance.

2.3 Network traffic characteristics

A key issue in the design of a communication facility is knowing or anticipating the usage patterns. Much effort has gone into characterizing the arrival patterns of packets on networks. Traffic patterns have a direct effect on protocol design, network interfaces, buffer design, buffer size considerations, congestion control, routing, and process scheduling.

This section examines the existing statistical models of network traffic. Rather than attempt to describe each of the models, we will concentrate on the unique qualities of each and the characteristics they exhibit for different traffic levels. This section also describes the results of a recent (1994) study of WAN traffic by Paxson and Floyd [PF94] that attempts to compare the validity of the different models.

Last, we will consider packet arrival locality. Arrival locality refers to the patterns in packets arriving at a particular processor. Looking at arrival locality provides insight into network-interface and operating system enhancements that improve performance.

2.3.1 Arrival statistics

Good models aid in the design of more efficient networks and provide a way to predict how added resources will affect performance. Currently there are five approaches being considered for modeling TCP/IP traffic:

- Poisson-related models [AMS82, HL86]
- Markov arrival processes [LCW94, Wan95]
- Packet Trains [JR86]
- purely empirical models [DJ95]
- self-similar distributions [LTWW93]

These models predict very different trends in performance for increased number of data sources, increased buffer size, and node congestion. This causes skepticism about their usefulness in the design of future network configurations. Consequently the current thrust is to validate the models more extensively to determine which seems to work best for TCP/IP data.

We will briefly describe the significant attributes of each model and then discuss attempts to validate them against observed LAN and WAN traffic.

2.3.1.1 Models

Early models of TCP/IP traffic were adapted from traditional Markov methods. The Poisson nature of these models gives them a few global features which conflict with observation. First, inter-packet intervals are not memoryless. Packets tend to arrive in bursts or runs. This behavior can be explained by considering bulk transfer programs (like FTP). These programs start up, send bursts of large, evenly-spaced messages, and then terminate. Consequently, their arrival rates are bimodal. Another issue is with the analysis of multiple sources. Poisson models predict that traffic will be smoother with the aggregation of multiple sources. This is the opposite of observation; traffic is observed to become more bursty with additional sources. Finally, Poisson models assume a constant rate of arrival. Constant rates fail to capture the cascade effects of the network news transfer protocol (NNTP) and the simple mail transfer protocol (SMTP) [PF94]. News is transferred from machine to machine using a flooding algorithm. NNTP often spawns secondary connections to offer new articles immediately to a set of peers. SMTP is subject to the explosive effect of large maillists. In addition, these two protocols are often run periodically. Periodic activity is not modeled well by the Poisson distribution.

The *Packet Trains* model, described by Jain, is essentially a two level Poisson model that introduces the concept of *trains* or bursts of packets. The arrival of trains is modeled with one

Poisson rate, and the arrival of *cars* within a train are modeled with a different rate. In [JR86], Jain validated parts of his model with data collected from a token ring network. However, aspects of the data could not be explained without modifications to the simple train theory. To this end, Jain extended the model with Tandem-Trailer Trains (to describe RPC traffic) and the separation of data by source and destination locality. The trains model proved to be difficult to cope with analytically. Very little additional work was done with this model, though Jain's work did motivate the source/destination and locality studies described in the next section. The *trains* concept continues in later work in the guise of *sessions* or *conversations*.

Danzig and Jamin modeled Caceres's characterization of TCP/IP behavior with *tcplib*. Tcplib is an empirical model. It is implemented with a hybrid combination of analytic models and stored data lookup. Arrivals within an interactive session are modeled by using the Poisson distribution. However, non-interactive sessions, the intervals between sessions, and machine generated sessions (like news and mail) are not Poisson. These are represented by stored data and heuristics.

Tcplib reflects observations in packet size as well as arrivals. Caceres, et al., looked closely at packet size as well, challenging the pre-conceived concepts that bulk traffic has different size characteristics than interactive traffic. They found this was not true. However, Caceres did find that short bursts of very large messages account for most of total bytes transferred. This observation has a major impact on congestion control schemes for TCP/IP data. They also found that all traffic (both bulk and interactive) reflected a large number of bursty conversations. Tcplib is useful as a workload simulator, but its hybrid nature makes it useless for analytic application.

The most recent model describes TCP/IP traffic as *self-similar.* Leland, et. al., demonstrated that LAN traffic collected at BellCore over a three year period exhibited many qualities unique to the self-similar distributions described by Mandelbrot [Man83]. Self-similarity is manifested in the absence of a natural length of a burst. At every time scale, bursts consist of bursty subperiods separated by less bursty subperiods. This traffic model predicts several useful characteristics of LAN traffic. It predicts that data from a single source looks much like the aggregate stream of data from multiple sources. It also predicts that source locality is not much of an issue in congestion control, and that the aggregate arrival rate becomes burstier as additional sources are added. Since the self-similar distribution is rigorously defined, this model has more potential for usefulness than purely empirical models.

2.3.1.2 WAN traffic

Paxson and Floyd investigated WAN traffic in [PF94]. Their study included traces from seven WANs. They confirmed Caceres' observation that arrival of user initiated sessions (*rlogin, telnet, ftp*) follow a Poisson model. However, the Poisson rates grievously underestimated burstiness of packet arrival within a session. They observed that Tcplib modeled this burstiness well. Paxson also found that FTPDATA connections (transfer sessions within a single user-initiated FTP) come in big bursts and account for a large percentage of the bytes transferred on the WAN. They suggest that congestion algorithms concentrate on this data source.

Paxson and Floyd also examined the data for self-similar properties. They found that burstiness does get worse as data sources are added to the aggregate stream. They were also surprised to discover that individual source streams do have the same characteristics as the aggregate streams. The FTPDATA spike reflects the prediction that linear increases in buffer

size do not result in large decreases in packet drop rates (contrary to Poisson results), and finally, that the data does look the same regardless of the time scale used to examine it.

2.3.2 Arrival locality

Arrival locality refers to the patterns in packet arrival at a particular processor. Jain (in his "trains" paper) was the first to segregate data in this way for analysis.

Two types of locality have been observed in packets arriving from the network: spatial and temporal. These are analogous, but not identical, to spatial and temporal locality in memory systems. Williamson [Wil89] calls spatial locality in networks *persistence*, and defines it as a packet arriving at a process from the same source process as the previous packet. Persistence is quite high in packet traffic and is exploited to improve performance through 1-behind caching of transport [Fel88] or session level [Wil89] protocol parameters. Temporal locality is a message arriving at a processor within a predictable time interval of the previous message. The existence of temporal locality in message traffic suggests using scheduling techniques to avoid queuing delays.

We now will define the terminology of arrival locality and review the studies that examine its occurrence in distributed systems.

2.3.2.1 Definitions

Since our study of Isis communication in Chapter 4 looks at the process and message level, we define persistence in terms of messages as opposed to packets. *Persistence* is defined as a message arriving at process B from process A when the previous message received at B was also from process A. Note that the messages may be widely separated in time, but are within a single run of the application. Thus persistence, by our definition, spans across bursts of messages. We also extend the definition of persistence to *n-behind persistence* such that a message is n-behind persistent if it is from the same source process as the nth previous message, and it does not exhibit 1 through n-1 behind persistence.

Temporal locality is more difficult to define explicitly. Temporal locality refers to the burstiness of communication patterns. Here is an example using packets. Assume a network where packets tend to arrive in bursts with 1 - 100 millisecond interarrival intervals, and are then followed by a long quiescent period of 10 to 30 seconds. A packet arriving from the same source after 100 milliseconds is considered to exhibit persistence and temporal locality. Such a packet arriving after 10 seconds exhibits only persistence. Messages in distributed applications may exhibit this burstiness as well, and the cutoff interval to distinguish these two attributes is not always intuitive.

2.3.2.2 Observations and exploitation

Mogul [Mog90] traced packets on eleven LANs consisting mostly of workstations, servers, and PCs. He then analyzed the traces on a per-process basis by matching up packets with identical TCP or UDP address tuples, and other explicit process identification. He observed that from one quarter to two thirds of incoming packets are destined for the process that most recently sent a packet. Mogul also observed that a significant fraction of such packets arrive within a few milliseconds of each other. Mogul tried to exploit this temporal locality by dallying, or spin-

waiting, processes in anticipation of the next packet rather than context switching. His simulations showed only minimally performance improvement.

Caceres, et. al. [CDJM91] characterized wide-area network traffic generated from applications that use TCP. He traced TCP packets and separated them into *conversations* using the TCP address tuple and a timeout scheme. Caceres observed high locality of reference (certain hosts communicate more with one another than with other hosts).

In measuring V-system traffic, Cheriton [CW87] found request/response traffic characteristics similar to that observed for conventional terminal and file transfer networks, including a bimodal distribution of packet arrivals. Williamson [Wil89] looked at V-system later for persistence and temporal locality. This was a transport level study (VMTP) of packet events with a workload consisting predominately of file I/O activity. Williamson observed that 71% of packets exhibit persistence, and 93% of interarrival times are less than 1 second. He demonstrated the feasibility of improving performance by caching the state of session level connection protocol.

2.4 Benchmarks and composite modeling

Two concepts heavily influenced the development of the Benchmark Blending technique described in Chapter 5: benchmarks with scalable workloads, and models that account for the overlap of computation and I/O. This review covers the types of benchmarks in current use, two examples of scalable benchmarks, and Hellerman's methods for modeling CPU and I/O overlap [HC75].

2.4.1 Benchmarks

Traditional benchmarks repeat an operation continuously for a known period and then report the number of completed repetitions. The result is expressed as Mflops per second, or transactions per second, etc. These benchmarks are generally useless for several reasons: 1) they do not measure the performance of the anticipated workload (i.e, they are not representative), 2) they ignore any form of concurrency, and 3) vendors are notorious for specifically tuning their systems to perform well running popular benchmarks.

A natural benchmark is constructed by extracting code and data from production software of the operating computer environment. These programs best represent the workload and its intricacies. Unfortunately, they are time consuming to build, usually do not port to new architectures, and often do not take advantage of new configurations. Probably the worst attribute of natural benchmarks is they provide no insight into the performance issues of the system.

Synthetic benchmarks are programs that replicate the instruction mix, and attempt to use resources in the same manner as true workload. They are typically driven by tables that list operations along with frequency of occurrence and resource consumption. This is a good technique because it can provide excellent insight by varying the operations and frequencies. Unfortunately, synthetic benchmarks are time consuming to run. Since their utilization is equivalent to production level software, it is sometimes necessary to duplicate the system hardware and dedicate one system to running benchmarks.

Scalable benchmarks have adjustable workloads based on a parametric description of the actual workload. They are designed this way so they can be used to predict performance on systems larger than what is available. The flexibility of scalable benchmarks, along with the parametric description of the workload, makes them excellent tools for understanding why the system performs as it does. Below we describe two approaches to constructing scalable benchmarks.

2.4.1.1 Chen & Peterson

In [CP91], Chen and Peterson present two contributions: *self-scaling I/O benchmarks*, and *predicted performance*. These two ideas work together to provide a flexible tool for measuring and understanding I/O in parallel systems.

Chen describes a self-scaling I/O benchmark that has significant advantages over the traditional single point benchmark. His method starts with a good workload-based benchmark, and then builds a synthetic program to create different workloads. The five parametric inputs to the synthetic workload are the:

- size of the total data
- average size of an I/O request
- read/write ratio
- number of processors simultaneously issuing I/O
- fraction of requests that sequentially follow the prior request per processor. (Essentially the burstiness of the requests)

During evaluations, Chen's self-scaling benchmark automatically explores the workload space, searching combinations of parameters that result in good performance (defined as 75% of maximum throughput). The benchmark program then generates a graph for each parameter that shows the I/O performance as the parameter is varied with all other parameters held constant. The result is a benchmark that describes throughput using the appropriate workload for an architecture, and provides information on how individual parameters affect performance.

This method has two drawbacks that Chen addresses in the second part of the paper with an idea called performance prediction. One drawback is that architectures can not be compared with each other since the benchmark finds a different workload to measure for each one. The second drawback is that the performance of a specific workload can not be determined. Performance prediction addresses these problems by empirically modeling the behavior of individual parameter graphs, and then combining them linearly to create an overall performance model. Once the model is built, performance of a particular workload is computed algebraically.

Self-scaling benchmarks combined with performance prediction is similar to the benchmark blending technique presented in Chapter 5, except: 1) it does not account for the varying parallelism (it assumes total concurrency), 2) self-scaling benchmarks do not directly measure the specific workload that is needed to configure a particular algorithm on a particular platform, 3) the generalized parameter set contains parameters that are unknown and difficult to measure in distributed algorithms (for example the burstiness and read/write ratio), 4) performance

prediction does not address the combination of the I/O benchmark with other components of an algorithm (such as computation or communication) to generate a global prediction model.

2.4.1.2 Harbitter

In [Har90], Harbitter describes a technique called hybrid benchmarking that combines analytical modeling, atomic-level resource measurement, and synthetic benchmarking. He targets the mainframe world where many performance models exist. His method describes a way to configure a model and validate it using single-point benchmarks, so that it can be used reliably to make procurement decisions.

The steps in Harbitter's hybrid benchmarking are: 1) specify the workload (in terms of transaction type and frequency), 2) specify which model to use, 3) specify resource consumption estimates for each workload transaction, and validate them with single-point benchmarks. 3) Project performance over the range of interest (up to the largest configuration being considered), 4) use synthetic workload benchmarks to validate the model at various points.

Harbitter's technique is a very good method for predicting performance. It is similar to benchmark blending except: 1) it depends on models already built, 2) it is described in terms of transaction style processing which makes it difficult to model compute intensive components (e.g. local sorting), 3) it does not consider the parallelism of the specific task at hand (i.e.: which components can overlap and which can not).

2.4.2 Modeling overlap

In [HC75] Hellerman presents an analytic model of I/O and computation on a traditional batch system. We are interested in this not for the model itself, but for the technique of how to logically think about, and model, overlap. Building effective models of overlapping functions is an inherent problem in performance prediction of distributed applications.

Hellerman begins by enumerating all the permutations of I/O and CPU overlap. For instance, he distinguishes between sequential processing (S) when no overlap occurs; IC-OC processing when either input and compute, or output and compute may take place a the same time; IC-OC-IO processing when any two of the three operations (input, output, compute) can take place simultaneously; and ICO processing where all three can occur at once.

After the overlap states are enumerated, Hellerman draws time lines of each configuration. A time line is drawn for both the I/O bound case and the CPU bound case. In Figure 2.4 the time lines are drawn for the case when there are two channels available, i.e., there is IC, OC, and IO overlap. The busy times are shown as solid lines, idle time is shown as grey bars. The operations on the ith block are labeled with i, while the operations on the next block are labeled $i + 1$, etc.

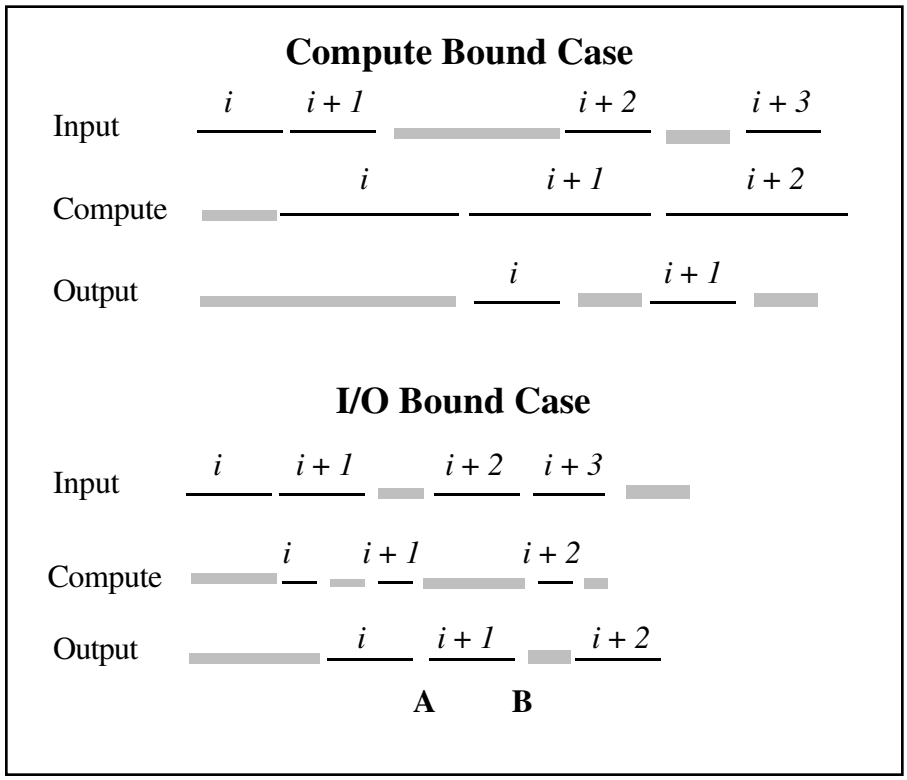


Figure 2.4: Time-line drawings of IC, OC, IO overlap with 2 channels

Solid bars indicate busy times, grey bars indicate idle time. The operations on the *i*th block are labeled with i, the next block is labeled *i* + 1, etc. We assume here that an operation (such as input) takes the same amount of time for all blocks.

The analytic representation of total execution time can now be determined from the drawings. In the compute bound case in Figure 2.4, the time to process a block is simply $t_b = C$, where C is the time it takes to complete the computation operation on a each block. The I/O bound case, however, is more complex and Hellerman finds it necessary to consider two cycles at once. Note that at point A the operations on block *i* are complete, as well as the input operation for block *i* + 1. Between point A and point B there is complete overlap of input with output. Therefore the cost of a cycle in this case is C + 1/2 (I + O). Combining the two cases, the cost of t_b is expressed as:

$$t_b = 1/2\ (C + \text{Max}\ (C, I + O))$$

Hellerman demonstrates that describing the time costs of overlapping operations is sometimes not intuitive, and that drawing a time line often clarifies complex interactions. We use Hellerman's method of drawing time-lines to simplify the task of "blending" benchmarks in Chapter 5.

Chapter 3

Clock Instability

3.1 Introduction

Developing sufficiently accurate clocks is a basic challenge to using time in distributed environments. This study evolved from our attempt to create the global traces described in the next chapter. We were surprised to discover that the clocks on our workstations were dramatically unstable, which made them unsuitable for generating time traces of any kind.

Clock stability is normally taken for granted. The mind's image of a computer clock is a stable crystal oscillator and a mechanism that counts the oscillations. If a clock keeps accurate time over a period of a day, then we assume that intervals smaller than a day are also accurate. In other words, we assume that the clock is ticking with stable even intervals. However, this is not necessarily the case for computer system clocks.

Instability arises from three sources: hardware design flaws, software "bugs", and synchronization (as described in Section 1.3.4). We have observed several problems at the hardware level. The processor can miss the hardware interrupt (as described in Section 2.1.2.2) causing a double length tick, or the oscillator frequency can drift. Oscillator frequency changes with temperature and other environmental factors. On the software level, we have observed the kernel introducing instability on Sparcstations with the interrupt flaw. On these systems, the kernel attempts to compensate by adding 10 milliseconds to the clock at periodic intervals. We have also observed the kernel left in an incorrect state by a terminated NTP program. This caused random 20 millisecond shifts in the clock value. But by far, the majority of clock instability comes from synchronization, due to the constant readjustments of time and clock rate.

3.1.1 Stability vs synchronization

With no synchronization, individual clocks will soon drift apart significantly. This causes problems in several ways. If a processor actively uses files from a distributed file server, then a greatly disparate time value may cause confusion in file timestamps or time-outs. Also, as the clocks drift apart, it is difficult to monitor transactions that migrate across processors. Trace generation is greatly complicated by multiple asynchronous clocks (Chapter 4). Finally, the detection of performance bottlenecks is constrained by the fact that intervals can only be measured within one processor, and cannot be accurately compared with intervals measured on other processors with different clocks.

Providing a centralized time service for these tasks is generally an unsatisfactory solution. A common hardware clock is prohibitively expensive and technically difficult to implement if large distances are involved (Section 2.1.2.3). Accessing a designated "master" clock over the network increases contention for a single processor (the host of the service), fails to utilize the inherent redundancy of the environment, and increases delays due to network latency (Section 1.3.1). Hence, the usual solution is to synchronize the local clocks using a network protocol (Section 2.1.2.4). Unfortunately this solution also induces instability.

Clock instability due to synchronization evolved with the development of distributed time services. Lamport's logical clocks were adopted for establishing the order of events in distributed algorithms [Section 2.1.1]. Logical clocks need not run at a constant rate, but they must increase monotonically. Marzullo implemented a synchronized, distributed time service [Mar84]. In his algorithms, each processor regularly polled the others for a timestamp and then adjusted the local clock accordingly. In order to maintain monotonicity, he slowed the clock rate for short periods of time to make backwards adjustments.

Slowing the clock rate to make backward adjustments is a particularly notable source of instability. The unstable period now lasts for an extended interval, making it more likely that the clock will be used during the period of instability. Any clock that maintains synchrony with others by adjusting the clock rate and time value is unstable to some degree, and it may be unsuitable for measuring accurate intervals. In Section 3.2, we document the cost of maintaining synchrony through software on DEC and Sun workstations. It is common to see time adjustments on the order of a half a second on these machines. This means that errors greater than 25% could be present in the measurement of any interval smaller than two seconds.

3.1.2 The need for stability

Clock stability is not generally a problem for the normal operation of a workstation. Distributed services are designed to expect some uncertainty in remote time values due to communication and scheduling latencies. This is often handled by not guaranteeing the order of events under a certain time threshold.

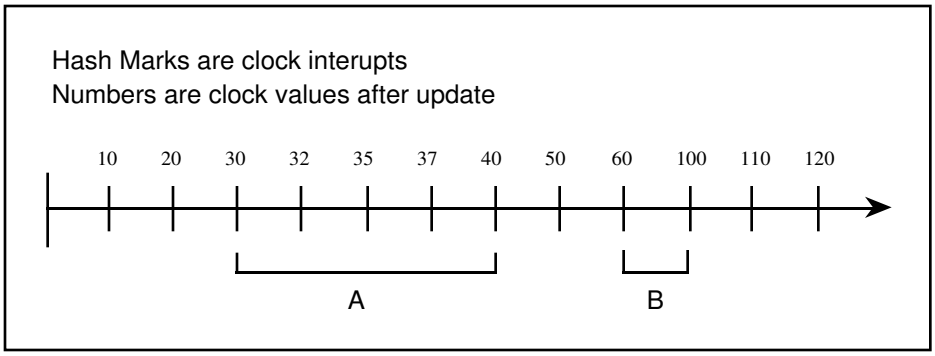


Figure 3.1: The timeline of an unstable clock

A) shows a slowed interval, B) shows a jump

However, there are a few situations where a stable clock is essential. We will present three examples in experimental studies of operating systems:

- performance studies
- workload characterizations
- global traces of distributed events

What happens if an unstable clock is used for interval measurements in a performance study? For example, suppose one wants to measure the time it takes to perform a null remote procedure call [Null-RPC]. A measuring technique often reported in such studies is to take the minimum of repeated calls [Che84, Kar89, Lie83, Mul84, vRvST88, SB89]. Intuitively this method is appropriate because, although the length of a Null-RPC is deterministic, it may trigger other features of the operating system which are not (such as a memory fault). The minimum of several calls equals the time taken by the Null-RPC itself, while the larger values measured include the time needed for additional actions taken by the operating system. This method, however, assumes a stable clock. If the clock is not stable, it can produce an artificially small number which then becomes the only measurement reported. The stability and accuracy of the clock is rarely reported in these papers.

Workload characterizations also need a stable clock. It is common to present histograms of small intervals such as interarrival time of packets, packet service times, file open time, read/write service times, time to acquire a lock, and time spent in idle–loops or at barriers, etc. [BHKSO91, CDJM91, DE95, FE89, HIM91, Mog90]. If the clock used to measure these intervals is unstable, then the distribution (histogram) will be the convolution of the actual intervals and the clock error. In other words, the shape and noise characteristics of the distribution will be altered from that of the actual intervals. When the whole histogram is not reported, often the author reports the range of observations. Consider what would happen if we used the clock in Figure 3.1 to measure an interval with an average length of 20. An individual interval might be recorded as anywhere from 5 (in region A) to 50 (in region B). With a large enough sample size, the average would still be 20, however the range of values would be incorrect due to measurement artifact.

Finally, unstable clocks are unsuitable for generating global event traces. Traces have to be timestamped locally since accessing a remote clock would introduce delays, thus seriously perturbing the events under study. These local traces are then merged into an overall list ordered by the timestamps [DHHB87]. However, if the individual clocks are only stable to a tenth of a second, then the order of events occurring within a tenth of a second cannot be guaranteed. Their

occurrence would have to be considered simultaneous. Though workstation-clock stability is commonly in this range, many investigators have ordered events by time with far more precision. This is a case where stability is more important than synchronization.

3.1.3 The goals of this work

There are two goals to this work: first, to present techniques for detecting clock instabilities in a LAN environment, and second, to discuss approaches to stabilizing clocks and collecting data correctly on unstable clocks.

Detecting instability is difficult because there is no well–behaved reference clock for comparison. Creating a time-line, like the one in Figure 3.1, requires a precision time source, plus the software and hardware to interface it to the CPU in a deterministic manner [Mil92]. However, the stability of a workstation clock can still be examined in a qualitative manner by using the other workstation clocks as a reference. Section 3.2 of this paper describes the poor man's techniques for detecting instability and estimating accuracy without a precision time source.

Section 3.3 describes three approaches to establishing the required functionality for experimental studies of operating systems. Large scale instabilities induced by time synchronization can be removed by disabling synchronization programs in the operating system. We also describe an implementation of a stable performance clock separate from the system clock. Last, we describe two approaches to using repetition and statistics to measure intervals beyond the accuracy of unstable clocks.

3.2 Detecting instability

It is not easy to examine the stability of a computer clock. Ideally we would compare it to a stable reference clock by requesting the time from both sources and comparing the offset (difference) in their values. If the offset is a constant, then the clock is stable.[4] Unfortunately, since hardware clocks are generally not available, the usual approach is to request the time from another computer over the network. The timestamp received from that other system returns through a network with unpredictable latencies. The measured clock offset will be the sum of the actual difference in clocks plus the time it took the message to traverse the network. Normal variation in the network latency will appear to be random noise in the clock offset.

One solution is to examine the *observed offset* (the sum of clock offset and communications costs) between the two systems over a quiescent network with lightly loaded systems. In this case there is a high probability of constant communication latency, so if the observed offset is measured as constant, then the clocks are stable. Note that the opposite is not true; a jittery observed offset does not imply unstable clocks.

---

[4] Actually, a constant offset indicates that the clock is stable and running at the same rate as the reference clock. If it is stable, but running at a different rate then the offset would be increasing or decreasing with a constant slope (2nd derivative equal to zero).

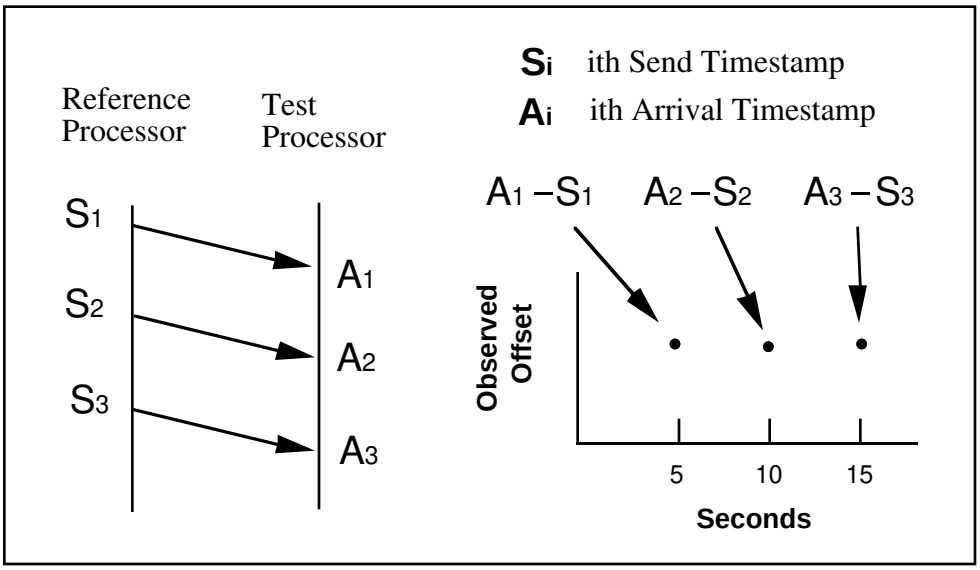


Figure 3.2: The timechk program and post analysis

With this in mind, we built two programs for examining clocks. One uses the Isis Distributed Toolkit [BJ87] which provides an efficient low-level message passing interface. Later we wrote another program in Perl [WS91] to examine the clocks on systems without an Isis license. Using these two programs we examined the clocks on five Sun 4/65 (XLCs) and two Sun 4/25 (Classics) all running SunOS 4.1.3, five DECstation 2100's and one DECstation 5000 all running ULTRIX V4.4.

> The timechk program timestamps when a message is sent using the reference processor clock, and when it arrives using the clock of the processor under test. These timestamps are matched up, subtracted and plotted to make the graphs in Figures 3.3, 3.4, & 3.5.

3.2.1 The *timechk* program

Our main tool for examining clocks is a simple program called *timechk*. It has two modes: master and slave. In master mode, timechk periodically sends a null message to another machine and timestamps the event to a trace file. In slave mode, it timestamps the message arrivals. During post–processing the two timestamps of each message are matched up and subtracted to get the observed offset between machines for each message (see Figure 3.2).

For the results described in this paper, timechk sent a message every five seconds for a period of five or ten minutes. Other experiments done on a much finer scale found that five second intervals caught most instances of instability in the clock while more data points primarily increased post-processing time and cluttered the graphs unnecessarily. Finer intervals were used to verify the *perfclock* described in Section 3.3.2.

3.2.2 Sun clock jitter

Figure 3.3 shows the observed offset from two XLCs. The flat regions on the graph indicate time intervals where the two clocks are stable with respect to each other. It shows three kinds of

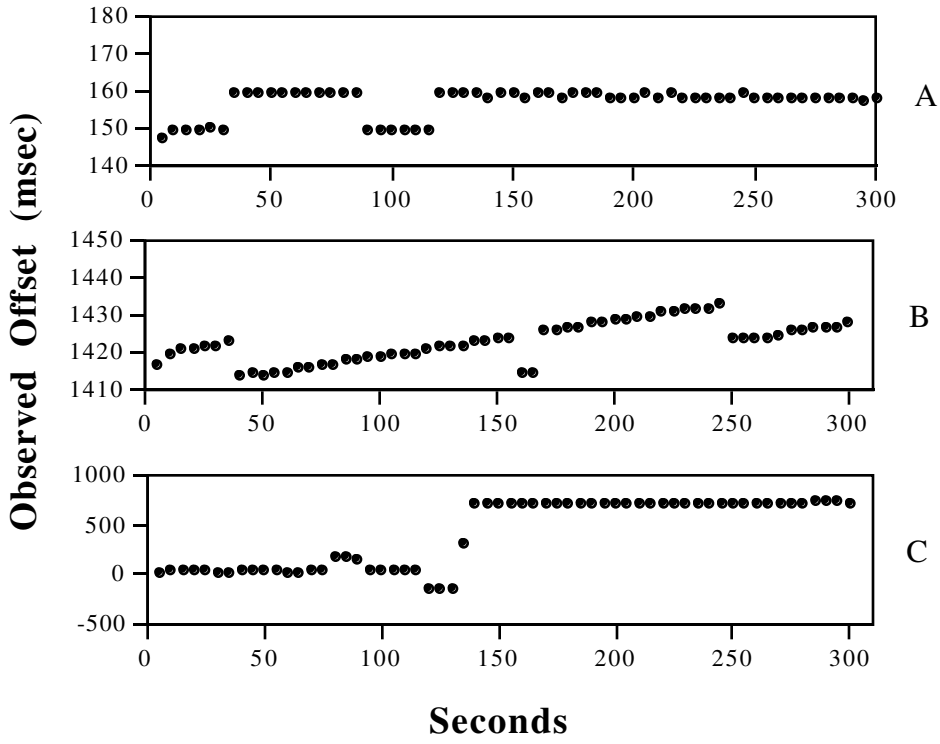


Figure 3.3: Offset between Sun 4/65's showing three types of jitter

Panel A shows 10 millisecond jumps, B shows ramping, and C shows 500 millisecond jumps.

clock jitter at three different frequencies: 10 millisecond jumps, ramping and 500 millisecond jumps.

In this section we explain what we believe causes each type of jitter. To be classified as jitter, the offset must jump to a new level and stay at that level for at least one subsequent data sample. The fact that the offset shift persists for two or more samples suggests that it is due to a clock adjustment rather than network latency. We distinguish between different kinds of jitter by the size of their jumps.

3.2.2.1 Inherent clock jitter

The top panel (A) of Figure 3.3 shows 1 millisecond and 10 millisecond jitter. In this panel there are three 10 millisecond jumps and several 1 millisecond jumps. The 1 millisecond jitter is artifact caused by round-off error in processing the trace files. The 10 millisecond jitter is from the kernel which *attends* the clock only during interrupts from the 100 Hz clock (once every 10 milliseconds). Thus if the kernel is trying to *catch-up* the clock, it adds 20 milliseconds to the time instead of 10 milliseconds.

The middle panel (B) shows the same 10 millisecond jitter and also shows ramping. Ramping is caused by a well known problem with hardware in early Sparcstations. The software misses detection of some of the 100 Hz clock interrupts because they are short pulses rather than

level changes as are used in later motherboards. Several of our XLCs have old motherboards. The middle panel shows the offset between an old and a new XLC. The offset ramps up as the old clock consistently falls behind the new one. The lost interrupt problem caused the clock to lose about 5 milliseconds/minute.

Sun added a software fix for old motherboards (OS4.1.1). The kernel on these machines simply adds the time back in by jumping ahead 10 milliseconds every couple minutes. However, it is important to notice that this clock will consistently underestimate the true length of short intervals. Since the ramping of observed offset is distributed over the whole time-line, it affects almost every time interval measured. However, the adjustments are instantaneous and sparse. To get a correct average value for an interval demands many repetitions to insure that it includes at least one forward adjustment.

The ramping fix accounts for most of the 10 millisecond jitter in the center panel. The Network Time Protocol (NTP) program accounts for the single upward jump and possibly some of the downward ones. From these graphs alone it is not possible to identify for certain the source of a particular jump in observed offset, or even to know which clock made the adjustment.

3.2.2.2 NTP jitter

The bottom panel (C) of Figure 3.3 has a much larger time scale on the y-axis. It shows small jumps of 10 milliseconds, several jumps of 150 milliseconds, and two huge jumps of half a second each. Since there is no ramping, we know that both machines have new motherboards. All three types of jitter in this graph are probably caused by the NTP program in an attempt to keep the clocks synchronized. The huge jumps are particularly damaging in performance studies because they occur nearly instantaneously. Thus a short event could be recorded as half a second long.

3.2.3 DEC clock jitter

In Figure 3.4, we look at DECstation clocks. These are synchronized with the Digital Time Synchronization Service (DTSS) [DEC89]. The top panel (A) has a small amount of 4 millisecond jitter but is otherwise stable. This was the first sample we collected, and it was extremely encouraging. Unfortunately, panel B is the same test on a different day. This time the 4 millisecond jitter is much worse and there are instances of even larger noise (single point jumps). Since the amount of jitter on these machines changed dramatically during the day, we suspected that the 4 millisecond jitter might be network latencies. However, there was no way of testing this suspicion without a reliable reference clock.

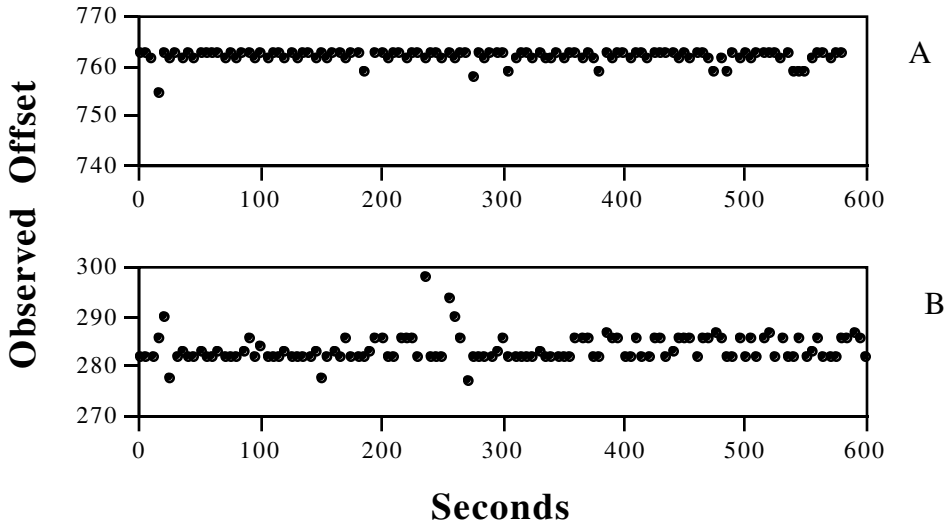


Figure 3.4: DECstation jitter

A and B show 4 millisecond jitter between two DECstation 2100s.

3.2.4 DEC versus Sun clocks

Last, we compare a Sun Classic clock with the DECstation 5000. We felt that this would be useful since they have different jitter levels, so it would be possible to isolate whether sender or receiver was causing an offset. Unfortunately, in our lab the two machines are connected to different LANs, thus network latencies would be even more unpredictable. We picked a time with little network activity and did one quick experiment. Figure 3.5 shows this experiment. To our surprise it showed huge 250 millisecond adjustments and ramping. This jitter seems entirely due to network time synchronization. It is even obvious that the ramping flattens out as the protocols begin to synchronize. We have not been able to explain the one second noise in this graph, and we did not repeat the experiment since we felt there were too many logistical problems to draw any conclusions from it. It is presented here as a demonstration of how inaccurate software clocks can be due to synchronization and other sources of instability.

3.2.5 Summary

In summary, neither the Sparcstations nor the DECstations provided us with a clock known to be stable. The Sun clocks have 10 millisecond jitter due to faulty hardware (in some cases) and NTP. DECstations have 4 millisecond jitter which we suspect might be communications latency, but we cannot know for sure. Sun clocks also have very large discontinuities due to NTP. It is possible that the DECstations display this behavior as well, though it was never observed.

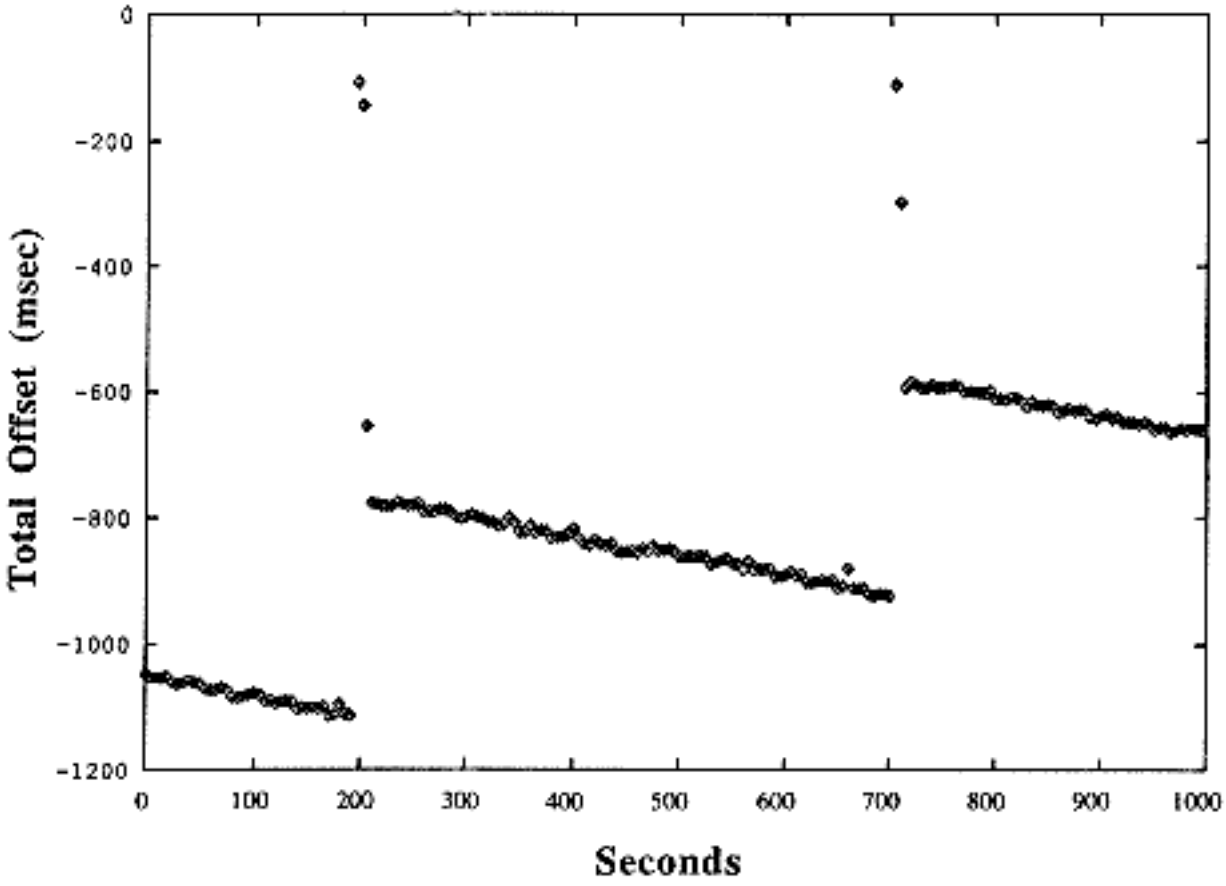


Figure 3.5: Sun clocks versus DEC clocks

3.3 Solutions

Here we describe and discuss three ways to deal with the unstable clocks shown above.

- Disable network time synchronization programs
- Build a separate performance clock
- Use statistics to collect data in spite of instability

These solutions were tried and tested on Sun 4/65's, however the techniques will generalize to other architectures. Each of these methods has merit and limitations. We begin with the simplest – turning off network synchronization.

3.3.1 Turn off synchronization

The quickest, simplest way to increase stability of a Sun workstation clock is to reboot the computer without restarting the NTP deamon. It may not be sufficient to simply kill it because it can leave kernel variables in incorrect states (depending on implementation). For Sun OS4.1.1

and 4.1.3 a rebooting without NTP causes the clock to be stable to the limit of the hardware, though it will be inaccurate.

Though this idea works fine on an isolated system, most of the performance studies done today involve inherently networked environments (distributed file systems, communications, concurrent memory access). Turning off clock synchronization could seriously confuse these distributed services. Even when studying a single system, external resources may be needed to implement the experiments. For example, the only disk space available for trace files may be on a remote system, or the data may be sent to another processor for on-line analysis. Without synchronization, the clock will quickly drift away from the others on the LAN and possibly cause serious inconsistencies in the environment.

It is important to recognize that turning NTP back on will also perturb many of the clocks on the LAN by introducing large corrections. It may take them weeks to stabilize again. This will not cause problems for daily operations, but throws off comparison studies that other users might be doing such as algorithm comparison. Thus turning off time synchronization, though simple to implement, has limited practical use in today's experimental environments.

3.3.2 *perfclock* - a stable software clock

Another solution is to add a second clock designed specifically for performance measurement. This creates a time service designed specifically for measurement of correct intervals, in addition to the system clock, which is available when the correct time is required. Here we describe the construction of *perfclock* in the Sun OS4.1.3 kernel.

Building perfclock involved two changes in the kernel: first, the addition of a new system call (*getperftime*), and second, the replication of the system clock itself in kern_clock.c. Optionally, another system call can be defined to synchronize the perfclock with the system clock. This produces a time service that is not affected by NTP and has the same resolution as the native system clock. This type of addition to the kernel costs little and should be easy on any UNIX variation where the source is available.

We installed the modified kernel on five Sun 4/65's and tested the clock by computing offsets between them pairwise. Again, we can only measure the sum of clock offset and network latency. We found the observed offset to be constant even under induced heavy CPU and network traffic conditions. Network congestion (over 50% of bandwidth usage) resulted in random noise in the offset similar to that seen in the DECstation plots. The 10 millisecond jitter was completely gone.

In Section 3.2.2 we attributed different types of jitter on SunOS clocks to round-off error, lost interrupt corrections, and NTP adjustments. Now that we have developed a stable clock for reference, we ran experiments to confirm these sources of jitter. Each run of an experiment logged timestamps from perfclock and the system clock (on the same machine) every 5 seconds for a period of 10 minutes. There were five runs in each of three environments: a Sparcstation with a new motherboard without NTP running, a Sparcstation with an old motherboard without NTP running, and a Sparcstation with an old motherboard and NTP running. Figure 3.6 shows a representative run in each of these environments which confirms the individual sources of jitter. Note the y-scale differences in the three plots. The top panel shows only 1 millisecond round-off jitter. The middle panel shows round-off jitter, plus 10 millisecond lost interrupt corrections that

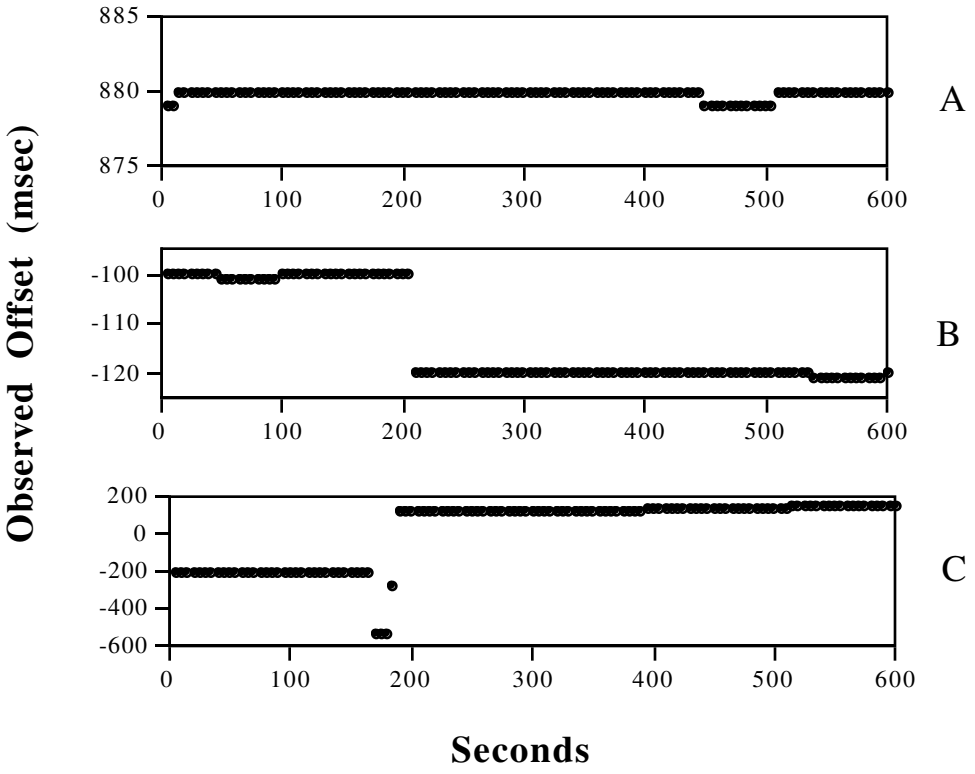


Figure 3.6: Offsets between *perfclock* and the system clock
Panel A is a new motherboard, panel B is an old motherboard, and panel C is an old motherboard with NTP running.

occur with old motherboards. Note that perfclock will experience the same lost interrupt ramping with respect to other systems, but will lack the compensating jumps. The bottom panel displays large scale corrections caused by NTP.

Building a separate kernel clock to measure intervals is a good solution. However, modifying the kernel is not always an option due to unavailable source or political/security issues.

3.3.3 Using statistics

If it is not possible to stabilize the clock, accurate data collection is still possible using statistical methods. We will describe two approaches. In the first approach, the resolution of the timestamps are reduced to the level of accuracy of the clock. Then Peter Danzig's [DM88] technique is applied for collecting data at accuracy levels beyond the clock resolution. The second approach retains the information in the high resolution timestamps. Each timestamp is treated as an estimate of the actual time and the Central Limit Theorem is used to assess the accuracy of the estimate.

3.3.3.1 Applying Danzig's technique

Peter Danzig addressed the problem of using a clock with insufficient resolution in [DM88]. He demonstrated how to use repetition and statistical methods to report results three significant digits beyond the clock resolution. This method is described in Section 2.1.2.5.

Danzig's method can be adapted to handle the situation where resolution is sufficient but accuracy is lacking. To do this, the measured time values are truncated after their smallest accurate digit; then Danzig's method is used to determine how many samples are needed to regain the necessary significant digits. To complete the Null-RPC example above, Danzig's table reports that $4 \cdot 10^4$ intervals must be measured to accurately report the UDP/IP overhead to three significant digits beyond accuracy. Only $4 \cdot 10^2$ intervals are needed for two significant digit accuracy.

3.3.3.2 Central Limit Theorem

This approach retains the information in the high resolution timestamps, and treats each timestamp as an estimate of the actual time. The individual intervals calculated from the timestamps are averaged, and the Central Limit Theorem is used to assess the accuracy of the average.

Consider the clock in Figure 3.1 again. Though measuring small intervals with this clock can be extremely inaccurate, for longer intervals it is quite good. Similarly, workstation clocks are unstable for intervals in the millisecond range, but quite stable over a period of an hour. If the error in a long measurement is small then intuitively the sum of errors in a series of short measurements is also small (given sufficient resolution). Thus if we repeatedly measure a small interval the average will be a good estimate of its value. Now the question is, "What is the quality of the average for a given number of measurements?"

We can answer this by applying the Central Limit Theorem from probability theory. This theorem states that if the observations $\{X_1, X_2, X_3 \ldots X_n\}$ in a sample are independent and have a finite mean, μ, and finite standard deviation, σ, then the distribution of

$$\frac{(X_1 + X_2 + X_3 + \ldots + X_n) - n\mu}{\sigma \sqrt{n}}$$

tends to the standard normal as $n \to \infty$, that is

$$P\left\{\frac{(X_1 + X_2 + X_3 + \ldots + X_n) - n\mu}{\sigma \sqrt{n}} \le a\right\} \to \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{a} e^{-x^2/2}\, dx, \text{ as } n \to \infty.$$

Thus the Central Limit Theorem allows us to collect a large sample[5], compute the mean and variance, and then use the Unit Normal Table to calculate confidence intervals on the mean.

To apply this to our problem, we let $\{X_1, X_2, X_3...X_n\}$ represent our individual interval measurements. Since *n* is finite, we know that the mean and standard deviation of these measurements are also finite. However we are not certain about the other pre-condition, that the measurements are independent. The intervals being measured are independent, but the error in the clock used to measure those intervals may not be.

Remember from Figure 3.1 that forward adjustments to synchronize clocks can be made directly, but backward adjustments are made by slowing the clock rate over an extended period. These slowdown periods can cause dependent errors in interval measurements. If multiple intervals are measured during a single slowdown period, then all the measurements would be underestimated by a similar amount. Fortunately, this dependent error is not large. In NTPD, the maximum adjustment is limited to 512 milliseconds over a minimum period of 16 seconds [NTPD94]. Therefore the clock is slowed by at most 3.2%. Error of the measurement due to slowdown periods would be at most -3.2%. We can use the Central Limit Theorem to compute confidence intervals, if we adjust the lower bound by this amount.

As an example, consider the Null-RPC measurement described in the Introduction. Assume we take 36 measurements. Also assume the mean, μ, of the 36 system calls to be 115 microseconds, with standard deviation, $\sigma = 0.85$. To compute the 95% confidence interval we get 1.960 from the Unit Normal Table and calculate an upper bound of

$$\mu + (1.960 * \sigma)/\sqrt{36} = 115 + (1.960 * 0.85)/6 = 115.3 \text{ microseconds,}$$

and a lower bound of

$$\mu - (1.960 * \sigma)/\sqrt{n} = 115 - (1.960 * 0.85)/6 = 114.7 \text{ microsends.}$$

Now subtract 3.2% of the mean from the lower bound to adjust for possible clock-slowdown error. That makes the lower bound equal to (114.7 - .032 * 115), or 111 microseconds. Finally, we say with 95% confidence that the length of a Null-RPC lies between 111 and 116 microseconds.

Clearly this is an extremely good estimate of the time of a Null-RPC measured in spite of large errors in clock intervals. However, the individual values are still unreliable and have no significance. That is, it is not legitimate to consider the smallest one to be best-case and the largest to be worst-case. Thus, it would be misleading to report the range.

3.4 Conclusions

We have demonstrated a problem with using unstable clocks to measure intervals in performance studies. We have described sources of this instability on Sun and DEC

---

[5] In "The Art of Computer Systems Performance Analysis" Jain defines *large* as greater than 30. Many statisticians accept samples sizes of 10 to 15 as adequate for stable estimates of μ and σ.

workstations on three levels: hardware, kernel, and network time service. We have discussed ways to avoid instability and how to collect accurate data in spite of it. We have described how to build a performance clock in UNIX that is independent of the native system clock. Along the way we have come to several conclusions.

Recent computer clocks are designed for accuracy rather than stability. However, stability is necessary for measuring intervals in performance evaluations. Clocks are overloaded with these two goals. It is simpler and safer to build performance clocks which are completely separate from the time-of-day clock supplied by the system. Then when the time of day is needed the user can use the system clock, and when correct interval timing is needed he or she can use the performance clock. Having two tools to meet the separate needs simplifies the implementations and avoids most of the problems.

The accuracy and stability of clocks used in performance measurements must be established before the study is designed. If the only available clocks are unstable, then a large number of repetitions can still present an accurate estimate of the mean value. In this case individual observations such as the minimum, maximum, and mode may be corrupted by the error in the clock.

Traditionally, we have taken clock stability for granted. But our image of a clock counting off regular intervals is no longer valid. Careful attention to the use of the time-of-day clock is necessary to obtain correct results in performance studies.

Chapter 4

Using Global Traces to Measure Message Traffic Patterns

4.1 Introduction

This chapter describes the generation and use of global event traces to study the time characteristics of message passing in a distributed environment. Knowing the time characteristics can lead to reductions in communication overhead by aiding in the design of efficient protocols, congestion control schemes, buffering structures, and high level constructs. Generation of global traces is necessary to study message traffic because there is no common place to monitor all messages as they traverse the system.

Several packages are available to ease distributed programming over a network of workstations [BJ87, ISIS91, LLM88, GBDJMS94]. These packages, or toolkits, provide the programmer with high level constructs like message passing, process control, and name service. To be effective, these services must be delivered without adding significant overhead to the already costly inter-processor communication facility (typically TCP/IP). Characterization of the communication that occurs in such systems aids in efficient design and implementation.

We are interested in the communication characteristics that result from applications written with distributed programming packages. We focus on production application programs using the Isis Distributed Toolkit. Isis provides high level constructs like process groups and reliable multicast. It is a mature product with customers in the financial services, telecommunications, manufacturing, and scientific computing fields [Str93]. Thus we are able to study sophisticated "real world" applications which were written explicitly to run in a modern distributed environment.

Many studies have characterized network traffic (Section 2.3). In this study we examine communication at the application level rather than the network level. There is rarely one-to-one correspondence between a message sent by the application process and the packets eventually sent over the network. Most systems break down large messages into smaller packets for transit. Similarly, multiple small messages are combined and encapsulated into one large packet. By tracing the messages at the application level, we can be somewhat independent of these architectural features and come closer to capturing the true characteristics of the communication patterns.

To examine message traffic over the network, we need a means of capturing the time of the message "send" operations. That is, we need to timestamp messages before they are fragmented into packets or buffered by the communication layer. Since this processing occurs on the individual processors, globally synchronized event traces are needed to gather data representing the overall pattern.

Generating global traces proved more difficult than expected (see discussion below). Consequently, this study had three goals. We planned to:

- develop a method for generating a global trace of message traffic,
- examine message traffic for characteristics similar to packet traffic,
- determine if messages arriving at a processor exhibit the spatial and temporal locality that packets do.

4.1.1 Problems in generating traces

A global trace is a list of events that occurred in a distributed system; the list is ordered using the timestamp of a global clock. The central issue in generating global traces is the absence of a global clock. There are two solutions to this problem: better synchronization of the individual local clocks, or the off-line synchronization and merging of individual traces.

Fine-grained synchronization has proved to be difficult (Section 2.1.2). In addition, the instability introduced by synchronization efforts (Chapter 3) decreases the accuracy of the timestamps in the local traces. The nature of our questions requires both of these conflicting attributes: synchronized clocks for merging local traces, and stable clocks for studying intervals between events in the trace. This led us to develop an off-line method.

In this approach, separate local traces are generated by each processor using stable, unsynchronized clocks. By measuring the rate and offset of the individual clocks, the timestamps in the files are adjusted to reflect a consistent, or global, time base. The files then are merged into an accurate global trace of events.

Several difficulties arise in the off-line implementation of global traces. It is difficult to measure the offset and rate differences between two clocks because of highly variable delays in message transmission across the network. Also, it is difficult to estimate the round-trip time of a message; roundtrip time is a key factor in computing the offset between two clocks. Additional problems develop with assessing the accuracy of the global traces produced, since no effective metric exists.

In [DHHB87], Duda, et. al. developed two algorithms for tackling these problems. However, his methods addressed a more general case than ours. In our case several system

parameters are directly measurable, and our clocks are known to be stable. In Section 4.3 we describe a method for generating global traces that take advantage of these attributes.

4.1.2 Network traffic characteristics

The design of a communication facility requires predictions of network performance under the estimated traffic load. Traffic models are one of several tools available to make these predictions. Other tools include computer simulation, projections from existing experience, and experimentation. Several models have been proposed to describe network traffic (Section 2.3.1). Two parameters are essential to all traffic models: arrival distribution and packet-size. For general applicability to any choice of model, we limited our characterization to these two parameters.

Arrivals are studied from two perspectives: before the network as messages are readied for transmission, and after the network as messages arrive at the destination processor. The characteristics of packets arriving for transmission aid in network design and congestion control. The characteristics of packets arriving from the network aid in the design of buffers and delivery processing.

The arrival of messages for transmission across the network is represented by a global list of send-message events, or departure times, from the processes. We create a global trace of send events, compute the interdeparture intervals and plot them as a histogram. The interdeparture interval distribution can be used to determine necessary bandwidth of the network. It is also used to estimate the size of router buffers and the effectiveness of congestion control strategies. For example, the mean and variance of interdeparture times reflect the burstiness of the traffic. Very bursty traffic demands larger buffers and reactive congestion control schemes [New94]. Less bursty traffic may need no buffers or congestion control at all.

Both Caceres, et. al. [CDJM91] and Paxon and Floyd [PF94] observed that although some interarrival intervals on a network are Poisson, most are not. In particular, they found that interactive programs seem to generate Poisson arrivals, as does the startup of some programs. However, intra-program traffic and some program startups are distinctly not Poisson. In particular NNTP and SMTP can be both periodic and explosive (packets triggering multiple additional connections and packet transfers). We examine the message interdeparture intervals to determine if they have these same attributes.

We also look at message size. Packets on LANs and WANs have been observed by many to be bimodal, with small packets being attributed to interactive applications and large packets to bulk transfer programs [CDJM91]. This is an important issue in traffic management schemes for providing best-effort service on ATM networks. It has been suggested that a separate FIFO queue may be required at each input port for each class of best-effort traffic to prevent the bulk transfers from interfering with the quality of services of interactive data [New94]. Large packet size is also an issue in ATM loss mechanisms. When overload occurs, a best-effort service will begin to toss out cells. [RF94] has shown that with long packets TCP traffic can drop to 60% efficiency due to delivery of cells belonging to packets that have already had a cell dropped. Several schemes (early discard, fast reservation) have been proposed to deal with this problem.

Given these issues, our general characterization of message traffic in Section 4.4.1 will focus on these three questions:

- What is the distribution of message interdeparture times?
- Is message size bimodal?
- Are there bursts of large packets that would require special buffers?

To answer these questions from our global trace of message send-events, we will plot the interdeparture distribution, the message size distribution, and the departure time versus message size. We will then compare our observations with previously published descriptions of packet arrivals and sizes.

4.1.3 Arrival locality

Two types of locality have been observed in packets arriving from the network: persistence and temporal locality (defined in Section 2.3.2). Persistence is quite high in packet traffic and is exploited to improve performance through 1-behind caching of transport [Fel88] or session level [Wil89] protocol parameters. If persistence is also high in message traffic, the same techniques could improve performance in a distributed toolkit. The transport level state could be cached by providing a custom protocol or by hinting to the system-provided protocol to use caching. Higher level connection management state is generally handled by the toolkit itself, thus caching could be added to it directly.

The existence of temporal locality in message traffic suggests using scheduling techniques to avoid queuing delays. Mogul [Mog90] investigated "dallying" or spin-waiting of processes since another message was expected within a millisecond. With distributed multimedia applications on the rise, time guarantees in scheduling are being investigated [LT91, KG94, AGK94]. If the current workload also exhibits temporal predictability, then these scheduling techniques could improve overall performance of the system.

Persistence and temporal locality have been observed in both LAN and WAN packet traffic, but that does not mean it is inherent to message traffic. Both characteristics may be present due to buffering and fragmentation of messages. When a long message is broken into smaller packets, those packets exhibit both persistence and temporal locality until they are recombined into a single message again. By examining the message level, we will determine if locality is inherent to distributed communication or an artifact of the network implementation. In Section 4.4.2, we will ask these specific questions:

- Is there persistence in *messages* arriving at a process?
- Are there characteristic frequencies in the arrival distribution?

To answer these questions with our Isis workload, we compute the ratio of messages displaying persistence to the total number of messages, and we examine the histogram of interarrival times.

4.1.4 Summary

Section 4.2 describeds the Isis application programs used as the workload and the details of the experimental setup. A brief description is given of the purpose of each application and the overall communication scheme. Section 4.3 describes a method for constructing global traces. Section 4.4 presents the results of the workload characterization, and finally, Section 4.5 discusses our observations and their significance in building distributed communication facilities.

4.2 Workload & experimental setup

The workload was solicited from the Isis user community. Isis was chosen because there is a large, mature user community from which to get production applications. We used all the programs that we received except interactive programs and those for which we could not establish an unbiased dataset. In most cases, the authors supplied the dataset of the program.

Surprisingly, although Isis has more than 75 commercial customers, we had limited (though adequate) success at collecting a workload. Due to the need to maintain customer confidentiality, Isis Distributed Systems, Inc. was not able to supply us with any contacts directly. Therefore, we solicited programs on the Internet. Three commercial sites and several academic sites replied. Only one of the commercial sites was willing to give us their source code. Fortunately, those applications we did receive represented the classic distributed programming problems quite well.

4.2.1 Isis overview

The Isis Distributed Toolkit is a commercial product for developing sophisticated distributed programs. The toolkit runs as a user-domain program on UNIX, VMS, and Mach systems that have the TCP/IP protocol suite. The programmer interface is a library of functions that provide facilities including reliable multicast, a process group mechanism, lightweight threads[6] and a uniform address system, distributed execution primitives, and automatic restart of applications when a computer recovers from a crash. There are many other features and facilities in the Isis Toolkit. Those mentioned here are those used by the applications in this study. Several are described in more detail below.

Messages are sent with a call to *bcast*. In the call, the user specifies the destination and an *entry* (thread) to run when the message is delivered. A destination can be a process, process group, or *site* (processor). On receiving a message, the process creates the specified thread and hands it to the thread scheduler (non-preemptive, FCFS). The message is buffered by Isis and deleted automatically when the receiving thread terminates.

---

[6] Isis calls this a lightweight-task facility and refers to threads within a UNIX process as tasks. We use the term threads to be consistent with common usage.

4.2.2 Definitions

The definitions of site, task, thread, and process are described above in the overview of Isis. We differ from Isis in our usage of the term *multicast*. In Isis all messages are sent with the multicast construct (actually called *bcast*). Broadcasts and unicasts are multicasts with the destination of ALL or a single process. We use the term *multicast* to mean messages with more than one destination. *Unicast* or *point-to-point* is used to describe single destination messages. The term *broadcast* is used sometimes to explicitly point out that the multicast went to all processes.

Persistence and *temporal locality* were defined in Section 2.3.2.1. Three more definitions are necessary. We define the *size* of a message to be the number of bytes in the entire message sent over the network, including headers, padding, and space added to make the data architecture independent. Using a test program, we measured this overhead to be approximately 64 bytes. Interarrival time is the time between messages arriving at a particular process; they need not originate from the same processor or site. The point in time at which the arrival is timestamped is discussed in Section 4.2.6 below. Finally, *interdeparture time* is the time between two consecutive bcast records in the global list of events.

4.2.3 The workload programs

4.2.3.1 Track

Track is part of a prototype track identification system developed at Martin Marietta's Advanced Technology Laboratories. The real application consists of a simulator and processing agents. The simulator reads data out of a file and distributes it to all processing agents in the system. The pieces of this system that we examine in this study are skeletal versions of the simulator (*sim*) and characterization (*char*) processes. Characterization is one of the more compute intensive processing agents. Both pieces were written at Martin Marietta specifically to simulate and analyze the message traffic patterns which occur in the system and, therefore, truly represent the whole system for our purposes.

The identification system works like this: sim starts up and joins a time service. Next, sim begins to give birth to tracks using a simple Markov model. Each track birth generates a notification message to all processes in the system. Some of the char processes send a periodic update request (*PUR*) for the track. Sim receives the PURs and uses the time service to send out position updates at the specified intervals. Deaths of tracks and changes in track-ID are also broadcast to all processes. After the simulation interval is complete, a shutdown message from sim notifies all characterization processes to shut down.

4.2.3.2 Genmap

Genmap [Vau91, YIGBF93] is a distributed implementation of Lathrop and Lalouel's Linkmap, which is a widely used program for mapping the positions of genes on human chromosomes. Linkmap is extremely compute intensive; previous implementations on Cray computers took days and even weeks to compute typical data sets. Genmap was developed in an attempt to reduce compute time and increase robustness.

The application consists of a control process and worker processes. The control process distributes work units to the workers and collects their results. Each worker maintains a work queue. When a work unit is complete, it sends the results to the control process and begins computation on the next unit in the queue. Units are initially statically allocated by the control process. An adaptive load balancing scheme migrates units from slower workers to more productive workers. This strategy was adopted so that Genmap would run efficiently in a multi-user environment.

4.2.3.3 Pmake

Parallel Make (*Pmake*) is a distributed application that performs executions like the UNIX make program. It is driven by a *Makefile* similar to UNIX make, but it distributes the executions over available machines. Pmake is delivered with Isis as a demonstration program [ISIS91], and it is a very useful tool as well.

Pmake runs in two phases: a serial phase, and a parallel phase. The serial phase converts the execution list into a dependency graph stored in a file. The parallel phase initializes the specified number of servers on different processors, and then executes the commands in the graph file. The first server started takes control of the execution list and distributes work to the other servers as they become available. The scheduling of work is dynamic, based on the actual performance of the servers.

4.2.3.4 DGS

DGS (Distributed Graph Storage) is a distributed data-storage service for supporting group collaborations [Sha93]. It was developed as part of the Artifact-Based Collaboration System [SSS92] which supports large groups of people working together to build a large complex structure of ideas called an artifact. Parts of the artifact often become a product such as a software system. The artifact may include concept papers, specification documents, programs, figures, reference manuals, administrative documents, and other kinds of information.

DGS consists of several types of cooperating programs. There may be any number of instances of each program. There is a storage server program, *serv*, which runs on sites that actually store information. All sites that need access to the servers run an interface program, *rgs_svc*, which provides an object interface and caching. Finally, user programs link to a library of routines for communicating with these other pieces. The user level routine that we use is a synthetic browser, *sb*, built to simulate the interactions of several users at a site.

This application is structured in the client-server model. User programs submit requests to the servers and receive a single response. However, since servers hold replicated data, there is additional peer-to-peer communication between the servers to keep each other updated on changes in the data.

Server storage is partitioned by artifact. In our experiment there is one artifact, thus one partition. If an artifact is shared, its partition is replicated in other servers.

4.2.4 Hardware and software configuration

The dedicated test environment consists of five Sun Microsystems 4/65s running SunOS 4.1.3 on an isolated subnet. One machine is configured as a disk server and has 28 Mbytes of memory. The other four run as diskless clients and have 12 Mbytes of memory. All of the necessary SunOS and user files are resident on this server; no remote file systems are mounted. The disk server machine also acts as a gateway to isolate the subnet from outside traffic. Isis v3.0.6 runs on all five sites in the standard configuration with the server as the initial boot site.

4.2.5 Run-time configurations

The workload programs were configured and run as specified by their developers. Where a data set was needed, it was solicited from the developers as "send us a typical data set". This was done to prevent us from unconsciously biasing the results with our own expectations. We tried to get the developers to specify every detail; however, we had to change a few things due to experimental constraints and local hardware configuration. These changes are described below.

The Track program came with instructions to run it with 5000 tracks. The code provided, however, limited the number of tracks to 640. The most tracks we were able to simulate was 200 due to insufficient memory (on a workstation with 28 Mbytes). This did not seem to be a bug, but rather a fragmentation problem due to a lot of threads being active. The Track program was run for 50 seconds with 3 characterization processes.

The Genmap developers sent two data sets. The small dataset (the one that was used in this study) ran approximately fifteen minutes with four workers. The larger data set, which ran for over a day, was not feasible to study due to sheer magnitude of data.

Pmake came with test files to make a small mock Isis application. It ran very quickly and was deemed too small to be realistic. Makefiles for building the Isis distribution were also supplied but found to be obsolete. Instead of modifying the Isis makefiles, we chose to build a locally written software package that involves five libraries and 57 individual programs. This build took Pmake approximately two minutes with five processes.

DGS was configured with two servers sharing an artifact, two graph cache servers, and one synthetic browser (Figure 4.1). Since we only had one disk server in our hardware testbed, we chose to run the DGS servers on diskless clients to provide equal access to the actual disk. The synthetic browser and one graph cache program were run on a third client and configured to read and modify data on the shared artifact. The synthetic browser ran with pre–configured settings for about ten minutes.

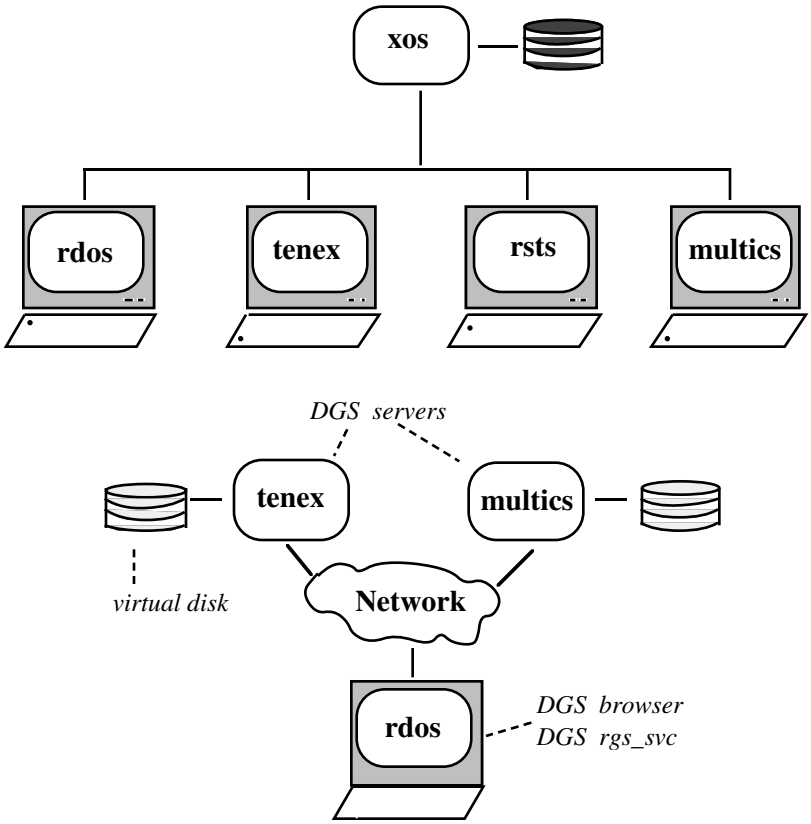


Figure 4.1: DGS test configuration

The actual network configuration (top) and the expected configuration of the DGS application (bottom) showing the mapping of DGS servers to diskless clients to provide the illusion of two disk servers. The labels rdos, tenex, rsts, multics, and xos represent the hostnames of the machines, not the operating systems.

4.2.6 Local tracing mechanism - mlogs

The Isis C libraries were modified to record communication events in trace files. Each programmer-defined process in the application creates a trace file when Isis is initialized. From that point on, the modified library functions automatically record all types of bcasts, replies, packet arrivals at the site, and message deliveries to the application's threads.

In all cases, we record events as close as possible in time to the application program's generation of events. *Bcasts* are recorded when the initial request is made, not when Isis finally sends the message over the network. Message delivery is timestamped and recorded at the instant that the receiving thread is enabled to run. That is the first time the application sees the message and has the opportunity to act on it. Thus our timestamps capture the timing of events as they are actually presented to the application.

Figure 4.2 shows example event records with annotated fields. On bcasts, replies, and message delivery, the event record includes the time, event type, message ID, source site, destination site, destination process, entry number, size, and the number of replies requested. The message id is generated by the trace software for each bcast and embedded in the message. It is a unique number for each message coming from that process. This number is extracted

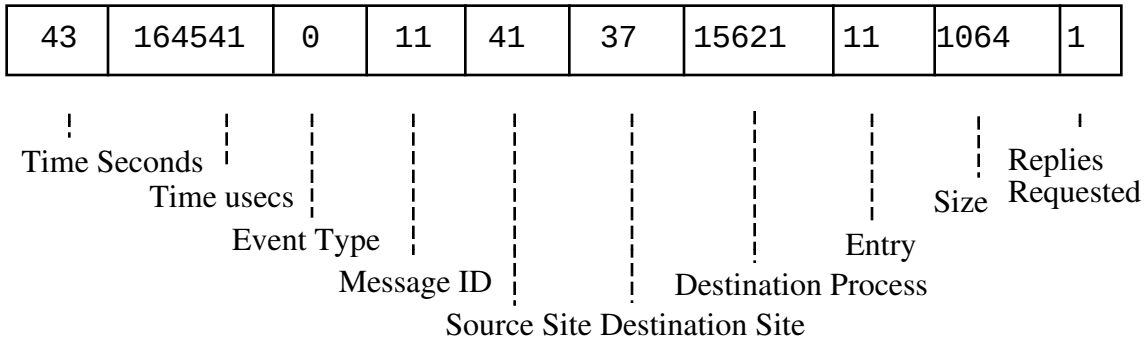


Figure 4.2: Example event record with annotated fields

when the message is received and recorded in the receive event record. This enables us to match up the send and receive events of all messages initiated by the application software.

The event records are stored in a large pre-allocated memory buffer until the application process attempts to exit. At this point the tracing software executes a UNIX sleep for one minute, followed by formatting the memory buffer and writing it to disk. We postpone processing of the tracefile in this manner to minimize perturbation of the application. To measure overhead, we compared the runtime of the applications without tracing code to their runtime with tracing code (but without the one minute delay and post processing). The runtimes differed by less than 1%.

Each program was run ten times with a timing program, *tbench*, run before and after the sequence of ten runs. The tbench program generates message bursts among all the precessors on the network. Site A sends a burst of 10 roundtrip messages to B, C, D, and E sequentially; then site B sends bursts to C, D, and E; and the pattern continues until all pairwise combinations of sites are included. The timestamps from these messages are used to synchronize the trace files and obtain a global trace. *Tbench* is discussed more in the next section.

The clock used for timestamps is a modification of the SunOS kernel that directly accesses the high resolution clock register on Sparcstations [DES94]. It provides microsecond resolution and has an estimated accuracy of 1 part per million. Since the longest run-time of any application in our workload is 15 minutes, the maximum clock drift for a single run is less than 1 millisecond.

All Isis internal messages[7] were removed. These were detected as messages received from unknown processes, messages sent to unknown processes, and messages with no embedded message id. Most of these occurred at the start of the trace and were probably due to Isis initialization. We were able to match up all the send and receive timestamps for remaining messages except some in the Track program. Further investigation showed that some of the Track processes terminate before reading all the messages sent to them by other processes in the system. These messages were left in the trace files for analysis wherever possible.

The last step in preparing trace files was the removal of startup and shutdown messages that are artifacts of the experiment rather than the application itself. Pmake and Genmap were

[7] Internal messages are generated to resolve names, to detect failed processors (during communication lulls), and for communication with higher level facilities like the transaction server and log server.

designed to run as single instances, so all of their messages were included in the file trace. The DGS program was designed to have the servers configured and running before a user logs in, therefore all the initial setup messages were deleted. This consisted of setting up the database, loading it with data, and configuring the sharing of data. The Track program also was designed to be running a priori; therefore, all the startup messages for characterization processes were removed though the startup of sim and all initial group joins were left in the trace file.

4.3 Global traces

A global trace is an ordered list of timestamped records that mark the occurrence of events in a distributed environment. It is created in a two-step process. First, local traces are generated by each processor, and then the local traces are merged into a single ordered list. The two-step process is necessary to minimize the effects of trace generation on the system. When an event occurs, the current time and other parameters are stored in local memory. After the program terminates, the data in memory is written to disk. By using a local clock for timestamps and deferring the disk write, the behavior of the system is only minimally affected by trace generation.

The second step of generating a global trace, the merge step, demands that the timestamps on the various local traces be synchronized to within the accuracy needed for the trace. That is, if two events occurred simultaneously on two different processors, then the difference between the timestamps of those two events should be less than the accuracy of the trace. This can be done in two ways: by synchronizing the clocks on the local processors or by adjusting the timestamps off-line (after the fact). We showed in the last chapter that clock synchronization causes clock instability. Because of this, the off-line method offers greater resolution and accuracy.

Off-line synchronization methods begin by defining $C_g(t)$ be the clock of the finished global trace. Then the timestamps in the local tracefiles are adjusted to be synchronized with $C_g(t)$. Since the local clocks are stable we know that any of one of them, $C_i(t)$, can be mapped onto the global clock with a linear function. That is, for each processor P_i, there exists an α_i and β_i, such that

$$\alpha_i + \beta_i \cdot C_i(t) = C_g(t).$$

In effect, β adjusts the rate of the local clock and α corrects for its offset with respect to $C_g(t)$.

Duda [DHHB87] presented two approaches to calculating α and β: the regression method and the convex hull method. These are described in Section 2.2.2. Both methods are needlessly complex for our purposes because they estimate both α and β. Except for gross changes in room temperature, our clocks run at a constant rate. Therefore we can measure the relative rates of the clocks and compute β directly. Once β is known, estimating α is simplified.

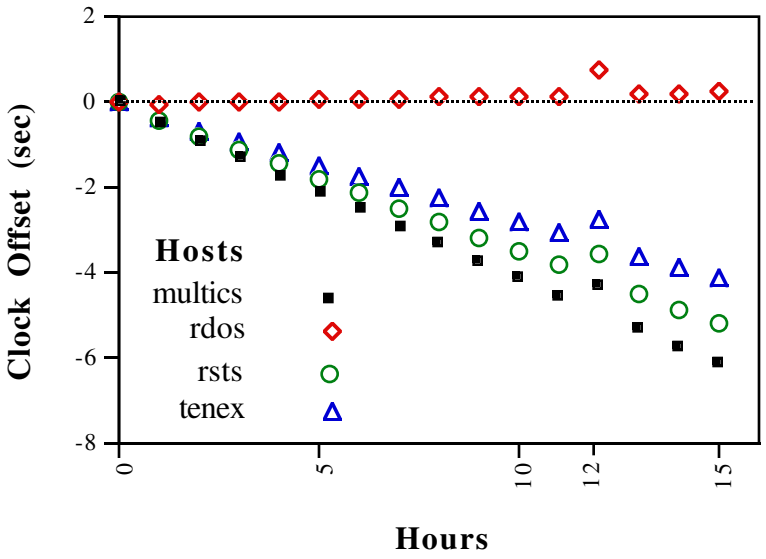


Figure 4.3: Clock rate measurements

The point at 12 hours is an anomaly due to a delay in transmission of the message from the master machine. This results in a positive offset jump of the same magnitude on all four slave processors.

4.3.1 Measuring rate difference (β)

To measure the rate differences between the various clocks, we use the *timechk* program described in Chapter 3. Timechk has two modes: master and slave. The master mode is run on one machine, and slave mode is run on all the others. The master sends out a broadcast and the slaves receive it. The send event is logged with a timestamp on the master, and the receive events are logged on the slaves. In post processing, the send and receive events for each message are matched up and subtracted to get the observed offset.

We chose one machine, *xos*, to be master and compared the clocks on the other four machines to its clock. A typical experiment produced a plot like that shown in Figure 4.3. This experiment ran 15 hours with timechk broadcasting once per hour. The observed offsets of each clock at the beginning of the experiment are subtracted so that they all start out with zero offset. Of the four clocks, *rdos* (shown by diamonds) ran slightly faster than the reference clock (*xos* shown by a dotted line), while the other three ran slower. A linear regression fit determined β for each clock.

Experiments like the one shown in Figure 4.3 were done at two resolutions: with timechk broadcasting one point per hour for up to 5 days, and with timechk broadcasting one point every 5 seconds for 5 minutes. The slopes of all experiments were consistent, with very little variability as indicated by the R values of the linear regression fit.

If we did not have clock stability over the long run, then Duda's algorithms would be helpful. That is, without clock stability, the rate is not a constant and our method would be inexact. In that case, estimating the clock rate over a short run of data would give better results.

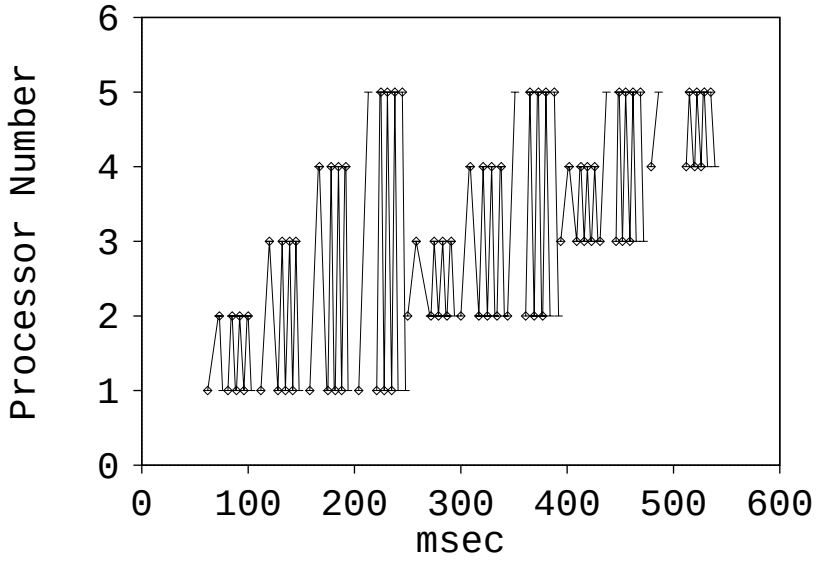


Figure 4.4 : The *tbench* communication pattern

Figure 4.4 shows an example of the global trace produced by the *tbench* program. Each line segment in this plot represents a message being sent. The end with the symbol is the origin of the message, and the end with a plain hash mark is the destination. Some messages take longer to traverse the network and appear slanted in this plot. We show this plot to clarify the communication pattern and to demonstrate the time variances experienced by the tbench program. But we still have not described how to compute the offset in order to generate a global trace like this one.

4.3.2 Measuring time offset (α)

The next step is to compute the offset. To do this we wrote the *tbench* program. In tbench each pair of processors exchange messages in a remote procedure call pattern. Processor A sends a message to B, and B responds by immediately sending a message back to A. This is repeated a number of times (typically 10 or 50) for each pair of processors.

Figure 4.5 shows a single roundtrip communication between P_0 and P_1. This communication is recorded in the local trace logs shown at the bottom of the figure. To compute an offset, we assume that a message traversing the network will take the same amount of time in both directions. This is a good assumption for machines on a LAN connected by an ethernet. We then estimate $C_0(t_1)$ as the midpoint between $C_0(t_0)$ and $C_0(t_2)$, or

$$C_0(t_1) \approx \frac{C_0(t_0) + C_0(t_2)}{2} \ ,$$

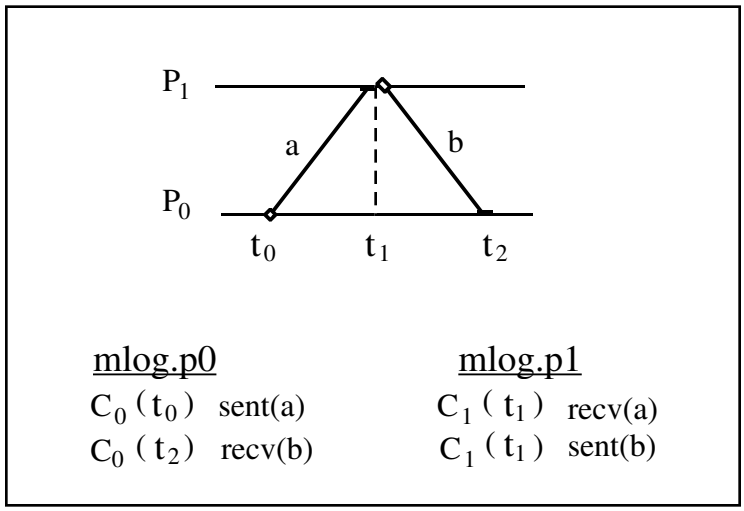


Figure 4.5: Timing chart and trace files

and finally compute α_1 as

$$\alpha_1 = C_0(t_1) - C_1(t_1).$$

The error in this method originates from the estimate of $C_0(t_1)$ which is based on the equal network traversal time assumption and the roundtrip time. With separate free running clocks, there is no way to determine if the equal traversal time assumption is true. However, roundtrip time is measured with a single clock, so we can take care to eliminate as much uncertainty there as possible. Logically, the smallest roundtrip time evidenced in our log would have the least amount of extraneous processing factored in. Therefore, we calculate α using the pair of messages that exhibit the smallest roundtrip time (approximate 6 milliseconds).

4.3.3 Quality of trace tests

We used two criteria to check the quality of our estimates of the α's and β's. First we checked that the global trace maintained the known precedence of events. That is, we checked to be sure that the send event for a particular message preceded the corresponding receive event. All messages with both send and receive timestamps passed this test. Second, we examined the traversal times of messages. Since we have an homogenous environment, with identical CPU's connected by ethernet, the average traversal time of a fixed-size message should be constant between any pair of processors. We checked this by plotting a histogram of the traversal times and examining the peaks. A single peak indicates that the timestamp offsets are correct. Multiple peaks indicate that several different modes of traversal time exist, which is a clue that the offsets are incorrect.

Both of these quality of trace measures are weak. We would like to use something better. Unfortunately, without a universal reference clock, our options are limited. Also, we lack a good way to estimate the accuracy of our global traces. The precedence check demonstrates that the accuracy is at least one half of a roundtrip time, or 3 milliseconds. The histogram of message traversal times suggests that it is better than that. Since the peaks vary only $\pm$ 1 msec, we estimate the accuracy of the traces as approximately 1 msec.

4.3.4 Ashton's results

Subsequent to this work, Ashton compared five algorithms for estimating α's and β's. One of his methods, the 2-point algorithm, uses an approach similar to ours. Our method works well because we can: 1) measure β directly, and 2), match up specific messages involved in a send-receive sequence produced by tbench. The 2-point algorithm is essentially the same except that it estimates both α and β, and it is not necessary to match up specific send and receive messages.

Ashton's 2-point algorithm uses a pair of messages, one from A to B and one from B to A, to compute what he calls *equivalences*. An equivalence is a pair ($a(m_i, m_j)$, $b(m_i, m_j)$) where $a(m_i, m_j)$ is the average of the two timestamps recorded by A for the two messages and $b(m_i, m_j)$ is the average of the two B timestamps. Ashton uses two equivalences to estimate β as

$$\beta = \frac{b(m_i, m_j) - b(m_x, m_y)}{a(m_i, m_j) - a(m_x, m_y)} .$$

Once β is determined, an equivalence based on some message pair (m_p, m_q) can be used to calculate α with

$$\alpha = b(m_p, m_q) - \beta \cdot a(m_p, m_q).$$

Picking appropriate message pairs to compute equivalences is key to the accuracy of the 2-point algorithm. Ashton found, as we did, that it is important to pick pairs exhibiting minimum roundtrip delay.

In his paper, Ashton compared his 2-point algorithm with Duda's two methods. He then enhanced all three algorithms by computing an accurate minimum roundtrip time estimate via an empirical model. His comparison criteria included precedence checking, checking the constancy of β, and counting the instance of messages traversing the network in less time than his estimated minimum. The 2-point method ranked better than either of Duda's original methods, but not as good as the model-enhanced versions of the 2-point method and the Convex Hull method.

Ashton's 2-point algorithm is essentially equivalent to our method, although it is generalized to compute β and uses unmatched messages. Therefore, our method performs at least as well as the 2-point method by some of Ashton's criteria. In fact, it would probably perform close to the enhanced 2-point algorithm since we use the minimum roundtrip pair for our estimation. This gives us additional confidence in our global traces.

4.4 Communication characteristics

This section presents the results of our measurements of the Isis workload. We examine the message-passing characteristics of the applications from two perspectives: globally as messages are readied for transmission, and per-process as messages arrive at their destination. Examining messages from the global perspective is analogous to examining packets traversing the network. Therefore we compare our observations with those of several network traffic studies [AMS82, HL86, JR86, DJ95, LTWW93]. The per-process perspective examines the arrival of messages at

a particular process. From this perspective we look for the persistence and temporal locality similar to packet arrivals described by Williamson [Wil89] and Feldmeier [Fel88, Fel94].

4.4.1 Traffic characteristics

Network traffic characterizations are used in the design of network hardware, congestion-control protocols, routing protocols, datalink level buffering, session level buffering, and operating system interfaces. We wondered if message traffic (at the application level) exhibits the same attributes as packet traffic (at the network level). Most systems split large messages into smaller packets for transit, and also combine multiple small messages to make a larger packets. Therefore the characteristics of packet traffic in some ways reflects the architecture of the network implementation. By examining the message traffic, we come closer to capturing the true characteristics of the communication patterns.

Section 2.3.1 describes several approaches to modeling traffic characteristics. We consider two measures of traffic that are common to all the models: interarrival times, and size. Our specific questions are targeted toward the characteristics of packet traffic observed by [CDJM91, PF94]. We wonder if these characteristics occur in the message traffic as well. Our questions are:

- Are interarrival times between messages Poisson?
- Is message size bimodal? Do a few messages account for most of the bytes transferred?
- Are there bursts of large messages?

	Track	Genmap	DGS	Pmake
Total Messages	857 (0.3)	3530 (97.3)	2829 (298)	144 (0.3)
Total Multicasts	805 (0.0)	276 (3.4)	708 (74.4)	115 (0.5)
% Multicasts	94%	8%	25%	79%
Number of processes	4	5	3	5

Table 4.1: Message passing statistics

The counts shown for total messages and multicasts are the means and standard deviations (in parentheses) for the 10 runs of the application.

Table 4.1 contains general information about message passing in our workload. It shows the total number of messages per run of each application, the number of those that were multicasts, and the number of processes used. The numbers presented are the means of ten runs of the application. The standard deviations are given in parentheses.

4.4.1.1 Interdeparture times

Both Caceres, et. al. [CDJM91] and Paxon and Floyd [PF94] observed that although some interarrival intervals on a network are Poisson, most are not. We wondered if this attribute is

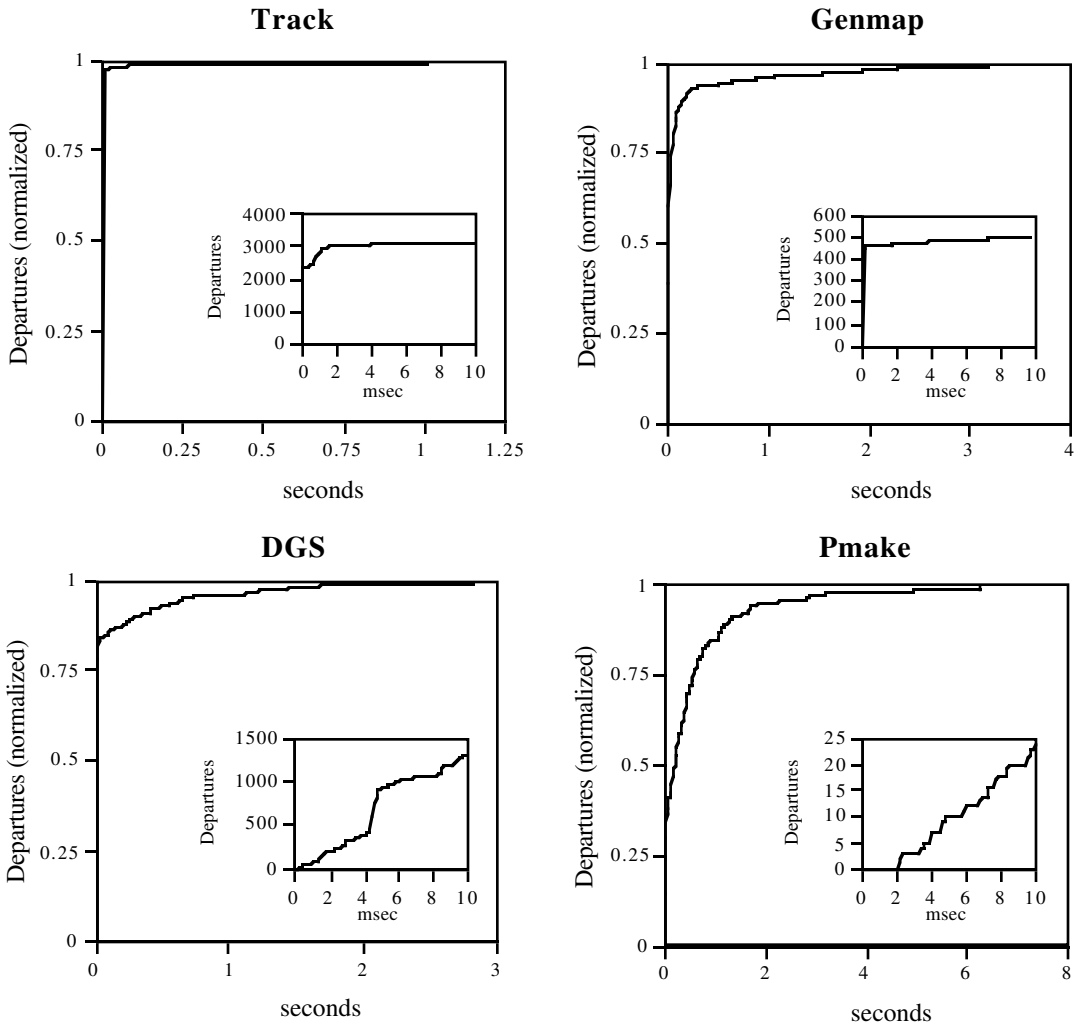


Figure 4.6: Cumulative histograms of interdeparture intervals

inherent to the message traffic, or an artifact of the network interface software. Therefore, we asked the question - Are the interarrival times between messages Poisson?

Arrival, in the studies mentioned above, refers to the arrival of a packet at the network interface. We examine message traffic from the application perspective; thus it was necessary to map message events to network-arrival events. We chose the intervals between send events, or *departures* from the application processes, to represent the intervals between arrivals at the network. Since we are using existing off-the-shelf TCP protocols that do not support multicasting, multicast messages were treated as a series of individual unicasts between the source and each destination.

The interdeparture distributions shown in Figure 4.6 were generated from the global trace file by extracting all send events, calculating the intervals between the events, and building a histogram. As stated in Section 3.2, the timing in a global trace is only accurate to 1 millisecond, so the many intervals measured less than that may not be exact. However, the overwhelming majority of these occur between departures from the same processor, and thus are correct to the full accuracy of the system clock. The outer graphs in Figure 4.6 have been normalized by the

total number of messages in each application to make it easy to determine what percentage of message departures occur within a particular interval. The inner graphs show the detail of short interdeparture times; their frequencies are left as actual counts.

The means and standard deviations of interdeparture times for the four programs were: Track 16.4 (150), Genmap 181 (1076), DGS 43.1 (135), and Pmake 145 (779) milliseconds. The inset graphs show that a large number of messages are sent within 1 millisecond of the previous message in Track and Genmap.

Qualitatively, we can see from the outer graphs that Track, Genmap, and DGS all have very steep curves with sharp knees. This indicates that they have at least two time constants and are therefore not Poisson. The inner graphs confirm this by showing the detail of the first 10 milliseconds. These graphs show the same sharp-knee characteristic. Both graphs of Pmake suggest that it could be Poisson. We did a χ^2 goodness of fit to test the hypothesis that Pmake interarrival times are exponentially distributed. The hypothesis was rejected with a χ^2 value of 471 with 50 degrees of freedom. The χ^2 values of Track, Genmap and DGS were all greater than 5000. Thus, the probability that the interdeparture intervals are represented by the Poisson distribution is less than 0.005 for all four applications.

4.4.1.2 Message size

Caceres, et. al., observed that packet sizes are bimodal, with interactive applications generating short packets, and file transfer programs (like FTP) very large packets. Paxon and Floyd observed that a few packets account for most of the bytes transferred. We examined the message sizes to determine if these facts hold true for messages as well as packets.

Figure 4.7 shows the distribution of message sizes. In these plots multicasts are represented for each specified destination. The size reported is extracted from the Isis *bcast* routine just before the message is buffered for transmission, thus including the headers and padding necessary to make the data architecture independent. The peaks in the histograms are plotted off scale so that the less represented messages sizes may be examined.

Each application has particular message sizes that account for the majority of traffic. These popular sizes appear to be unrelated across applications. Pmake and Genmap each have one message size that accounts for virtually all their traffic. Track has two sizes, 104 and 216 bytes, and DGS has five popular sizes. DGS is also the only application to exhibit bimodal behavior with 50 messages that were 11,800 bytes long. Note that 50 is only 2% of the 2419 messages sent in this application, but those messages represent 36% of total number of bytes transferred.

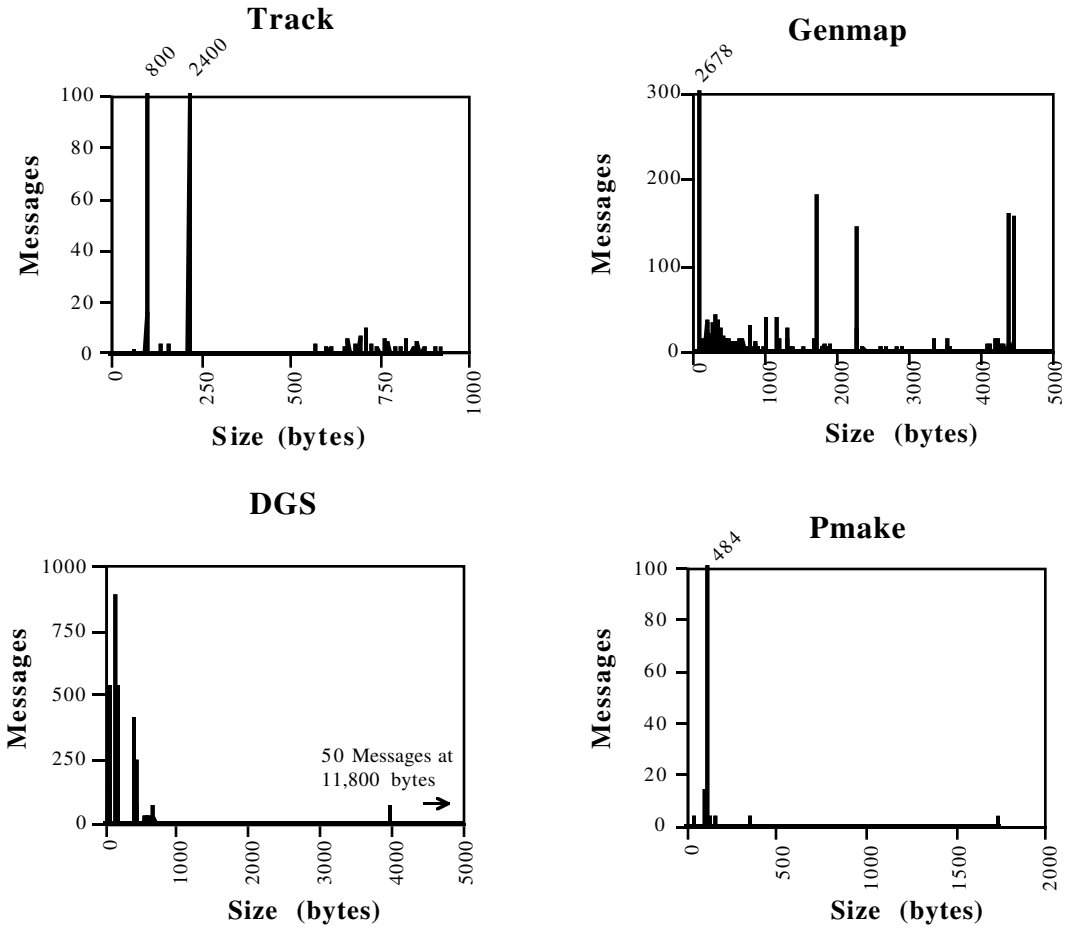


Figure 4.7: Message size histograms

Our results suggest that networks be designed to handle medium size messages (80 to 200 bytes) efficiently. This would accommodate 75% of messages in our workload. It might also be possible to exploit the uniformity of message sizes in buffer allocation . If common sizes could be detected, then fixed-length buffer schemes could be used to reduce overhead, and perhaps buffers could be pre-allocated in bulk rather than individually on demand. Separate buffers for large messages would probably benefit one of the four applications.

4.4.1.3 Size and burstiness

A critical question in the design of asynchronous message protocols is the existence of bursts of large messages. If multiple large messages are sent in a burst, then pre-reservation schemes must be used to insure buffers are available at the destination site. This adds 1 to 3 roundtrip times to the transmission latency. If bursts consist of only small messages, then packets can be sent optimistically expecting a destination buffer to be available. To look at both bursts and size, we created Figure 4.8 by plotting the size versus departure time for each message. The horizontal dashed lines mark the popular sizes from Figure 4.7.

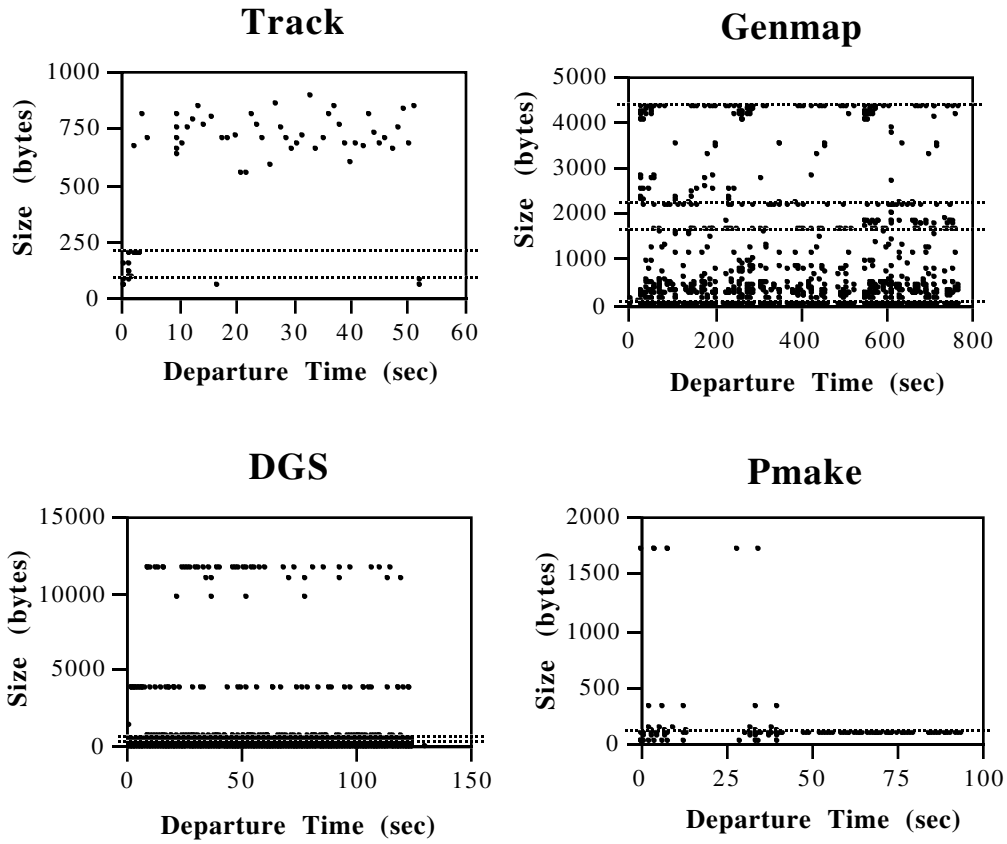


Figure 4.8: Size vs departure time

The dotted lines mark the peaks in Figure 6.

Figure 4.7 shows small peaks of large messages in two programs: Genmap and DGS. We arbitrarily define *large* to be messages larger than the common block size 512 bytes[8]. Genmap has four peaks greater than 512 bytes. Figure 4.8 shows that these messages are evenly distributed throughout the run of the program. DGS has two very small peaks at 4 and 12 kbytes. These messages combined account for only 4% of total traffic. Again, Figure 4.8 shows them to be spread evenly over the entire run of DGS. Thus, there are apparently no bursts of large messages in this workload; however, several other points about these plots raise interest.

First, the majority of messages in Track all occur in extremely short bursts at the beginning of the program. After that there are only a few scattered messages which contribute little to traffic. This apparently is due to the birth of tracks at the beginning of the simulation and a very slow death rate chosen in this dataset. The birth notices overwhelm the later steady-state traffic.

Second, note the repeating patterns in Genmap where the identical combinations of large messages occurs three times. We believe this is due to the load average statistics being sent to the central control process. DGS and Pmake also exhibit a repeating pattern, though it is a bit less obvious. The origin of these patterns is unknown.

Our data indicate that the occurrence of bursts of large messages does occur, but not often. It may be treated by the transport protocol as an exception, or perhaps as an error to be avoided since it does not balance communication and computation evenly. On the other hand, very fast bursts of small messages do occur and should be handled with the least amount of transmission latency.

4.4.2 Message arrival locality

This section describes the locality in arrival patterns of messages at an individual process. Specifically, two kinds of locality are examined: persistence and temporal locality. Both of these attributes have been observed in packet arrivals. Several techniques are currently in practice that exploit locality to expedite processing of packets. We wondered if locality is inherent to distributed communication, or if it is a consequence of the network level protocol fragmenting large messages and combining small ones. We examine persistence and temporal locality separately.

4.4.2.1 Persistence

Persistence has been observed in packets by [Wil89, Mog90]. Williamson reported 71% persistence in packet traffic on the V system. Mogul reported 60-70% persistence in network traffic to specific processes. Persistence is exploited by 1-behind caching of transport or session level state to reduce latencies in packet delivery.

Table 4.2 gives the count of persistent messages for the individual programs in our workload. A message was counted as persistent if it arrived at the process directly following another message from the same source process. A count was made of all persistent messages and

[8]Maximum packet size in Isis is 1600 bytes. Larger messages are broken into multiple packets. Defining *large* as 1500 bytes does not change our observations.

the total number of messages for each process. These numbers were then summed for all processes in the application. The amount of persistence observed varied 10-20% from run to run. This table gives the means and standard deviations over the 10 runs of each program.

Figure 4.9 shows persistence as a percentage of total arrivals. Track and Genmap both show very high persistence (shown in black), while DGS is remarkably low. Low persistence in DGS is especially unusual considering there are only three communicating processes in the application. Intuitively, one would expect something on the order of 30-50% just from randomness. A direct examination of the DGS data showed that messages arrived in almost perfect alternation from the other two processes. We investigated this phenomenon further by examining the DGS source code and talking with the implementors.

Persistence						
	All Arrivals	1-behind	2-behind	3-behind	4-behind	5-behind
Track	2419 (1.66)	2358 (10.3)	10 (1.3)	1.7 (1.6)	0.1 (0.3)	0.2 (0.4)
Genmap	2178 (47.7)	1489 (22.0)	88 (13.9)	125 (22)	238 (9.9)	104 (7.6)
DGS	4242 (447)	250 (48.3)	3741 (407)	237 (45.8)	6.0 (3.9)	0.1 (0.3)
Pmake	581 (21.4)	180 (21.2)	144 (13.7)	23.5 (8.95)	12 (4.4)	2.3 (2.3)

Table 4.2: Persistent message arrivals

The values represent the means and standard deviations of arrivals over the 10 runs of each application.

DGS is a distributed database service with replication. Clients request service by broadcasting to all servers. If there is no server on the same machine as the client, the servers respond to requests in a round-robin fashion to distribute the load. Since there are two remote servers in our test configuration, this resulted in exactly alternating messages among the three processes. We spoke with the DGS implementors and learned that this load balancing scheme was intentionally chosen, though no studies were done to determine if it improved performance. From a communications perspective, it increases traffic and communication latencies if used in a system that has 1-behind caching of transport protocol state. Application programmers often make these kinds of counter-productive choices since their understanding of the system implementation is low.

This alternating behavior has been noticed in other applications [Fel94], and there is some question of whether 2-behind caching (storing the transport protocol state from the last 2 arrivals) would be cost effective. To investigate this idea, we extend the definition of persistence to *n-behind persistence,* which is a message arriving from the same process as the nth previous message. Figure 4.9 shows the instances of 2-, 3-, 4- and 5-behind persistence graphically, and Table 4.2 presents the same data numerically. Note that our applications use few processes, and the 4- and 5-behind persistence numbers may reflect that. Both DGS and Pmake have a large amount of 2-behind persistence. These data indicate that for this workload 2-behind caching would improve performance.

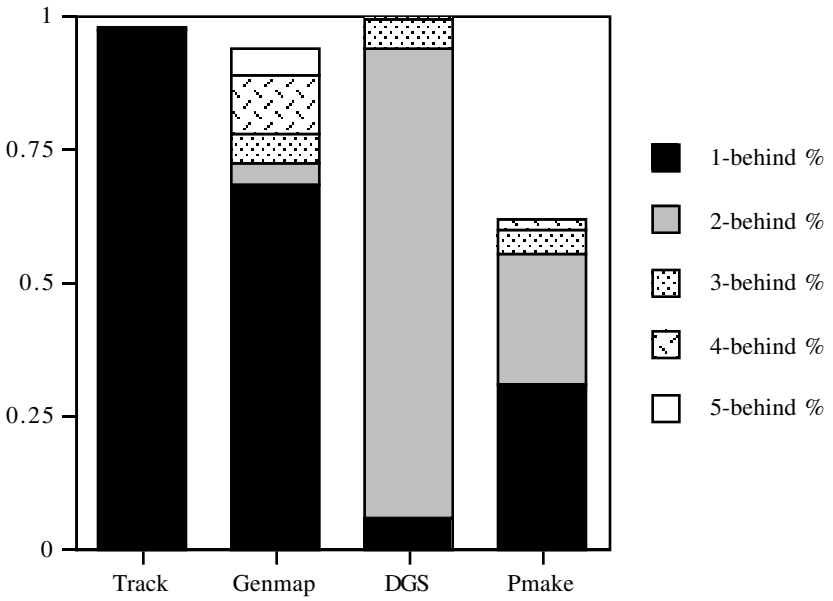


Figure 4.9: Persistent messages as a percent of total arrivals

4.4.2.2 Temporal locality

Temporal locality is the tendency of another packet or message to arrive within a fixed amount of time after the previous packet or message. It is examined by plotting a histogram of the interarrival intervals. Caceres, et. al. measured TCP traffic in [CDJM91]; they observed many packets arriving in the 100-1000 millisecond interval for interactive applications and in both the 1-10 millisecond and 100-1000 millisecond intervals for bulk transfers. Williamson [Wil89] was looking at longer intervals and found interarrival spikes at less than 1 second, at 20 seconds, and at 60 seconds for packets in V system. In that paper, he suggested that caching of session level state could take advantage of this phenomenon. Mogul [Mog90] observed spikes at 1 millisecond and 8 milliseconds and explored the feasibility of spin-waiting processes rather than blocking.

We ask two questions in this paper.

- Is temporal locality present in messages analogous to that observed in packets?
- Are there characteristic frequencies in the arrival distribution?

To answer these questions we calculated the interarrival intervals of all messages arriving at a process. The intervals for all processes in the application were used to build the cumulative histograms in Figure 4.10. The outer plots are of the total distribution, the inner plots are a blowup of the first 10 milliseconds. The y-scale of the inner plots is in arrival counts to demonstrate that a considerable number of arrivals occur at intervals less than 1 millisecond.

Note that messages were timestamped when received by the application process, not on their arrival at the destination site from the network. This means that latencies due to processing, assembling, queuing, and scheduling are all represented in these plots. The most significant latency might be scheduling delays in the application itself. Isis has first-come-first-serve, non-

preemptive scheduling of threads. Therefore, if the process starts a long thread, arriving messages are queued until the long thread is done and the receiving threads are run. The arriving timestamps for these messages occur after this scheduling delay.

The cumulative histograms in Figure 4.10 show several jumps indicating characteristic frequencies, though none are very dramatic. In Track, almost 75% of messages arrive within one millisecond of the previous one, with smaller jumps at 1.5 milliseconds and 2 milliseconds. This is unexpected given the roundtrip time of the system is 6 milliseconds. It is clearly indicative of the "update" style, unidirectional communication in this application. The other three applications show generally longer interarrival intervals. Genmap and Pmake (the two task-style applications) have many arrivals within 1 millisecond. Genmap has a small jump at approximately 6 milliseconds (the roundtrip time), which suggests some synchronized communication is occurring. DGS has a considerable number of arrivals at less than 1 millisecond (30%) and another 30% from 1 to 20 milliseconds. Our conclusion is that there is no natural cut off frequency for "session" level caching, although remembering state for 3 seconds captures 80% of traffic in all cases.

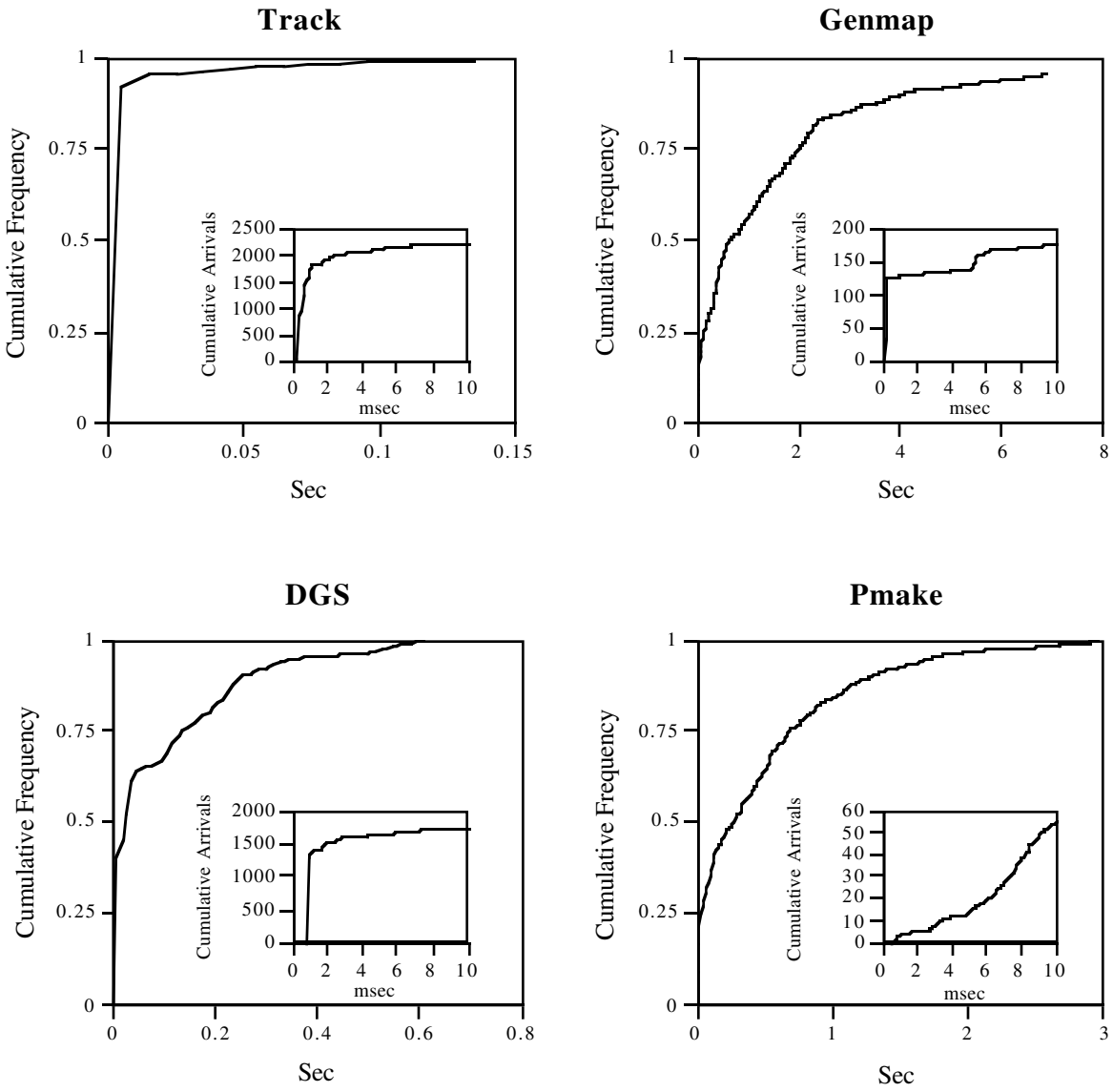


Figure 4.10: Interarrival time distributions

The nested graphs are blowups of the first 10 milliseconds of the outer graphs. The y-scale is actual number of arrivals in the embedded graphs.

The commonality here is in the shorter regions where all applications show a considerable number of messages arriving at short intervals. These graphs show that significant amounts of communication happen within 1 millisecond. Using the method described here, it is not possible to know if messages are arriving at those short intervals, or if they are being held in a queue and then many threads are run in quick succession after a long thread has finished. The interdeparture time data in Section 4.4.1.1 showed many messages sent at less than 1 millisecond intervals, and therefore supports this evidence that messages arriving at the site (before queuing delays) exhibit very short interarrival times.

4.5 Discussion

The three goals of this study were to:

- develop a method for generating a global trace of message traffic
- examine message traffic for characteristics similar to packet traffic
- determine if messages arriving at a processor exhibit the spatial and temporal locality that packets do

We developed a method of generating global traces that is accurate to at least one half the roundtrip time of a message over the network (3 milliseconds in our study). Evidence suggests that the traces are actually accurate to 1 millisecond. We related our method to those of Duda and Ashton.

We examined message traffic and found it has many characteristics in common with packet traffic. The message traffic arrival was found to be inconsistent with the Poisson distribution. The interdeparture time histograms all showed a significant number of messages sent at less than 1 millisecond intervals. This means that a high throughput is maintained in spite of network latency. This attribute is reminiscent of satellite voice communication, and perhaps their slotted reservations schemes (similar to time division multiplexing) could be applied to prevent congestion. This could be especially useful with more periodic traffic as expected due to multimedia applications of the future.

In examining size, we found some occurrence of bimodal message sizes; however, 75% of messages were in the small to medium range (less than 512 bytes). We speculate that most of the bimodal nature of packet size may be caused by fragmentation of packets rather than anything inherent to the classic network tasks: telnet and ftp. Caceres, et. al.[CDJM91] observed that bulk transfer programs generate a considerable number of small packets. A future study of our workload at the packet or network level would determine whether this effect is inherent to the workload or fragmentation artifact.

Few bursts of long messages were detected, which suggests that traditional optimistic buffering schemes should be continued, since overflow conditions would occur only rarely. Pre-reservation of receive buffers would add unnecessary latency. The burstiness does reflect the necessity of reactive congestion control schemes such as rate-control or credit-based control schemes in the design of ATM networks.

Our study of arrival locality found significant persistence which clearly suggests the usefulness of 1-behind caching of protocol state. An alternating persistence pattern was observed in the distributed database application which was determined to be caused by programmers attempting to load-balance requests in a round robin fashion. This behavior suggests that 2-behind caching would be effective as well. More extended caching would have limited effect on the performance of this workload. However, this study was limited to only five sites (and thus usually 5 processes per application). Applications with more processes may benefit from additional caching.

The high temporal locality suggests that improved scheduling techniques may reduce message delievery latency. Isis has a pop-up thread mechanism which creates a new thread for each arriving message. Since the arrival of the next message is expected within a millisecond, idle periods could be used to pre-create a thread for the anticipated messages. Then when the message arrives, it is simply attached to the already scheduled thread. There was some evidence

of periodic messages in the Track application. Periodic traffic is expected to increase in the future with the development of multimedia applications. This suggests the investigation of predictive scheduling techniques that would anticipate the arrival of periodic messages. For applications with many short message handlers and a few large compute intensive threads (such as Track), this would help to reduce message delivery latency due to scheduling, and increase the overlap of communication and computation.

This is the only study of which we are aware that describes message characteristics at the application or message level rather than the network or packet level. As such, the observed usage and patterns are not only useful in improving distributed programming packages like Isis, but they also provide insight into the nature of communication in such programs. This may prove valuable in the design of distributed interconnection networks, architectures, and operating systems.

Chapter 5

Benchmark Blending:
A Hybrid Performance Prediction Technique for Configuring Distributed Programs

5.1 Introduction

Programmers need tools to help them understand the interactions between components of a system, and to project the execution time of prospective designs and configurations. Without these tools, their superficial understanding of the system often leads them to make choices that poorly utilize system features (like caching, scheduling policies, and buffering). We saw an example of this in Chapter 4 where the designers of the DGS system chose a round-robin servicing policy that defeated the 1-behind caching feature of TCP (Section 4.4.2.1).

Tools of this nature are called *performance prediction models*. They involve measuring and predicting the execution time of the various operations of the system and subsystems. They are built using direct measurements (benchmarks and monitors), simulators, and analytic techniques. Performance prediction models are used by management for making procurement decisions, by programmers for developing code, by system administrators for tuning systems, and by the operating system itself for making scheduling and configuration decisions.

This chapter presents a technique for constructing performance prediction models to be used by programmers to gain understanding of system performance, and by the operating system for scheduling and configuration. We give two examples of systems that require performance prediction models. In real-time scheduling, complex distributed tasks often involve parallel execution of subtasks at different nodes. To meet the deadline of a global task, the scheduler must estimate the execution times of the subtasks and assign them to processors in such a way that all will finish before the deadline of the global task [LT91, KG94]. The second example occurs in large distributed database systems. There exists no single sorting algorithm that provides good performance over the range of large list sizes. Thus, sorting a list in minimal time requires that the strategy, buffer-size, and number of processors all be chosen after the dataset is

known [MLP87, BBDW83]. Later in this chapter we will develop a performance prediction model designed to aid this adaptation of sorting engines to input.

From these examples, we note several applications in which performance prediction models can be used by operating systems:

- scheduling subtasks of a complex global task,
- run-time adaptation of programs to a particular input,
- configuring input dependent system parameters (buffer sizes, number of processors to use, memory allocation, etc.).

Prediction models used by an operating system must arrive at their prediction quickly. Simulation and benchmark techniques can not be used on-the-fly to meet these needs since they take far longer than the given time constraints.

Validated analytic models are ideal for picking strategies and configuring key parameters at run-time. Several excellent models of subsystems exist [LK93, Dan91, NT88]. Existing models can be used individually or combined to model the needed algorithm. However, analytic models have two drawbacks when used for adaptation. First, they have generalized parameters that are difficult to understand by the applications programmer. This makes the models difficult to code and configure for the particular system at hand. Often these parameters are not known for the application. Second, building and validating accurate analytic models is difficult and time consuming. Modern system enhancements (like memory and disk caching) add tremendous complexity to the models, and often makes them specific to a given architecture. The models generally are not flexible enough to transfer to a another architecture. In summary, if an appropriate validated model exists for the specific system, it can provide an excellent basis for predicting performance; however, if such a model does not exist, building or adapting one is usually beyond the resources of the application programmer.

Benchmarks provide a more attainable and flexible prediction of system behavior. There are several different kinds of benchmarks (Section 2.4.1); synthetic benchmarks are ideal for performance prediction because they have adjustable workloads based on a parametric description of the actual workload. The parametric description of the workload makes them excellent tools both for predicting performance of a specific dataset, and for understanding why a system performs as it does. Unfortunately, benchmarks take a long time to execute. Therefore, they are ill-suited for scheduling or run-time adaptation of software.

We present a new technique, *benchmark blending*, that combines the speed of analytic models with the flexibility of benchmarks. It is heavily influenced by Hellerman's model of I/O and CPU overlap in [HC75] (see Section 2.4.2). The technique involves four steps. First, the application programs are broken down into large chunks called components. Second, synthetic benchmarks are built to measure the performance of these components on the target system. Third, the measured execution time of each benchmark is described analytically. Finally, the analytic descriptions are combined (blended) to form a model that predicts the execution time of the entire original program.

In addition to being fast and accurate, models built with benchmark blending have two advantages over other analytic prediction methods. First, benchmark blending implicitly accounts for the costs and benefits of operating system features (such as caching, paging) that are too complex to model analytically. This makes them much easier to build, and more likely to adapt to new architectures. Second, the parameters chosen for the model more intuitively match the parameters of the application. This provides better insight into system performance, which

can lead to faster algorithms and better use of system resources. Though benchmark blending assumes some determinism of execution time, the resulting validated models provide accurate prediction of overall performance.

Benchmark blending is similar in concept to *scalable benchmarks*. Scalable benchmarks are designed to predict performance on systems larger, or in some other way different, than what is available. Section 2.4.1 describes two approaches to scalable benchmarks. In [CP93], Chen and Patterson present an I/O benchmark that can be used to predict performance of a particular workload on a variety of architectures. However, since its output is non-analytical, it cannot be combined with other components of the algorithm (such as computation or communication) to generate a global prediction model. Harbitter's scalable benchmarks [Har90] are a very good method for predicting performance; unfortunately, they depend on external analytic models which though available for his application (transactions systems), are not common for other domains. Both approaches to scalable benchmarks do not consider the parallelism of the specific task at hand, i.e., which components can overlap and which can not.

There are two other approaches to performance prediction which use composition of pre-existing models. One is the notion of parallel program *templates* [NPACKZZ93, VSSG88]. To use these techniques the programmer must pick the template that correctly maps the application-level constructs onto the hardware. The templates are actually analytic models, which are then calibrated to predict the execution time of the program. The second approach, *lost cycle analysis*, uses previous analytic descriptions of overhead [CQ93, CL93]. This method relies heavily on compiler techniques to quantify how much each type of overhead contributes to execution time. Both of these approaches attempt to finely model overhead, while our strategy is to describe it macroscopically using benchmarks.

The remainder of this chapter comprises three sections. The benchmark blending technique is described in Section 5.2. An application of this technique to distributed sorting algorithms is described in 5.3 In 5.4 we conclude with a discussion of the method's strengths and weaknesses.

5.2 Benchmark Blending

This method can be described by a series of steps:

1) break the program down into components
2) build benchmarks to measure component execution time
3) run the benchmarks, and model the results
4) determine the combination equations (*blending*)
5) validate the model with an implementation of the program

There are considerations associated with each step of benchmark blending. We discuss the considerations for each step below along with suggestions that might ease their implementation.

5.2.1 Breaking into components

In the first step, the application or program is decomposed into components. Components can be either macroscopic or atomic. There are advantages to making components as macroscopic as possible. Macroscopic components automatically capture the unanticipated

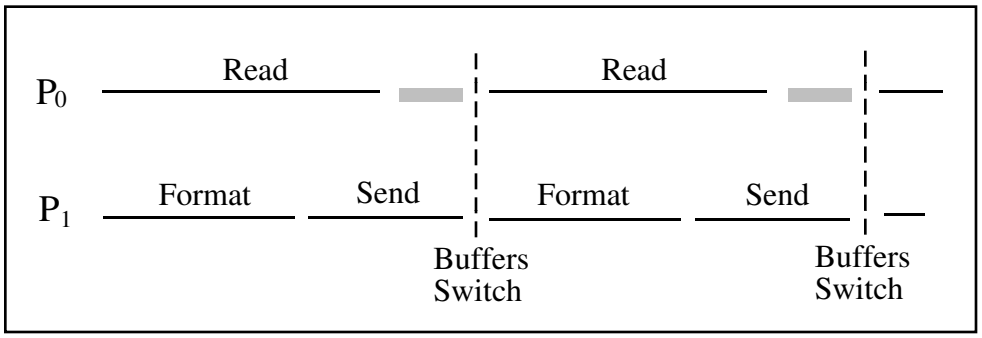


Figure 5.1: Ticker-tape processor timings

performance factors of the system and the algorithm, thus they are more likely to represent the behavior of the program. There are two reasons for choosing more atomic components: to enhance the programmers understanding of why the system performs as it does, and to simplify the modeling task. By separating functions that overlap, such as I/O and computation, both the benchmarks and their modeling are simplified by deferring concurrency issues until later in Step 4.

To explain this decomposition step more clearly, we present an example of decomposing a simple program into components. Our example is a ticker-tape reader that is designed to read data from a ticker-tape machine (presumably on Wall Street), reformat the data, and send it to a remote database server. We assume a shared memory environment. The reader is designed with two shared buffers. Data are read into one buffer by processor zero (P_0) until the buffer is full. Concurrently, P_1 formats the data in the second buffer and sends it to the database. When P_1 is done, the buffers are switched. P_0 begins reading in data again, and P_1 begins processing the new data.

Figure 5.1 shows the timing of the ticker-tape reader program. Each processor is represented by a time-line. The solid horizontal bars represent computation and communication. The gray bars represent processor idle time. Dotted lines are used to indicate where the buffers are switched. We have labeled the separate functions of the program as *read*, *format*, and *send.*

There appear to be two natural decompositions of the ticker reader program. It can be broken down into three components corresponding to its three functions. It can also be decomposed into two components representing the operations of each processor. Which decomposition to use depends on what level of insight the programmer desires, and what is easy to model. For instance, if the programmer is interested in adjusting the buffer size to reduce the amount of data lost during P_0's idle time when the buffer is full, then the two-way decomposition is appropriate. If the programmer wishes to optimize processor utilization, then the three-way decomposition will provide more insight into how P_1 is being used.

Reusability can also be considered when choosing components. Often after gaining more insight about performance issues from the completed model, the algorithm is modified to take advantage of an under-utilized resource. Reusable components save time when modeling the new algorithm.

5.2.2 Designing benchmarks

The main concept in this step is to build benchmarks that emulate the components as closely as possible. These are synthetic benchmarks, with variable workloads that span the range of possible inputs to the component.

It is important to parameterize the workload of the synthetic benchmark in the same manner as input is specified to the component. If the benchmark has additional parameters beyond those of the component, then assumptions will have to be made in the global model about how to set those parameters. For example, say we decompose the ticker-reader into two components, READ and PROCESS, where PROCESS incorporates the two functions *format* and *send.* In designing a benchmark for READ, buffer size would be a good choice of parameter, since it would have to be specified to the READ component in the program. Specifying input rate as a parameter would not be a good choice, since it is not specified to the READ component. If input rate were included in a benchmark model, then its value would have to be estimated somehow at the blending level (Step 4). By not including input rate as a parameter, we force the benchmark to be designed to capture that value implicitly by reading the actual input stream. This reduces the generality of the benchmark but increases the accuracy of the overall model.

An alternative way to capture significant parameters is to use natural benchmarks where possible by lifting the code directly from the program. Natural benchmarks capture the performance of the system due to not only parameters that elude specification, but all the unanticipated parameters as well. In the ticker-reader example, the benchmark representing the PROCESS component could use the exact code that runs on P_1. This would be especially advantageous if the *format* and *send* functions are interleaved.

When designing benchmarks, it is tempting to design them with more generality than the components. However, generalization can lead to inaccuracies. For example, the complex features of network protocols (like the one-behind caching described in Section 4.4.2) and memory management (cache and paging policies) can be affected by seemingly insignificant changes in access patterns. It is safer to stick with the real messages and the real data structures used in the program wherever possible.

5.2.3 Modeling benchmarks

In this step, the benchmarks are run with various input parameters, and analytic models are generated that predict their execution times. One approach to this process is to generate a graph for each parameter that varies that parameter while holding the others constant at midrange. The shape of the graph supplies clues about the parameter's effect on the execution time of the benchmark.

For performance prediction, simple models are better than complex ones with a lot of parameters. In many cases a straight line can approximate behavior. Such a line can be fit using a linear regression routine available in any statistics package. Nonlinear models (polynomials, logarithms and exponentials) are also very powerful, since the behavior of many algorithms are already described in these terms. For instance, almost all sorting algorithms are known to run in $O(n \log n)$ time. There are several packages available for fitting non-linear functions of this type [Bah95, MAT95, Ult95].

Empirical models can also be used to model the benchmarks. Lee and Katz have developed a model of I/O [LK93]. Danzig has a model for TCP latencies [Dan91] which can be enhanced into a model of communication time. Nelson and Tantawi have a model for Fork/Join synchronization of distributed processes [NT88]. The disadvantage of using existing models is that they usually have parameters that are difficult to measure, and, in the end, assumptions must be made in order to set those parameters. It is often easier to derive a new model that empirically describes the behavior of the benchmark. There are several papers describing this process [BHH78, Sch93].

Whatever method is used, one must be sure to validate the model over the range of inputs that are expected by the overall program. This is important for the accuracy of the global model. If a benchmark model is inaccurate at any point in this range, it will effect the accuracy of the global model. Finding the limits of accuracy now will simplify debugging of the total model during the validation in Step 5.

5.2.4 Blending

In Step 4, we determine the proper balance for combining the benchmark models. This process is facilitated by drawing time-lines like those discussed in Section 2.4.2 and used in Figure 5.1. The diagrams generally clarify which components contribute to each term in the total execution time equation.

There are two methods for drawing timelines. In one method, a line is drawn for each component, and then it is determined how much each function can overlap with another. In the second method, a line is drawn for each processor; and then it is determined what each processor does during each phase of the program.

When drawing timelines, it helps to consider which component bounds the program. The program may be known to be bounded by I/O, CPU, or synchronization. Timelines can be drawn to reflect this case. For example, the ticker-reader in Figure 5.1 is shown to be bounded by the PROCESS component. At this step, we wish to capture the average normal case, not the pathologies. The complexity of adding unusual cases can be added later if the validation step demands it.

Processor idle time may need to be considered. Processors are often idle due to synchronization or poor load balancing. Much of this can be accounted for with minimum and maximum functions as described by Hellerman (Section 2.4.2). For example, the blending equation to describe one cycle of the ticker-reader would be

Maximum (READ, PROCESS)

Sometimes idle time can be ignored. If communication occurs in a grid pattern, the processors at the end of a line are generally idle more than the others; however, if they also do less communicating and less computation, this idle-time will probably have no effect on total execution time.

Several other attributes can be ignored during construction of the blending equations. For instances, transient phases (like startup and shutdown) typically contribute little to execution time. If they do make a significant contribution, they can sometimes be treated as independent

components of the algorithm. It may be easier to develop expressions for each phase of the algorithm and then add them together, rather than attempting to describe the general situation with one expression. There is an example of this in the case study (Section 5.3.2.4), where the sorting algorithms have an asynchronous phase and a synchronous phase.

Sometimes components can be grouped across iterations, as done in the time-indexing example given in Section 2.4. Pipelining algorithms are easy to describe this way. Combining parts from different iterations can simplify dealing with processor idle time as well. There is an example of this in Section 5.3.2.4.

5.2.5 Validating the model

Validation involves measuring an actual implementation of the algorithm and comparing it against the computed performance. The first step is checking the performance prediction of configurations known to exist in the real application. If the model does not work in this region, there is usually an error in the blending equations. Since we validated the benchmark models in Step 3, problems are much more likely to exist in the blending step than in earlier steps. Often times the modeler has to iterate between Steps 4 and 5 to refine the accuracy of the model.

Another step in validating a model is to plot a single parameter graph for each parameter. The trends (1st derivative) on these graphs should match the measured data. Do the slopes vary the correct way? Does larger input take longer to process? The shape of the plots (2nd derivative) are also clues to the validity of the model. The shapes should be the same as those outlined by the measured data. If they are not, some range of that parameter will be grossly inaccurate. This is fine if the inaccurate range lies outside the region where the model will be used.

Similarly, it is important to determine the boundaries of accurate prediction for each parameter. That is, find a low and high value for each parameter where the model predictions begin to drift away from the measured values. This defines the bounds of accuracy of the model, and is an important piece of the model description.

5.3 Case study: sorting

Up until now we have been very general in our descriptions of the steps in benchmark blending. Here we present a case study in attempt to concretize those concepts and procedures just described. We demonstrate how to develop two models that predict the execution time of two LAN sorting algorithms. These algorithms are modeled with the same set of components, which reduces the model development time. We show how to use the models to configure the sorting algorithms for optimal buffer size and the number of participating processors. We also show how to gain insight into the nature of the algorithm by examining how much each component contributes to total execution time.

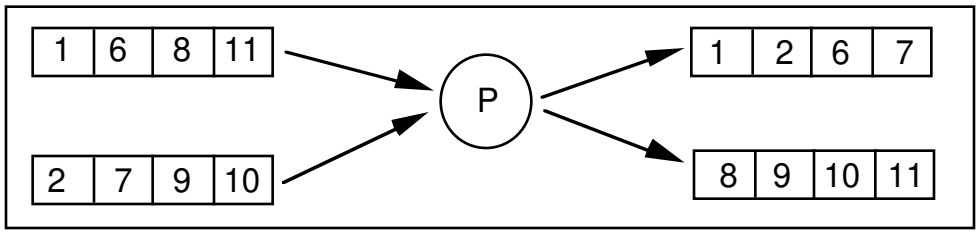


Figure 5.2: Two-way merge-split

5.3.1 The algorithms and their implementation

Distributed sorting is the problem of sorting a list of records or keys using more than one processor. The list is usually distributed across the local memory of the processors both before and after it is sorted. A *sorted distributed list* is one where, for all i, the keys on processor P_i are in ascending order and all the keys on processor P_i are less than any of the keys on P_{i+1}. The specific algorithms we consider here (*Block Odd-Even Sort* and *Neighbor Sort*) are both memory-based algorithms that assume the input is evenly distributed across the processors.

Block Odd-Even Sort and Neighbor Sort are based on the *Two-Way Merge-Split* [BS78] as described by Bitton in [BDHM84]. The merge-split was developed as a building block to deal with the limited parallelism available in present day parallel architectures. Many famous theoretical algorithms (such as Batcher's bitonic sort [Bat68]) assume a single key per processor and one key comparison-exchanges. Replacing the comparison-exchange with a merge-split creates block-based algorithms that can be implemented on machines with fewer processors. Thus the merge-split is the cornerstone to many practical distributed sorting algorithms.

A two-way merge-split merges two sorted blocks of keys, and then splits the result into two sorted halves. Both operations happen within the local memory of a single processor. Figure 5.2 demonstrates this process. Two sorted lists of the same size, *b*, are received and stored in the processor's memory. Then they are merged into a single buffer of size 2*b*. Finally, each half of the resulting buffer is sent to a destination processor. The merge-split uses extra memory to minimize the execution time of the merge.

5.3.1.1 Block Odd-Even Sort

Block Odd-Even Sort [BDHM84] is derived from Batcher's Odd-Even Sort [Bat68]. In this algorithm, each of the *p* processors initially have *n* keys in their local memories. In the first (asynchronous) phase, each processor sorts its local list. This is typically done with a sequential quicksort. The second (synchronous) phase consists of a series of steps, during which the processors exchange blocks of keys to combine the local lists into a globally sorted sequence.

The communication pattern during the synchronous phase is shown in Figure 5.3. During the odd steps the odd-numbered processors: 1) receive a sorted block from their right neighbor, 2) perform a two-way merge-split with the received block and the highest *b* records from their local list, 3) send back the higher *b* records that result from the merge-split, and 4) merge the lower *b* records into the local list. We call this fourth step the *insertion sort,* though it may be implemented with the same code as the merge-split. During the even steps, the even-numbered processors receive a sorted block from their right neighbor and perform the same process.

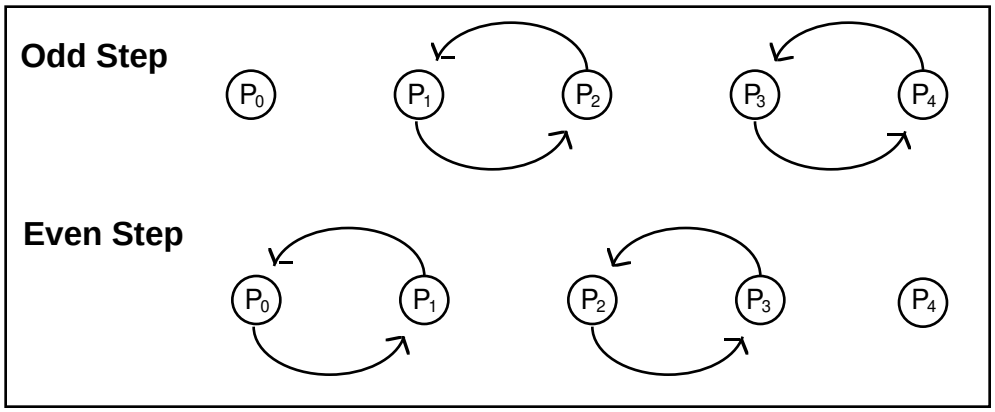


Figure 5.3: Odd-Even Sort communication pattern

The steps are described here as totally synchronous, but they can be implemented with looser constraints. In our implementation, synchronization is accomplished without additional overhead by cueing off the arrival of messages. Each processor strictly alternates between the tasks of receiving a block and performing a merge-split, and of initiating a merge-split by sending a block to its left neighbor. Even with this looser synchronization, there are still idle periods where no computation occurs. Section 5.3.2.4 discusses this more fully.

The algorithm, as described by Bitton, does only p exchange or swap steps (where p is the number of processors available), and buffer size (b) is equal to the number of local keys (n). This can be generalized to any buffer size smaller than n by performing $p + n/b$ steps.

5.3.1.2 Neighbor Sort

Neighbor Sort was developed by us to eliminate the idle periods in Odd-Even Sort. We begin by combining the odd-even steps. In Neighbor Sort, each processor is responsible for initiating a merge-split by sending a block to the left, and servicing the merge-split request from the right neighbor. These two steps may happen in any order, but the processor must wait until both are done before doing the insertion sort, and initiating the merge-split in the next cycle. Using this scheme, the number of steps is still $p + n/b$, but there is less idle time on each processor.

Synchronization is accomplished by sending the step number with each data block. Comparison with a local step counter determines whether or not the processor is ready for this data. If not, the data block is queued until the previous merges are done.

5.3.1.3 Hardware and software configuration

The algorithms and benchmarks have been implemented in a dedicated test environment consisting of five Sun Microsystems 4/65s running SunOS 4.1.3 on an isolated subnet. One machine is configured as a disk server with 28 Mbytes of memory and the other four machines run as diskless clients with 12 Mbytes of memory. All of the necessary SunOS and user files are resident on this server; no remote file systems are mounted. The disk server machine also acts as a gateway to isolate the subnet from outside traffic.

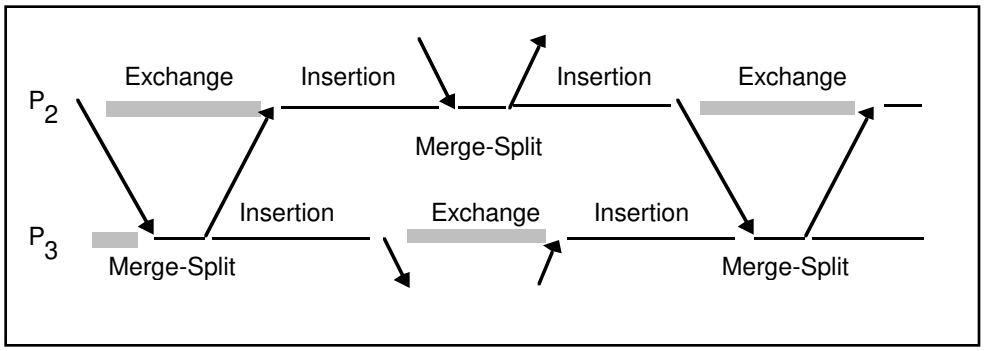


Figure 5.4: Odd-Even Sort Functions

The solid lines represent communication (slanted) and computation (horizontal). The gray bars represent processor idle time.

The programs are implemented using the Isis Distributed Toolkit [ISIS91]. Isis v3.0.6 runs on all five sites in the standard configuration. The times in Figure 5.7 were calculated using the trace generation software described in Chapter 4.

5.3.2 Building the model

5.3.2.1 STEP 1: Components of block sorting algorithms

We begin the decomposition of Odd-Even Sort by dividing it into two phases: the asynchronous phase during which it does the sequential sort of the local list, and the synchronous phase when block-exchange is occurring.

The asynchronous phase can be considered a single component which we designate SSORT (for sequential sort). SSORT represents the entire asynchronous phase of the program. This includes the program startup time and the actual sorting. The performance of SSORT is affected by a single parameter, *n*, which is the size of the local list.

The synchronous phase is more complex. We began its decomposition by drawing the timeline in Figure 5.4. The synchronous phase of the algorithm involves three functions: block-exchange, merge-split, and insertion sort. Both the merge-splits and the insertion sort were implemented with the same subroutine in our programs, so we chose to represent them both with a single component, MERGE. The MERGE component represents the act of merging two lists, which are both of size *b* in the merge-split case, and of size *n* and *b* in the insertion sort case.

The exchange function encompasses the act of sending a buffer to a neighbor process and waiting for the return buffer to arrive. To simplify the modeling of the benchmark, we decided to define a communications component, COMM, that is independent of *n*. Therefore we define COMM as the time it takes to communicate with the neighboring processor less the actual merge-split time. COMM is conceptually equivalent to the roundtrip of a message, or a null remote procedure call (RPC). The parameters that effect COMM are: buffer size, *b*, which is the number of keys in a message; and processors allocated, *p*, or the number of processors simultaneously sending and receiving messages.

COMM is a complex component encompassing the network transmission time, network latencies, the operating system handling of messages at the source and destination, the task switching of the local processor between send and receive processing, and the idle intervals due to the synchronization constraints of the algorithm. COMM was deliberately chosen to be as macroscopic as possible to avoid modeling this underlying complexity. It is not simply the act of sending message and receiving a message, but rather the act of sending and receiving a message in the particular environment of the algorithm. The benchmark for COMM must be designed carefully to capture the execution time costs of that entire environment.

In summary, Odd-Even sort is broken down into three components: SSORT, COMM, and MERGE. The notations for the three components and the functions they represent are:

SSORT	The entire asynchronous phase
COMM	Communication time less the merge-split
MERGE	The insertion sorts, and the merge-splits

The same components conveniently describe Neighbor Sort as well.

5.3.2.2 STEP 2: benchmarks

The components of our model are SSORT, MERGE, and COMM. When talking about the corresponding benchmarks, we use the same names except that the parameter list is added. We sometimes abbreviate the parameter list with a pair of empty parentheses. Therefore the benchmarks are called SSORT(), MERGE(), and COMM().

In both sorting algorithms, the asynchronous sorting phase is implemented using the UNIX sort utility. Our SSORT(n) benchmark is a program that sorts lists of various sizes using UNIX sort [Sun93]. The parameter, n, is the maximum local list size, or n = ceiling (N/p). Since the algorithm assumes that the global list of size N is evenly distributed across the processors, using the largest possible local n automatically incorporates the synchronization time at the end of the asynchronous phase.

The MERGE() benchmark is a program that merges two lists. The lists are typically of sizes b and b (a merge-split), or n and b (an insertion sort). This is a natural benchmark; it is the actual subroutine used in the sorting algorithms. We noticed that the execution time of this algorithm is solely dependent on the sum of list sizes, since it just copies each element in both lists to a (new) third list. Therefore, the MERGE(m) benchmark has been simplified to have only one parameter, m, which is the sum of the two list sizes.

Finally, the COMM(b, p) benchmark is the most complicated. It must mimic the exact pattern of communication in the algorithm. COMM() is a natural benchmark that was extracted from the sorting algorithms by starting with a full sorting program and deleting all computational components. This process reduced the complexity of the benchmark by making it independent of input list size. COMM() has two explicit parameters: buffer size, and the number of processors communicating. Logically, there are many implicit parameters to COMM(). We resist the temptation to add them in this step. When modeling COMM() during the next step we may have to add parameters to achieve the desired level of accuracy.

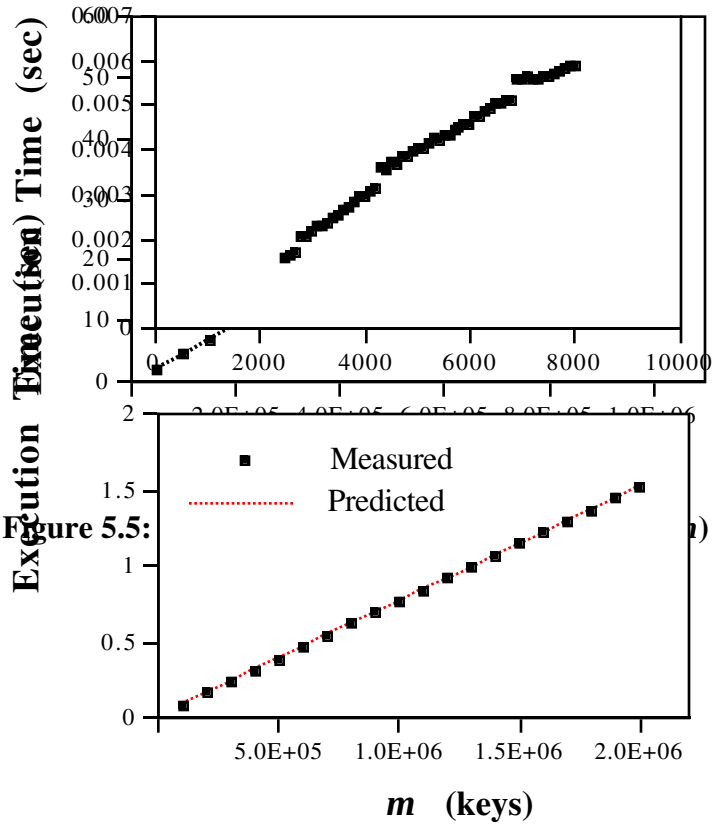


Figure 5.6: Merge benchmark - MERGE(*m*)

5.3.2.3 STEP 3: benchmark models

In this step we ran the three benchmarks in our test system and varied their parameters over the range of expected values. The execution times of these runs are shown as points on the graphs in this section (Figures 5.5, 5.6, and 5.7). Next we choose a function that describes the observed points, and calibrate its coefficients to fit the observed data. The resulting model predictions are shown on the graphs as lines.

SSORT(n)

The UNIX sort utility is implemented with a quicksort, which has a execution time of $O(n \log_2 n)$. Therefore, we expect that function to accurately describe the behavior of SSORT(). Figure 5.5 shows the time it takes SSORT() to sort lists of size 50,000 to 1,000,000. As suspected, the observed data is easily fit with the equation $A \cdot n \log_2 n + B$ to these observations. The result was A = 2.69e-6, and B = 0.061, with a residual standard deviation of 0.07. This was a minimal least squares nonlinear fit using Marquardt's algorithm.

MERGE(*m*)

Figure 5.6 displays the results of the MERGE(*m*) benchmark. The top panel shows the execution times for merging a list of 500 keys into a list of *m* keys, where *m* varies from 2000 to 8000. Note the discontinuities at *m* values of 2400, 4100, and 6400. We believe these are due to

additional page faults occurring as the list grows larger than available memory. We found that additional discontinuities occurred at irregular intervals as the list continued to grow.

Since the sorting algorithms will be sorting much larger datasets, we suspected that these discontinuities would be insignificant with respect to the time of the entire merge. Therefore, we plotted the execution time for lists of size 100,000 through 2,000,000 in the bottom panel. The paging anomalies were indeed insignificant at this resolution, and the behavior of MERGE() could easily be approximated with a straight line. We fit the curve A + Bm to this data using a linear regression program. The result (A = -0.06, and B = 0.000725) had a correlation of 0.991, and a standard error of the estimate of 0.109.

The paging behavior of MERGE() demonstrates the power of using benchmarks at this level rather than attempting to predict execution time with traditional analytic methods. Traditional modelers would either fail to incorporate paging effects, and thereby cause the model to be inaccurate in the smaller range of m, or they would incorporate that complexity (with tremendous effort) and then determine if it is significant in the parameter's range of interest. The benchmark approach is superior because it demonstrates the insignificance of the paging complexity on large datasets before we expend energy attempting to model that complexity.

COMM(b, p)

The top panel of Figure 5.7 displays the results of running COMM() for p = 2 through 5, and b varying from 10 to 1800 keys. To model COMM(), we first consider the bottom line, the case of only two processors communicating. The roundtrip time of a message is plotted against its size.[9] We discovered that the transmission time of a message is linear with respect to its size; therefore, we defined our model of COMM() using a linear model of transmission time. We used regression to fit the line (A + Bb) to this data. The result (A = 3.45e-3, and B = 5.43e-6) had a correlation of 0.999, and a standard error of the estimate of 1.15e-4.

Now we examined the effects of additional simultaneous communication and of multiple communication "partners" per processor. Again we measured the roundtrip times of messages between two processors, but this time with the other processors simultaneously sending messages as well. As expected, additional traffic increased the average roundtrip time of individual messages. We discovered that the effects of this additional traffic is normalized by dividing the roundtrip times by p. The bottom panel of Figure 5.7 shows the four lines from the top panel after being normalized.

[9] This is actually the average round trip time for 100 messages sent and returned from the neighboring process.

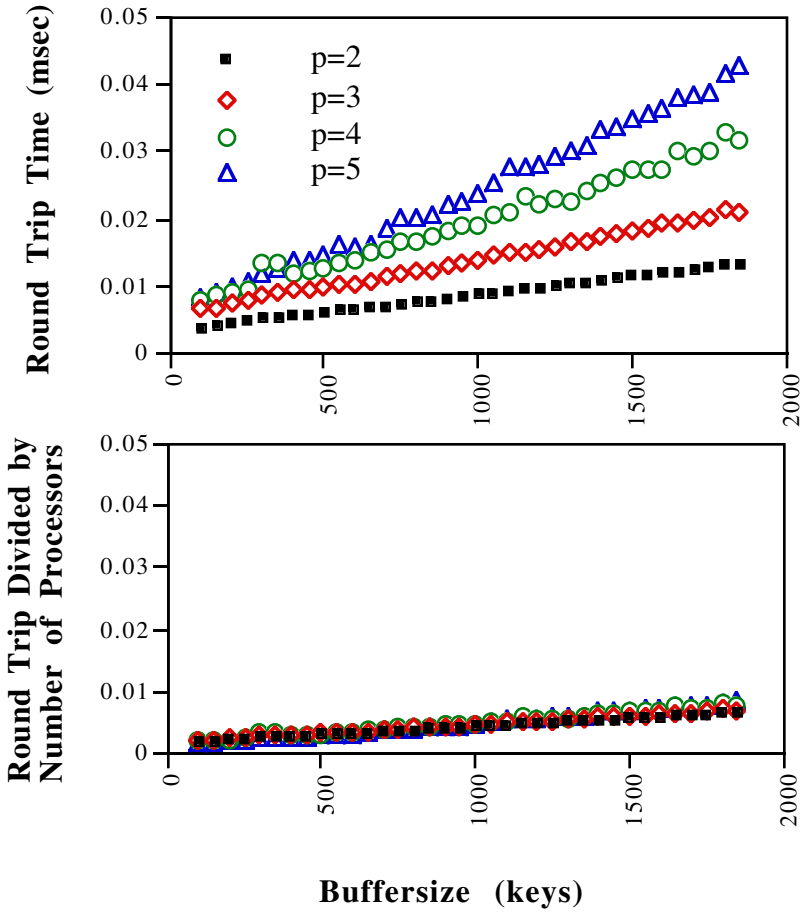


Figure 5.7: The communication benchmark - COMM(*p*, *b*)

This method of normalizing the transmission times, by dividing by p, was an unexpected result. We expected COMM(b, p) to be equal to the two processor case, COMM(b, 2), while within the overall bandwidth of the network. We expected the roundtrip times to increase only as throughput neared the bandwidth limit. The result showed roundtrip times slowing long before the bandwidth became a problem.

Further investigation revealed that processor contention caused this slowdown. Since Isis runs non-preemptable tasks, there was often a delay before a message could be delivered because the processor was servicing the other neighbor. The delayed response slows the computation in the neighbor processor, which in turn slows the next neighbor . Thus the delay accumulates down the line, resulting in a total delay of p transmission times (p/2 roundtrip times). Figure 5.7 demonstrates that this contention can be described sufficiently by

$$\text{COMM}(b, p) = p \cdot \frac{\text{COMM}(b,2)}{2}$$

within the range of our experiments. This model predicts performance well when processor contention is light. If the sorting is occurring in an environment with a heavy workload, Danzig's model of processing overhead [Dan91] might prove necessary to accurately describe the COMM() benchmark.

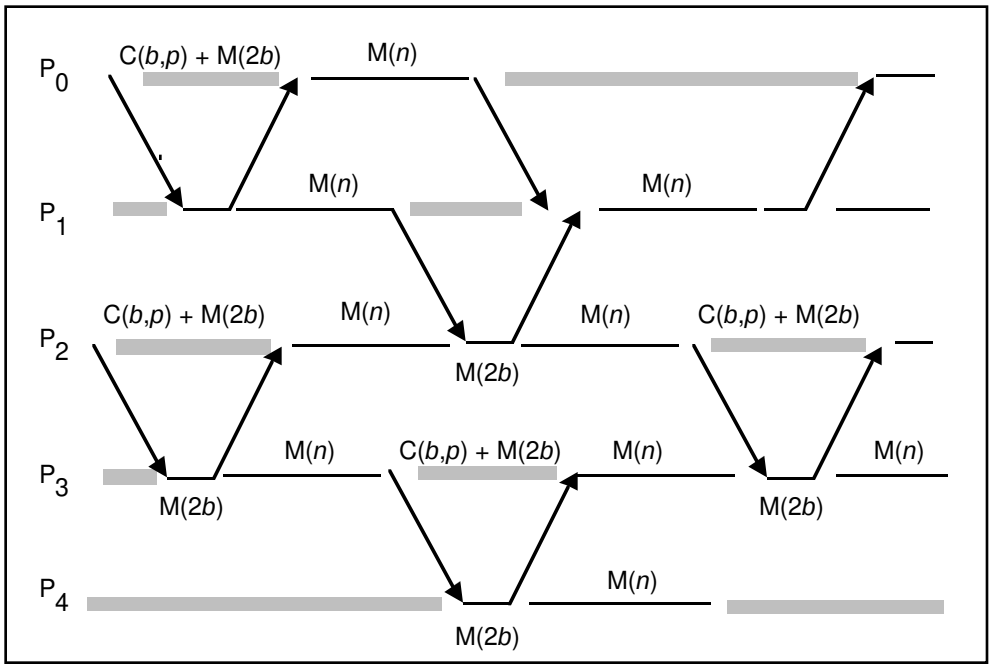


Figure 5.8: Odd-Even Sort timings

The solid lines represent communication (slanted) and computation (horizontal). The gray bars represent processor idle time. Where the length of an interval is known, its execution time is expressed in terms of the benchmark equations. The names of the benchmarks have been abbreviated to one letter for clarity.

5.3.2.4 STEP 4: blending

Now that we have validated models for all the benchmarks, we consider how to blend them to model the performance of the entire algorithms. Again, we start with the Odd-Even Sort. The naive composition of Odd-Even Sort would be the execution of the asynchronous phase, plus the total number of iterations in the synchronous phase (denoted steps) multiplied by the actions that happen once during each step (communication + the merge split-merge + the insertion sort). Algebraically, this would be (using the abbreviations S, C, and M for the benchmark functions):

$$S(n) + \text{steps} \cdot [C(b, p) + M(2b) + M(n)].$$

However, this equation does not take into account processor idle time or function overlap. To get a clear idea of idle time and overlap, we drew the timing chart in Figure 5.8.

Each line in Figure 5.8 represents the activity of a processor. The top and bottom processors are those configured to be at the end of line of communication in the algorithm, i.e. they only communicate with one neighbor. Note how in the first step, P_0 and P_2 send a block of keys to their right neighbor requesting a 2-way merge-split. The merge-split takes place (at a cost of MERGE($2b$)), and the lower b keys are returned. This whole operation takes time COMM(b, p) + MERGE($2b$). Next, all processors merge the result of the merge-split with their local lists with an insertion sort. This takes time MERGE(b + (n - b)) which is shown as MERGE(n) in the diagram. Now the whole process is repeated with the odd numbered processors initiating the merge-split.

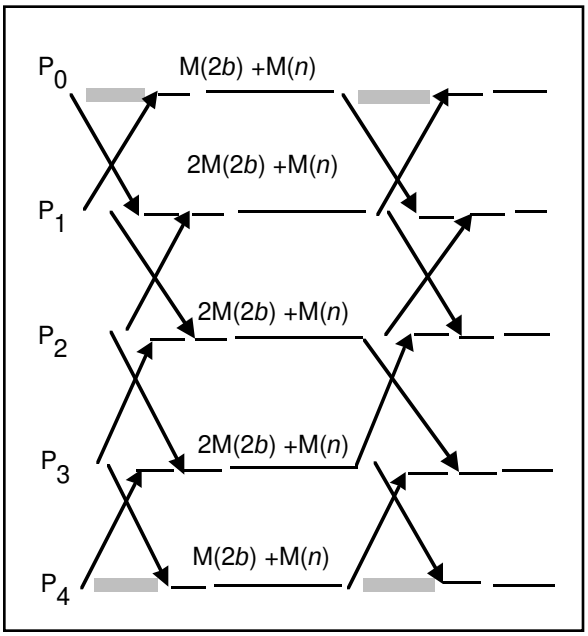


Figure 5.9: Neighbor Sort timings

To predict execution time, we are concerned only with the processors that take the longest to complete their tasks. In this case processors 1, 2, and 3 do more work than the end processors. Because of the alternating nature of Odd-Even Sort, it is easier to compute the execution time of a pair of steps instead of just one. Consider a pair of steps in processor 2. Each pair consists of a communication and merge split [COMM(b, p)+MERGE($2b$)], the computation end of a merge split [MERGE($2b$)], and two merges of the buffer with the local list [$2 \cdot$ MERGE(n)]. The two communication processes overlap and thus are only counted once. Therefore the composition that describes Odd-Even Sort is:

$$S(n) + \text{steps}/2 \cdot [C(b,\ p) + 2 \cdot M(2b) + 2 \cdot Mn)]$$

Now consider Neighbor Sort. Neighbor Sort (Figure 5.9) was designed to have minimal waiting periods by using an asynchronous communication style rather than the request-response style used in Odd-Even Sort. This doubles the amount of communication, but since communication overlaps with computation, the overall execution time of the algorithm is faster for large datasets. Each processor in this algorithm sends messages to both neighbors during each step as soon as the data is ready. Thus there is no significant idle time on the inside processors (1, 2, 3).

Again, we examine the inside processors (1, 2, and 3) since they will take the longest to complete their tasks. The cost of this algorithm is the asynchronous phase, plus the number of steps times the maximum of: a communication and a merge-split [COMM(b, p) + MERGE($2b$)], or a merge-split and an insertion sort [MERGE($2b$) + MERGE(n)]. All together, this is:

$$S(n) + \text{steps} \cdot [\text{Max}\ (2 \cdot M(2b) + C(b, p),\ 2 \cdot M(2b) + 2 \cdot M(n))]$$

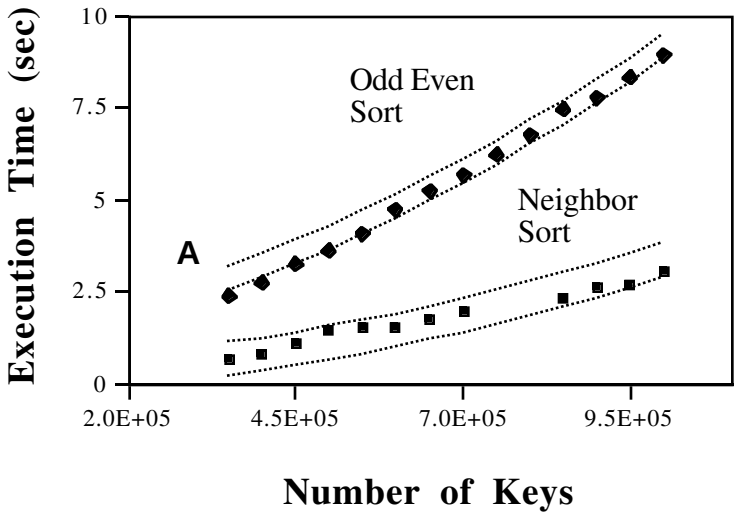


Figure 5.10: 95% Confidence intervals of the prediction

This graph shows that the model predictions for Neighbor Sort in the critical range of *N*. Points show measured values, and lines show the confidence intervals of the model predictions. These runs were made with *p*=5, *b*=1700, and *N* varying from 350,000 to 1,000,000 keys (*n*=70,000 to 200,000).

5.3.2.5 STEP 5: validation

Finally, we have complete models of both sorting programs. The last step is comparing the predictions of the models against actual implementations of the programs. The sorting programs were implemented as follows.

Each processor generated its local list of keys using a random number generator. The keys were evenly distributed long integers with some duplicates both locally and globally. Execution time was measured from the beginning to the end of the sort phase in each processor. These execution times were averaged to find the mean time for the algorithm. Some experiments were duplicated using the maximum of these execution times rather than the average; the results were not significantly different. Each run was repeated ten times with the same keys, and the results averaged to produce the graphs in Figures 5.10 and 5.11.

First, detailed studies were made with the input parameters set to critical values, i.e., those commonly seen in database systems. Figure 5.10 shows the performance of Neighbor Sort as the input list, *N*, was varied from 350,000 to 1,000,000 keys. For these runs, buffer size was set to 1700 keys (integers), and five processors were used. The lines on the graph represent the 95% confidence intervals (using the Standard Normal Table) for the predictions in this region. This means that the model predicts that 95% of program runs will have execution times that fall between the two lines. Figure 5.11 shows the execution time of Neighbor Sort for various buffer sizes with a fixed input of 100,000 keys per processor (n) using $p = 2$ and $p = 5$ processors. On this graph the actual prediction is shown rather than the confidence intervals for clarity. The execution times in these regions are predicted well by the model.

Next we look at the slopes and shapes of the lines. All the curves appear to be shaped correctly, i.e. the observed points map out a curve of the same shape as the prediction lines. The slopes are also very close. There are two regions where the prediction begins a trend away from

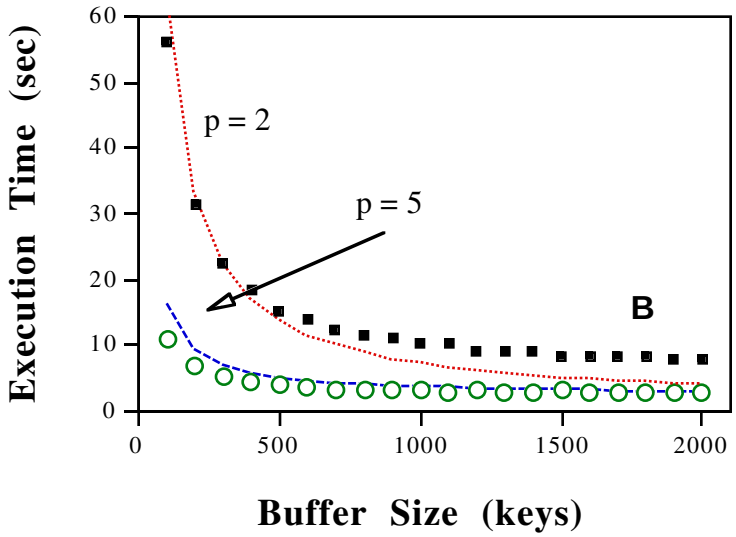


Figure 5.11: The effects of buffer size on Neighbor Sort

This graph shows the model predictions (lines) and measured values (points) for the range of buffer sizes. The top line was made with p=2 and N=200,000 (n=100,000). The bottom line was made with p=5 and N=500,000 (n=100,000).

the observed data. In region A on Figure 5.10, Odd-Even Sort runs faster than the prediction for the smaller dataset sizes, and in region B on Figure 5.11, Neighbor Sort runs slower than the prediction for the large buffer sizes when p = 2.

Region A is at the very edge of the validation area of the underlying MERGE() benchmark model. The number of keys per processor is below 100,000, whereas our approximation of the MERGE component was only calibrated from 100,000 to 1,000,000. This approximation is probably responsible for the inaccurate trend in region A. Fortunately, the model predicts well enough within the critical region that we may leave that complexity unexplored.

In region B, the buffer size is greater than the measured values of the COMM() benchmark (Figure 5.7). Also, p = 2 is actually a special case which should have entirely different blending functions since the processors communicate with only one neighbor. This would cause additional idle time, especially at large block sizes.

The trends away from the predicted value and the unexplored parameter regions define the region of accuracy of the model. Our model is accurate within the boundaries shown in Table 5.1.

	Low Value	High Value
Processors, p	2	5
Buffer Size, b p=2 p=3,4,5	 100 100	 800 2000
Input Size, N	350,000 · p	1,000,000 · p

Table 5.1: Validation boundaries

5.3.3 Using the model

5.3.3.1 Configuring the sort

To configure a sorting engine, the parameter space is searched to find the combination of settings that predicts the shortest execution time. These models are quite efficient for configuring sorting engines. As Figure 5.11 shows, the optimal buffer size does not vary much with input size or changes in the other parameters. Any buffer size greater than 1000 keys will give good performance in all situations. Therefore, the run-time adaptation of the sorting engine is reduced to choosing a sorting strategy and the number of processors to allocate. Since there are two strategies and only five processors, we can compute the execution time of all ten cases and pick the minimum.

With other models, this computation step might not be so limited. There may be more parameters involved, or some parameters that need to be searched with a finer resolution. Additional analysis may be necessary to limit the scope of the search and reduce computation time.

5.3.3.2 Gaining insight from the model

Analytic models are useful for gaining insight into the nature of a program, and for using that knowledge to improve performance by eliminating bottlenecks. There are two approaches to examining performance with these models. One approach analyzes with respect to the parameters of the model, and the other examines the contribution of each component. Because the parameters and components are very different things, it is useful to examine the algorithm both ways.

To examine the parameters, standard experimental design and analysis of variance may be used. A description of these methods is given by Jain in *The Art of Computer Systems Performance Analysis* [Jai91]. He describes various ways to set up the experiments. The simplest design is to vary one parameter at a time while holding the others constant. We did this in Figure 5.11 and discovered that increasing the buffer size beyond 1500 keys has no significant effect on execution time. Jain's methods all examine the variance of the execution time to determine the percentage of variance contributed by each parameter. These methods allow one to analyze the effect of individual parameters and their interaction with each other.

Evaluating the contribution of each component is a very different approach. The components are a combination of several (and maybe all) parameters in the model. Examining at this level is helpful for recognizing the faults of a particular algorithmic strategy, isolating resource problems, and predicting the effect of proposed system resources. The example below shows how the analysis of Odd-Even sort led us to the design of Neighbor Sort.

The blending equation for Odd-Even Sort is:

$$S(n) + \text{steps}/2\ [C(b, p) + 2 \cdot M(2b) + 2 \cdot M(n)]$$

We wish to break this equation down into three expressions that describe the contribution of each component to execution time. The contribution of SSORT is easy to isolate, but contributions of COMM and MERGE are slightly intermixed. Refer back to the timing diagram of Odd-Even sort in Figure 5.4. Notice the idle time a processor spends in the communication cycle includes

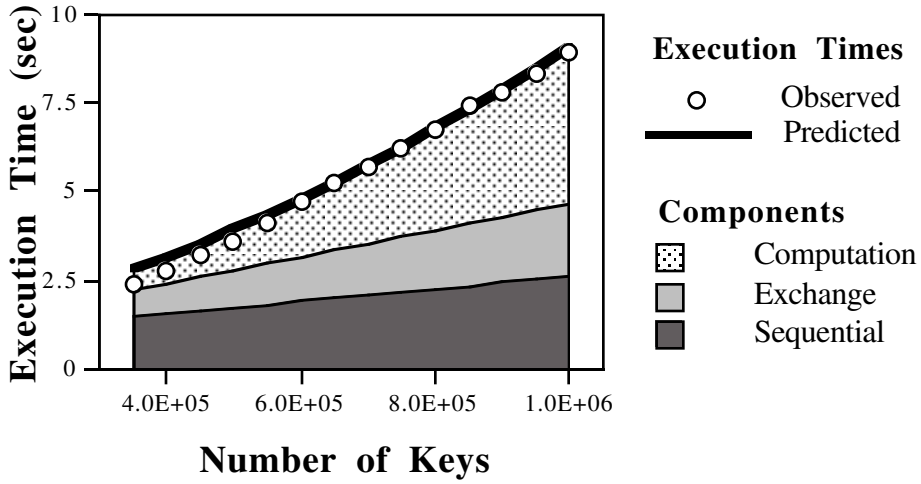


Figure 5.12: Breakdown of Odd-Even execution time by component

the time it takes its neighbor to do a merge-split. We want to include this small idle time as part of the cost of communication, therefore we broke the equation down into $S(n)$, $C(b, p) + M(2b)$, and $M(2b) + 2 \cdot M(n)$. For clarity, we give them different names: sequential, exchange, and computation.

Figure 5.12 shows the contribution of each these newly defined components. Note that as the list gets larger the computation factor is growing much faster than the exchange factor. Clearly, computation is the bottleneck in larger datasets. To take advantage of this, we designed Neighbor Sort which reduced idle time by doubling the amount of communication. Since the problem is compute-bound, Neighbor Sort performs better than Odd-Even sort for large datasets.

5.4 Conclusion

We have presented an approach to modeling called benchmark blending that can be used to create models that predict the execution time of distributed algorithms. The method is described in terms of steps, and the issues that need to be considered at each step are discussed. We demonstrated how to apply this method by building models of two sorting algorithms and validating their accuracy.

Benchmark blending produces models that are fast and accurate estimates of performance. The models are calibrated during their construction to be accurate on the specific target platform where they will be used. These models have the accuracy of fully analytic models, but are much easier to create since the benchmarks implicitly capture much of the complexity of the system.

A key attribute of benchmark blending is that the benchmarks implicitly account for the costs and benefits of operating system features (such as caching and paging) that are too complex to model analytically. This makes the resulting model easier to adapt to another environment or architecture, thus potentially enhancing its useful life-time.

A second attribute of models developed with benchmark blending is that they lend themselves to analysis by components. This leads to better algorithms, and better utilization of

the advanced subsystems available. Understanding the underlying system helps prevent the applications programmer from making decisions that poorly utilize system features.

We conclude that this is a good method to add to the performance toolbox. It is a simplified method for producing fast, accurate performance predictors that can be used at run-time to configure programs for improved performance.

Chapter 6

Conclusion

6.1 Synopsis

This thesis included three studies that examined and used time measurements in a distributed environment. The first study described a pervasive problem in workstation clocks: the tradeoff between synchronizing clocks and maintaining their stability. We demonstrated that measurements made with these clocks may be grossly inaccurate, and we presented methods for stabilizing clocks and for collecting data using unstable clocks.

The second study demonstrated the capabilities of asynchronous clocks when ordering events occurring on several processors, and measuring the intervals between those events. This study characterized the message-passing in applications that use the Isis Distributed Toolkit. A method for generating a global trace of events was developed, and the trace was used to get a system-wide view of network traffic.

The last study described how to weave time measurements with modeling techniques to build accurate analytic models of algorithm performance. It presented benchmark blending, a technique that provides fast and accurate performance predictions as an aid to run-time adaptation of distributed programs. We showed, through a case study, that models developed with benchmark blending implicitly account for many of the costs and benefits of operating system features (such as caching and paging) that are too complex to model analytically. Also, the input parameters of these models closely match the input parameters of the application; therefore, they provide relevant insight into the effects of configuration changes.

6.2 Contributions

This dissertation made the following contributions:

- Described the inherent tradeoff between clock synchronization and clock stability.
- Developed two statistical solutions to the problem of measuring intervals with an unstable clock.

- Developed a procedure for generating an accurate global trace of events using individual asynchronous clocks.
- Characterized message traffic in a distributed system and determined the arrival statistics to be similar to observations of packet traffic.
- Presented a technique for developing analytic models that quickly and accurately predict execution time, and implicitly incorporate much of the underlying system complexity.

6.3 Future work

6.3.1 Improving clocks and traces

GPS synchronized clocks

Tom Clark's Totally Accurate Clock [Cla95] project interfaces a Motorola Oncore Basic Global Positioning Service (GPS) receiver with a small home-built printed circuit board to create a clock with a tenth of a millisecond accuracy. This project suggests the feasibility of equipping all processors in a distributed system with a GPS receiver interfaced directly to the memory bus, and incorporated directly into the system clock with phase-locked-loop synchronization. This would automatically synchronize distributed processors without regard for network technology or geographic distance. A project like this would necessitate a collaborative approach combining hardware engineering with operating system design.

Characterizing clock error due to synchronization

Synchronization is an inherent source of instability in workstation clocks. Instability makes the clocks unsuitable for measuring precise intervals. Unfortunately, synchronization is necessary in today's distributed environment. One approach to this problem is to characterize the error in clock measurements, and apply statistical methods to compensate for that error when measuring intervals. The two methods described in Chapter 3 are gross applications of this concept. They greatly overestimate the usual amount of error in order to encompass the worst case. An accurate characterization of the clock error would facilitate development of finer techniques.

Global trace infrastructure

Stable and accurate unsynchronized clocks are available on many platforms. However, using them to build global time constructs, such as global traces or logs, is beyond the resources of the system analyst. We propose building an architecturally independent package that generates global traces in post processing from unsychronized distributed clocks. A package of this sort would provide critically needed infrastructure for the performance analyst involved in tuning client server systems [CMG93].

6.3.2 Message delivery latencies in Isis

The cause of message delivery latency in Isis applications

In our characterization of message traffic in Isis applications, we observed instances of delayed delivery. This latency could have several causes. Messages are delayed in order to provide causal ordering of delivery. They are also delayed while waiting for their "pop-up thread" to be activated by the thread scheduler in the receiving process.

The cost and benefits of causal ordering of messages has been hotly debated [CS93, Bir93]. Determining the contribution of causal ordering to this message delivery latency would resolve some of this debate, and thereby affect the designs of messaging systems.

If the thread scheduler contributes greatly to message delivery latency, then perhaps another scheduling policy would provide better performance. Isis currently has non-preemptive first-come, first-serve scheduling. A first take on this issue suggests that a preemptive scheduler would reduce latencies considerably.

Modeling message traffic with packet models

In our characterization of message arrival traffic, we found many attributes in common with previous observations of packet traffic. Several models of packet traffic exist (Poisson models, Self-Similar models, empirical models) [AMS82, HL86, LTWW93, DJ95]. We wonder which of these would accurately describe message traffic. Communication protocols are designed using models of packets rather than messages. If an accurate model of message traffic were used instead, it might result in better designs of the network- and session-level protocols.

The effects of traffic load on common communication patterns

We discovered in Chapter 5 that resource contention had a linear effect on RPC performance. This suggests that the bottleneck is operating system message overhead rather than network bandwidth. An analysis of a congested environment could separate and quantify the contribution of network traffic and processor contention to decreased performance. By using the global trace mechanism developed in Chapter 4, we would be able to trace messages through the protocol stack, over the network, and through destination processing. Using the traces we could determine where the performance loss is occurring and target that region for improvement.

6.3.3 Distributed sorting

Configuring distributed sorting engines

We used Benchmark Blending to develop a model that predicts performance of block-exchange sorting algorithms. This model can be used for adaptation of sorting engines in large relational database systems. Optimizing sorting in relational databases is actually a more complex issue; however, since the data are rarely distributed evenly across processors. Load balancing has a profound effect on sorting algorithms; redistribution of the data before and after sorting is effective in enhancing performance. One approach to this problem is to develop a tool that analytically combines the sorting model with data transfer costs, thereby providing a global model for configuring the sort. This tool would have to consider several different distributions

(or shapes [Mac87]) to determine which would most be cost effective for each algorithm and input size. A tool of this nature would find practical use in today's database systems.

References

[AGK94] Adelberg, B., Garcia-Molina, H. and B. Kao. Emulating Soft Real-Time Scheduling Using Traditional Operating System Schedulers. Technical Report, Stanford University, Dept. of Computer Science, STAN//CS-TN-94-8, August 1994.

[AMS82] Anick, D., Mitra, D. and M.M. Sondhi. Stochastic Theory of a Data-Handling System with Multiple Sources. *The Bell System Technical Journal 61,* (Oct) 1982, 1871-1894.

[AG88] Aral, Z. and L. Gertner. Non-Intrusive and Interactive Profiling in Parasight. In *Conference on Parallel Programming: Experiences with Applications, Languages, and Systems*, pages 21-30. September 1988.

[Ash95] Ashton, P. Algorithms for Off-line Clock Synchronisation. Technical Report, Department of Computer Science, University of Canterbury, TR-COSC 12/95, December 1995.

[Bah95] Bahder, T.B. *Mathematica for Sicentists and Engineers.* Addison-Wesley, New York, NY, 1995.

[BHKSO92] Baker, M.G., Hartman, J.H, Kupfer, M.D., Shirriff, K.W. and J.K. Ousterhout. Measurements of a Distributed File System. In *Proceedings 12th ACM Symposium on Operating Principles,* pages 198-212. ACM (Dec) 1989.

[Bat68] Batcher, K.E. Sorting Networks and their Applications. In *Proceedings of the 1968 Spring Joint Computer Conference, vol. 32,* pages 307-314. AFIPS Press, 1968.

[BS78] Baudet, G., and D. Stevenson. Optimal Sorting Algorithms for Parallel Computers. *IEEE Transactions on Computing C-27,* 1 (Jan) 1978.

[Bea95] Beadles, J.T. GPS for Time and Frequency. Available from http://galaxy.einet.net/editors/john-beadles/sum_frq.htm#gpstime, 1995.

[BRSS92] Bentley, R., Rodden, T., Sawyer, P. and I. Sommeville. An Architecture for Tailoring Cooperative Multi-user Displays. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW'92),* pages 187-194. ACM PRESS, 1992.

[BALL90] Bershad, B.N., Anderson, T.E., Lazowska, E.D. and H.M. Levy. Lightweight Remote Procedure Call, *ACM Transactions on Computer Systems 8,* 1 (Feb) 1990, 7-55.

[Bir93] Birman, K. A Response to Cheriton and Skeen's Criticism of Causal and Totally Ordered Communication. *ACM SIGOPS 28,* 1(Jan) 1994.

[BJ87] Birman, K.P. and T.A. Joseph,. Reliable Communication in the Presence of Failures. *ACM Transactions on Computer Systems 5,*1 (Feb) 1987, 47-76.

[BSS90] Birman, K.P, Schiper, A. and P. Stephenson. Fast Causal Multicast. Technical Report, Department of Computer Science, Cornell University, Ithaca, NY. TR90-1105, 1990.

[Bir85] Birrell, A.D. Secure Communication Using Remote Procedure Calls, *ACM Transactions on Computer Systems 3,* 1 (Feb) 1985, 1-14.

[BN84] Birrell, A.D. and B. J. Nelson. Implementing Remote Procedure Calls, *ACM Transactions on Computer Systems 2,* 1 (Feb) 1984, 39-59.

[BBDW83] Bitton, D., Boral, H., Dewitt, D.J. and W.K. Wilkinson. Parallel Algorithms for the Execution of Relational Database Operations. *ACM Transactions on Database Systems 8,* 3 (Sep) 1983, 324-353.

[BDHM84] Bitton, D., DeWitt, D.J., Hsiao, D.K. and J.A. Menon. A Taxonomy of Parallel Sorting. *ACM Computing Surveys 16,* 3 (Sep) 1984, 287-318.

[BHH78] Box, G.E.P., Hunter, W.G. and S. Hunter. *Statistics for Experimenters: An Introduction to Design, Data Analysis, and Model Building.* Wiley Series in Probablility and Mathematical Statistics. John Wiley and Sons, Inc., NY, 1978.

[CDJM91] Caceres, R., Danzig, P.B., Jamin, S. and D.J. Mitzel. Characteristics of Wide-area TCP/IP Conversations. In *Proceedings of ACM SIGCOMM '91.* ACM PRESS, (Nov) 1991.

[CP93] Chen, P.M. and D.A. Patterson. A New Approach to I/O Performance Evaluation – Self-Scaling I/O Benchmarks, Predicted I/O Performance. In *ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems,* pages 1-13. ACM PRESS, (May) 1993.

[Che84] Cheriton, D.R. An Experiment Using Registers for Message Based Interprocess Communication. *ACM Operating Systems Review,* (Oct) 1984, 12-20.

[CS93] Cheriton, D.R. and D. Skeen. Understanding the Limitations of Causally and Totally Ordered Communication. In *Proceedings of The 14th ACM Symposium on Operating Systems Principals,* pages 44-57. ACM Press, 1993.

[CW87] Cheriton, D.R. and C.L. Williamson. Network Measurement of the VMTP Request-Response Protocol in the V Distributed System. In *1987 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, pages 216-225. ACM Press, 1987.

[CZ85] Cheriton, D.R. and W. Zwaenepoel. Distributed Process Groups in the V Kernel. *ACM Transactions on Computer Systems 3,* 2 (May) 1985, 77-107.

[Cla95] Clark, T. Totally Accurate Clock (TAC). Available from ftp://aleph.gsfc.nasa.gov/GPS/totally.accurate.clock/announce.tac, 1995.

[CQ93] Clement, J.J. and M.J. Quinn. Analytical Performance Prediction on Multicomputers. In *Proceedings of Supercomputing '93*, pages 886-894. November 1993.

[CMG93] CMG Open Systems Working Group. Requirements for Planning, Measuring, and Reporting Open and Distributed System Performance. *CMG Transactions* 80. CMG (Summer) 1993, 5-18.

[CL93] Crovella, M.E. and T.J. LeBlanc. The Search for Lost Cycles: A New Approach to Parallel Program Performance Evaluation. Technical Report, Computer Science Department, University of Rochester, Number 479, December 1993.

[Dan91] Danzig, P.B. An Analytical Model of Operating System Protocol Processing Including Effects of Multiprogramming. In *Proceedings of ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, pages 11-20. ACM Press, 1991.

[DJ91] Danzig, P.B. and S. Jamin. Tcplib: A Library of TCP Internetwork Traffic Characteristics. Technical Report, Computer Science Department, University of Southern California, CS-SYS-91-01, 1991.

[DM88] Danzig, P.B. and S. Melvin. High Resolution Timing with Low Resolution Clocks and a Microsecond Timer for Sun Workstations. *ACM Operating Systems Review 24,* 1 (Jan) 1988, 23-26.

[DE95] Dietz, M.A and C.S. Ellis. A Characterization of Communication Patterns in Isis Distributed Applications. Technical Report, Department of Computer Science, Duke University, CS-1994-05, February 1995.

[DES94] Dietz, M.A., Ellis, C.S. and C. F. Starmer. Clock Instability and its Effect on Time Intervals in Performance Studies. In *Computer Measurement Group 1995 (CMG95) Proceedings,* pages 439-448. CMG, (Dec) 1995.

[DEC89] Digital Time Service Functional Specification version T.1.0.5. Digital Equipment Corporation, 1989

[DHHB87] Duda, A., Harrus, G., Haddad, Y. and G. Bernard. Estimating Global Time in Distributed Systems. In *Proceedings of 7th International Conference on Distributed Computing Systems,* pages 299-306. September 987.

[DMRW94] Duffy, D., McIntosh, A., Rosenstein, M. and W. Willinger. Statistical Analysis of CCSN/SS7 Traffic data from Working CCS Subnetworks. *IEEE Journal of Selected Areas in Communication 12*, 3 (Apr) 1994, 544-551.

[EGLT76] Eswaran, K.P., Gray, J.N., Lorie, R.A. and I.L. Traiger. The Notions of Consistency and Predicate Locks in a Database System. *Communications of the ACM 19,* 11 (Nov) 1976, 624-633.

[FE89] Floyd, R. and C. Ellis. Directory Reference Patterns in Hierarchical File Systems. *IEEE Transactions on Knowledge and Data Engineering 1,* 2 (Jun) 1989.

[Fel88] Feldmeier, D. Improving Gateway Performance with a Routing-Table Cache. In *Proceedings IEEE INFOCOM '88.* IEEE, March 1988.

[Fel94] Feldmeier, D. Personal communication. SIGCOMM in London, UK, September 1994.

[Fid88] Fidge, C. Timestamps in Message-Passing Systems that Preserve the Partial Ordering. In *Proceedings of the 11th Australian Computer Science Conference*. University of Queensland, Feb 1988.

[FJLOSW88] Fox, G., Johnson, M., Lysenga, G., Otto, S., Salmon, J. and D. Walker. *Solving Problems on Concurrent Processors (Vol. 1),* Prentice-Hall, Englewood, NJ, 1988.

[GW94] Garrett, M. and W. Willinger. Analysis, Modeling and Generation of Self-Similar VBR Video Traffic. In *Proceedings of ACM SIGCOMM,* pages 269-280. ACM PRESS, (Sep) 1994.

[GBDJMS94] Geist, G.A, Beguelin, A., Dongarra, J.J, Jiang, W., Mancheck, R. and V. S. Sunderam. *PVM: Parallel Virtual Machine. A User's Guide and Tutorial for Networked Parallel Computing.* MIT Press, 1994.

[GBCvR92] Glade, B.B., Birman, K.P., Cooper, R.C. and R. van Renesse. Light-Weight Process Groups. In *Proceedings of the Open Forum '92 Technical Conference,* pages 323-336. Utrecht, The Netherlands, (Nov) 1992.

[GH93] Goldberg, A.J. and J.L. Hennessy. Mtool: An Integrated System for Performance Debugging Shared Memory Multiprocessor Applications. *Parallel and Distributed Systems 4,* 1 (Jan) 1993, 28-40.

[Gra80] Gray, J. A Transaction Model. IBM Research Report. IBM Research Lab., San Jose, Calif., RJ2895, Aug. 1980.

[Har90] Harbitter, A. Hybrid Techniques in Benchmarking: A Scalable Approach. In *Computer Measurement Group 1990 Proceedings (CMG90) Proceedings,* pages 887-894. CMG, 1990.

[HL86] Heffes, H., and D.M. Lucantoni. A Markov Modulated Characterization of Packetized Voice and Data Traffic and Related Statistical Multiplexer Performance. *IEEE Journal on Selected Areas in Communications 4,* (Sep) 1986, 856-868.

[HC75] Hellerman, H. and T.F. Conroy. *Computer System Performance.* McGraw-Hill, New York, NY, 1975.

[HIM91] Hollingsworth, J.K., Irvin, R.B. and B.P. Miller. The Integration of Application and System Based Metrics. In *Proceedings of the 3rd ACM SIGPLAN Symposium on Principals and Practice of Parallel Programming,* pages 206-217. ACM Press, April, 1991.

[IBM90] In IBM*AIX Version 3 for RISC System/6000.* IBM (1990), Vol 1, p. 220.

[ISIS91] The ISIS Group. *The ISIS Distributed Toolkit Version 3.0 User Reference Manual.* Department of Computer Science, Cornell University, 3.0 edition, May 1991.

[Jai91] Jain, R. *The Art of Computer Systems Performance Analysis.* John Wiley & Son, Inc. New York, 1991.

[JR86] Jain, R. and S. Routhier. Packet Trains - Measurements and a New Model for Computer Network Traffic. *IEEE Journal of Selected Areas in Communication 4,* 6 (Sep) 1986, 986-995.

[Joh95] Johnson, M.W. Measuring the Response Time of Distributed Applications. In *Computer Measurement Group 1995 Proceedings (CMG95) Proceedings,* pages 853-864. CMG, (Dec) 1995.

[KG94] Kao, B. and H. Garcia-Molina. Subtask Deadline Assignment for Complex Distributed Soft Real-Time Tasks. *IEEE Transactions on Parallel and Distributed Systems,* 1994, 172-181.

[Kar89] Karger, P.A. Using Registers to Optimize Cross Domain Call Performance. In *Proceedings of 3rd Conference on Architectural Support for Programming Languages and Operating Systems,* pages 113-126. April, 1992.

[NTPD94] A Kernel Model for Precision Timekeeping. XNTP-3.4i Distribution. Available from ftp://louie.udel.edu/pub/ntp/xntp-3.4i.tar.Z, 1996.

[Lam78] Lamport, L. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM 21,* 7 (Jul) 1978, 558-565.

[Lam81] Lampson, B.W. Atomic Transactions. In *Distributed Systems - Architecture and Implementation: An Advanced Course.* G. Goos and J. Hartmanis (Eds.), Springer-Verlag, New York, 1981, Ch. 11, pp. 246-265.

[LK93] Lee, E.K. and R.H. Katz. An Analytic Performance Model of Disk Arrays. In *ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems,* pages 98-109. ACM PRESS, (May) 1993.

[LTWW93] Leland, W., Taqqu, M., Willinger, W. and D. Wilson. On the Self-Similar Nature of Ethernet Traffic (Extened Version). *IEEE/ACM Transactions on Networking,* (Feb) 1994, 1-15.

[Lie83] Liedtke, J. Improving IPC by Kernel Design. In *Proceedings 12th ACM Symposium on Operating Principles,* pages 175-187. ACM PRESS, (Dec) 1993.

[LT91] Lin, T. and T. Wernhuar. Scheduling Periodic and Aperiodic Tasks in hard Real-Time Computing Systems. In *1991 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems* , pages 31-38. ACM Press, 1991.

[LLM88] Litzkow, M.J., Livny, J. and M. W. Mutka. Condor: A Hunter of Idle Workstations. In *Proceedings of the 8th International Conference of Distributed Computing Systems,* pages 104-111. San Jose, CA, (Jun) 1988.

[LCW94] Lucantoni, D.M., Choudhury, G.L., and Whitt, W. The Transient BMAP/G/1 Queue. *Stochastic Models, 10*(1), 1994

[Mac87] Mace, M.E. *Memory Storage Patterns in Parallel Processing.* Kluwer Academic Publishers, Norwell, MA, 1987.

[Man83] Mandelbrot, B.B. *The Fractal Geometry of Nature.* Freeman, New York, 1983.

[Mar84] Marzullo, K. Maintaining the Time in a Distributed System. Ph.D. Thesis, Stanford University, Department of Electrical Engineering, June 1984.

[MO85] Marzullo, K. and S. Owicki. Maintaining the Time in a Distributed System. *ACM Operating Systems Review 19,* 3 (Jul) 1985, 44-45.

[MAT95] The Mathworks, Inc. *The Student Edition of MATLAB.* Prentice Hall, Englewood Cliffs, NJ, 1995.

[Mat89] Mattern, F. Time and Global States of Distributed Systems. In *Proceedings of the Internatioal Workshop on Parallel and Distributed Agorithms*. Bonas, France, (Oct) 1988. North-Holland 1989.

[MDRG95] Menasce, D.A., Dregits, D., Rossin, R. and D. Gantz. A Federation-oriented Capacity Management Methodology for LAN Environments, *Computer Measurement Group 1995 (CMG95) Proceedings,* pages 1024-1035. CMG, (Dec) 1995.

[Mil83] Mills, D.L. Algorithms for Synchronizing Network Clocks. DARPA Network Working Group Report, M/A-COM Linkabit, RFC-956, September, 1985.

[Mil89] Mills, D.L. Internet Time Synchronization: the Network Time Protocol. DARPA Network Working Group Report, University of Delaware, RFC-1129, October 1989.

[Mil90] Mills, D.L. On the Accuracy and Stability of Clocks Synchronized by the Network Time Protocol in the Internet System. *ACM Computer Communication Review 20,* 1 (Jan) 1990, 65-75.

[Mil91] Mills, D.L. Internet Time Synchronization: the Network Time Protocol. *IEEE Transactions on Communications COM-39,* 10 (Oct) 1991, 1482-1493.

[Mil92] Mills, D.L. A Computer–controlled Loran–C Receiver for Precision Timekeeping. Electrical Engineering Department Report, University of Delaware, 92-3-1, March 1992.

[Mil94] Mills, D.L. Improved Algorithms for Synchronizing Computer Network Clocks. *Computer Communications Review 24,* 4 (Oct) 1994, 317-327.

[Mog90] Mogul, J.C. Network Locality at the Scale of Processes. *ACM Transactions on Computer Systems 10,* 2 (May) 1992, 81-109.

[MLP87] Mohamed Salehmohamed, Luk, Ws. and J.G. Peters. Performance Evaluation fo LAN Sorting Algorithms, In *Proceedings of the 1987 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems,* pages 226-232. ACM Press, 1987.

[Mul84] Mullender, S.J., Et. al. *The Amoeba Distributed Operating System: Selected Papers 1984-1987.* Centrum voor Wiskunde en Informatica, Amsterdam 1987.

[Nel81] Nelson, B. Remote Procedure Call, Ph.D. Thesis, Carnegie Mellon University, 1981.

[NT88] Nelson, R. and A.N. Tantawi. Approximate Analysis of Fork/Join Synchronization in Parallel Queues. *IEEE Trans. on Computers 37*, (Jun) 1988, 739-743.

[New94] Newman, P. Traffic Management for ATM Local Area Networks, *IEEE Communications Magazine,* (Aug) 1994, 44-50.

[NPACKZZ93] Nudd, G.R., Papaefstathiou, E., Papay, Y., Atherton, T.J., Clarke, C.T., Kerbyson, D.J., Stratton, A.F., Ziani, R. and M.J. Zemerly,. A Layered Approach to the Charaterisation of Parallel Systems for Performance Prediction. In *Performance Evaluation of Parallel Systems (PEPS) '93,* pages 26-34, U. Warwick, UK, November 1993.

[PF94] Paxson, V. and S. Floyd. Wide-Area Traffic: The Failure of the Poisson Modeling. In *Proceedings of ACM SIGCOMM,* pages 257-268. ACM PRESS, (Sep) 1994.

[PB95] Pendergrass, B.H. and R.N. Bonham. Distributed System Analysis Case Study. *Computer Measurement Group 1995 (CMG95) Proceedings,* pages 1124-1129. CMG (Dec) 1995.

[Pos83] Postel, J. Daytime Protocol. DARPA Network Working Group Report, USC Information Sciences Institute, RFC-868, May 1983.

[PA95] PreVue Specification Sheet. Performance Awareness, Corp. Available from http://www.PACorp.com/.

[vRBCGS92] van Renesse, R., Birman, K., Cooper, R., Glade, B. and P. Stephenson. The Horus System. Technical Report, Dept. of Computer Science, Cornell University, July 1993.

[vRvST88] van Renesse, R., van Staveran, H., and A.S. Tanenbaum. Performance of the World's Fastest Distributed Operating System. *Operating Systems Review,* (Oct) 1988, 25-34.

[RF94] Romanow, A. and S. Floyd. Dynamics of TCP Traffic over ATM Networks. In *Proceedings of ACM SIGCOMM,* pages 79-88. ACM PRESS, (Sep) 1994.

[Ros95] Rosenberg, JL. A Delicate Balance: The Capacity-Performance Symbiosis. In *Computer Measurement Group 1995 (CMG95) Proceedings,* pages 85-93. CMG, (Dec) 1995.

[Ros85] Ross, S.M. *Introduction to Probability Models.* Academic Press, Inc. New York, 1985.

[STP95] Sahner, R.A., Trivedi, K.S., and Puliafito, A. Performance and Reliability Analysis of Computer Systems: an ExampleBased Approach using the SHARPE Software Package, Kluwer Academic Publishers, 1995.

[SES89] Schiper, A., Eggli, J. and A. Sandoz. A New Algorithm to Implement Causal Ordering. In *Proceedings of the 3rd International Workshop on Distributed Algorithms, Lecture Notes on Computer Science 392,* pages 219-232. Springer-Verlag, 1989.

[SB89] Schroeder, M.D. and M. Burroughs. Performance of Firefly RPC. In *Proceedings 12th ACM Symposium on Operating Principles,* pages 83-90. ACM PRESS, (Dec) 1989.

[Sch92] Schwartz, J.A. Techniques for Developing a Validated Analytical Model. In *Computer Measurement Group 1992 (CMG92) Proceedings,* pages 354-365. CMG, (Dec) 1992.

[Sha93] Shackelford, D.E. The Distributed Graph Storage System: A User's Manual for Application Programmers. Technical Report, Dept. of Computer Science, Univ. of North Carolina, January 1993, TR93-003.

[SSS92] Shackelford, D.E., Smith, J.B. and F. D. Smith. A Distributed Data-Storage Service for Supporting Group Collaborations. Technical Report, Dept. of Computer Science, University of North Carolina, TR92-044, October, 1992.

[Str93] Stratus Computer, Inc. (NYSE:SRA) Stratus Computer targets Distributed Computing Market with Acquisition of ISIS Systems. Press release, Marlboro, Mass., December 7, 1993.

[Su81] Su, Z. A Specification of the Internet Protocol (IP) Timestamp Option. DARPA Network Working Group Report. SRI International, RFC-781, May 1981.

[Sun93] In *SunOS reference Manual,* Sun Microsystems, Inc. (1990), p. 760.

[Sun93a] SunOS Version 4.1.3 Source Code, 1993.

[Tan92] Tanenbaum, A.S. *Modern Operating Systems.* Prentice Hall, Englewood Cliffs, NJ, 1992.

[TvR88] Tanenbaum, A.S. and R. van Renesse. A Critique of the Remote Procedure Call Paradigm. In *Proceedings of the EUTECO '88 Conference,* pages 775-783. R. Speth, editor. Vienna, Austria, April 1988.

[TA90] Tay, B.H. and A. L. Ananda. A Survey of Remote Procedure Calls, *Operating System Review 24,* 3 (Jul) 1990, 68-76.

[NBS79] Time and Frequency Dissemination Services. NBS Special Publication, U.S. Department of Commerce, 432, 1979.

[Tri82] Trivedi, K.S. *Probability & Statistics wth Reliability, Queuing, and Computer Science Aplications.* Prentice-Hall, Englewood Cliffs, NJ, 1988.

[Ult95] UltraFit Demo Disk. Available from Martin Pion (mpion@peach.wustl.edu) , Biosoft, PO Box 10938, Ferguson, MO 63135.

[Vau91] Vaughan, M.S. A Distributed Approach to Human Genetic Linkage Analysis. Master's Thesis, Department of Computer Science, Duke University, May, 1991.

[VSSG88] Vrsalovic, D., Siewiorek, D.P., Segal, Z.Z., and E.F. Gehringer. Performance Prediction and Calibration for a class of Multiprocessor Systems. *IEEE Transactions On Computers 37*, 11 (Nov) 1988, 1353-1365.

[WS91] Wall, L. and R.L. Schwartz. *Programming Perl.* O'Reilly and Associates, Sebastopol, CA, 1991.

[Wan95] Wang, Chang-Yu. Some Problems in the Specification and Analysis of Computers and Networks. PhD Thesis, Department of Computer Science, Duke University, December 1995.

[Wel86] Welch, B.B. The Sprite Remote Procedure Call System. Technical Report, The University of California, Berkeley, CA, No. UCB/CSD 86/302, June 1986.

[Wil89] Williamson, C.L. Dynamic Transport-Level Connection Management in a Distributed System. In *Proceedings of the 14th Conference on Local Computer Networks,* pages 315-322. October, 1989.

[YIGBF93] Yakowendo, B., Illiano, V., George, C. Boes, and K. Furst, J. Distributed Genetic Mapping. Dlinkmap User Manual. Department of Computer Science, University of North Carolina, April 1993.

Biography

Water plays a dominant role in life of Margaret (Margie) Dietz. Margie abruptly left the water-based environment of the womb on February 13, 1957, in Abington, PA, USA. She grew up in a mixed city and country environment with winters spent in a Philly suburb (city pools), and summers at her extended family's home in the Pocono Mountains (trout streams, mountain lakes). The family traveled a lot, the most memorable places being Ocean City, NJ (salt, waves!), Ocala State Park, FL (free-diving in a sink-hole!), and Arcadia National Park, ME (tidal pools!).

After graduating from Hatboro Horsham High School, in Horsham, PA, in 1975, Margie Dietz moved on to Lebanon Valley College, Anneville, Pennsylvania, where she won the Max F. Lehman Prize in Mathematics, and earned a B.S. in Mathematics and Physics in 1979. Memorable water events during this period included the annual tug-of-war across the muddy Quitta-Pahilla creek, and a flood that covered the entire campus with 18 inches of water.

After college Margie worked as an electronics technician in a cardiovascular research laboratory at Hershey Medical Center, Hershey, PA. This position included computer system administration and construction of new equipment that interfaced computers with analog and video equipment. Unfortunately, though rich in chocolate, Hershey is not near any large bodies of water. Margie had to make due with playing with the garden sprinkler, and splashing through every available mud puddle (luckily it rained a lot). It was about this time that Margie realized a very deep-seated need for a 4-wheel drive vehicle.

In 1982, Margie did a short stint in industry at ITT Terryphone in Harrisburg, PA, and then moved to Durham, NC, to pursue a master's degree in Computer Science at Duke University. As a poor student with no air conditioning in this hot environment, Margie swam almost daily in the Eno River (swimming prohibited), and had several run-ins (or rather run-aways!) with sheriff's deputies at local quarries.

After receiving her A.M. in 1984, Margie took the position of Engineering Scientist in the Department of Medicine at Duke University. In this position she collaborated with investigators in a variety of medical sciences on projects including: DNA sequence matching, design of computer-aided tools for measuring heart mechanics, design of experimental control systems, and analytic modeling of myocardial tissue structure. In her free time she explored local lakes in a 12-foot sailing boat. In 1988 she took a scuba diving course at Water World, Inc., and had finally found a way back into a total water environment!

Margie became an avid scuba diver and had many encounters with sharks, turtles, stingrays, eels, whales, manatees, and dolphins. She bought an underwater video camera in 1991, and began documenting these experiences. Most of them occurred while diving on shipswrecks off the North Carolina shore. Margie has contributed to several video productions, and her work has been instrumental in designing displays at the North Carolina Aquarium. She is also an active leader in the Triangle Dive Club.

In February and March of 1990 Margie went to Cairo, Egypt, as a consultant for the US National Institutes of Health, to design a multiprocessor database program for a genetics clinic. This culminated in a five day diving excursion in the Red Sea (alligator fish, 6' napolean wrasses, white-tipped reef sharks).

In 1991, Margie returned to school full-time to work toward a PhD in Computer Science. In July, 1993, Margie became both a scuba instructor and a college instructor with her first courses being Open Water Diving at Metrosport Athletic Club, and Introduction to Pascal at Duke University. This was the beginning of a deep love for teaching.

Finally, after getting an above ground pool at her home, Margie was able to spend less time traveling to water and more time on Computer Science. She has received three achievement awards for her recent work:

- 1994 ACM Travel Grant for SIGCOMM'94 in London.
- 1995 J. William Mullen Award (Computer Measurement Group)
- 1995 Best Paper Award, Computer Measurement Group National Conference

Margie has many publications from her work at Hershey and Duke in both Medicine and Computer Science. The good ones are listed here. A complete list is available by writing Margie at her (hopefully permanent) email address, **dietz@cs.duke.edu**.

- Dietz, M. A., C. S. Ellis, and C. F. Starmer. Clock Instability and its Effect on Time Intervals in Performance Studies. *CMG95 Proceedings,* December 1995, pp. 439-448. [Best Paper Award]

- Dietz, M.A. and C. S. Ellis. A Characterization of Communication Patterns in Isis Distributed Applications. Technical Report, Department of Computer Science, Duke University, February 1995, CS-1994-05.

- Dietz, M. A. and Starmer, C. F. *An Object Oriented User Interface for Analysis of Biological Data.* Computers and Biomedical Research 23:82-96, 1990.

- Starmer, C. F., Cherveny, D. J., Dietz, M. A. and Smaltz, J. M. *Managing Research Data with Self-documenting Files.* Computers and Biomedical Research 20:264-278, 1987.

- Starmer, C. F., Dietz, M. A. and Grant, A. O. *Signal Discretization: A Source of Error in Histograms of Ion Channel Events.* IEEE Trans. on Biomed. Eng. BME-33(1):70-73, 1986.

- Grant, A., Dietz, M. and Starmer, F. Voltage-dependent block of single cardiac sodium channels by disopyramide. Circulation 80:II-136, 1989.

- Grant, A. O., Dietz, M. A., Gilliam, F. R. and Starmer, C. F. Blockade of Cardiac Sodium Channels by Lidocaine: Single Channel Analysis. Circulation Research, 65:1247-1262, 1989.

- Hurwitz, J. L., Starmer, C. F., Dietz, M. A. and Grant, A. O. Sodium Channel Inactivation from Closed States. Circulation 74(II), p. II-19, October, 1986.